Engineering Specification

PART NAME	PART NUMBER
Global Diagnostic Specification	DS-YF1T-1A294-AB

LET																		
FR																		
LET																		
FR																		

	LET	FR	REVISIONS	DR	CK	REFERENCE
						PREPARED/APPROVED BY
						Steve DiLodovico
						CHECKED BY: WWDiag; DETAILED BY:
						CONCURRENCE/APPROVAL SIGNATURES
						Design Engineering Supervisor
						John Markee
						Design Engineering Management
						Prabhaker Patil

FRAME		OF		REV4.2	
PD May 1988	3947a1e	(Previous editions may **NOT** be used)			

Ford Automotive Operations

GLOBAL DIAGNOSTIC SPECIFICATION

(Part One)

Worldwide Requirements

(FOR MY 2000 AND BEYOND)

PART NUMBER
DS-YF1T-1A294-AB

Version 4.2

23 December 1997

Advanced Vehicle Technology

Electrical/Electronic Systems Engineering

Worldwide Diagnostic Organization

Ford Motor Company

FOREWORD

This document contains information that is proprietary to the Ford Motor Company. Recipients of this document shall not duplicate, use or disclose information contained herein, in whole or in part, for other than company purposes.

Distribution	Date Issued	Ver.	WERS #	Date Issued	Written By:	PROFS	Prepared By:
Original	30 January 1992	1.0	*not issued*	n/a	Ron Brombach x05565	RBROMBAC	Joseph G. Toth
Second Edition	27 March 1992	1.1	*not issued*	n/a	Ron Brombach x05565	RBROMBAC	Joseph G. Toth
Third Edition	19 June 1992	1.2	*not issued*	n/a	Ron Brombach x05565	RBROMBAC	Joseph G. Toth
REVISION	14 October 1993	2.0	*not issued*	15 October 1993	Greg A. Maida 59-46612	GMAIDA	****
Fifth Edition	12 January 1994	2.1	A10343301	12 January 1994	Greg A. Maida 59-46612	GMAIDA	Joseph G. Toth
Sixth Edition	27 March 1995	2.2	*preliminary*	n/a	Greg A. Maida 59-46612 Adam Witchell 110-44-1268-404706	GMAIDA AWITCHEL	****
REVISION	15 May 1995	3.0	A10518832	xx May 1995	Greg A. Maida 59-46612	GMAIDA	Joseph G. Toth
REVISION	14 August 1996	4.0	EE00E10684653 001	14 August 1996	Greg A. Maida 59-46612	GMAIDA	Joseph G. Toth
Seventh Edition	9 June 1997	4.1	C10749874	13 June 1997	Gary Rushton	GRUSHTON	Sonia Singh
Revision	15 December 1997	4.2			Steve DiLodovico 24-88815	SDILODOV	Steve DiLodovico

PLEASE READ

This is a dynamic document within FORD Motor Company that will be periodically updated as required to make recommended and/or necessary improvements. Any recommendations for improving this document would be greatly appreciated. Please forward any and all suggestions, comments, and/or questions regarding this specification to the Advanced Vehicle Technology Worldwide Diagnostic Organization.

All references to *"Mandatory"* and *"Optional"* requirements throughout this specification identify which diagnostic features/functions an Electronic Control Unit must (*"Mandatory"*) implement and which may (*Optional"*) be implemented. However, if the *"Optional"* feature/function is implemented, it MUST adhere to the requirements defined in this specification.

Note that all technical adjustments made to this document and changes incorporated since the last edition was released are recorded. A summary listing of these additions and changes can be found in the back of this issue under the heading "Appendix C". An electronic copy of this document is located at the following Universal Resource Locator (WEB address):

http://eesftp.pd8.ford.com/diag/released/_generic/part_one/pt1_v42/pt1_v42.doc.

Table of Contents

SEE FILE "PT1_42A.DOC" FOR APPENDIX TABLE OF CONTENTS

1. INTRODUCTION

1.1 PURPOSE

The purpose of this specification is to provide guidelines and to define requirements for the development of electrical system diagnostics that will be used to functionally evaluate serial communications link Electronic Control Units (ECUs) and their associated subsystems. This specification also defines requirements and provides general information necessary for the development of diagnostic service tools.

This document supports the diagnostic standards and guidelines defined in the **FORD Worldwide Customer Requirements (WCR)** documents for **Passenger Cars and Light Truck** (section number 00.00-EA-D05, Diagnostics). It also supports recommended diagnostic practices specified by the Society of Automotive Engineers (SAE 1930 - Electrical/Electronic Systems Diagnostic Terms, Definitions, Abbreviations, and Acronyms, SAE J1962 - Diagnostic Connector, SAE J2012 - Diagnostic Trouble Code Definitions).

This document, is one of three unique types of diagnostic specifications required for the implementation of vehicle serial communications link ECU diagnostics. The three Diagnostic Specifications are as follows:

- GLOBAL DIAGNOSTIC SPECIFICATION (GDS - Part I)
- SUBSYSTEM SPECIFIC DIAGNOSTIC SPECIFICATION (SSDS - Part II)
- VEHICLE INTERFACE DIAGNOSTIC SPECIFICATION (VIDS - Part III)

CAUTION:
Any deviation to this "Global Diagnostic Specification", for any ECU Subsystem, shall only be valid if agreed upon in whole by the Worldwide Diagnostics Organization (WWDO), diagnostic application engineer. Any deviation so granted shall be fully documented and adequately explained in the SSDS for the ECU subsystem.

1.1.1 SUBSYSTEM SPECIFIC DIAGNOSTIC SPECIFICATION (SSDS - Part II)

The **Subsystem Specific Diagnostic Specification (SSDS)** provides specific details about the diagnostic capabilities (commands, tests, parameter identifier, diagnostic trouble codes, etc.) that an ECU shall support.

Each Diagnostic SSDS for the ECU shall adhere to all of the diagnostic requirements and recommended diagnostic practices described in this document.

1.1.2 VEHICLE INTERFACE DIAGNOSTIC SPECIFICATION (VIDS - Part III)

The **Vehicle Interface Diagnostic Specification** defines the diagnostic requirements for a vehicle's ECU-to-ECU Network Communications (SCP) and the hardware interfaces.

2. DIAGNOSTIC DEVELOPMENT PROCESS

2.1 PURPOSE

This section explains the process for developing accurate, serial link ECU diagnostics. It also explains their relationship with the development of service manual procedures.

2.2 BENEFITS OF DIAGNOSTICS

A partial list of benefits for implementing serial link diagnostics in an ECU are as follows:

- Customer Satisfaction by achieving "Right First Time" repairs (eliminating repeat repairs)
- Warranty Cost Reduction by:
  a) reducing Trouble Not Identified (TNI) rates
  b) reducing repeat repairs
  c) reducing the time required to isolate failures

2.3 SUBSYSTEM DESIGN SPECIFICATION

Each Subsystem Design Specification (SDS) explains the application of a vehicle's subsystem. It is generated by the subsystem developer from the functional requirements of the subsystem's features. Understanding the operation of the subsystem is essential for the development of the subsystem's Failure Modes & Effects Analysis (FMEA) and diagnostics.

2.4 FMEA DRIVEN DIAGNOSTICS

Failure Modes & Effects Analysis (FMEA) is a methodology whereby all of a subsystem's failure mode causes and effects are identified and prioritized for the purpose of analyzing and improving the system/subsystem's reliability. According to the Ford Product Development System (FPDS), a FMEA is required to be performed on every subsystem by the Program Approval milestone (FPDS PA).

An effective subsystem FMEA will take into account the following:

- normal and abnormal operation of the system
- the probability of a component failing
- information on how a component could fail

The subsystem's FMEA should contain cause and effect information on:

- hardware/component failures
- wiring failures
- operating software failures
- network communication failures

NOTE: A FORD "FMEA Plus" software package exists to assist in the development of a FMEA.

2.4.1 FMEA Fault Occurrence Ranking

Table 2-1 shows the fault occurrence probabilities used for ranking FMEA faults. A higher ranking corresponds to a higher repair rate and necessitates diagnostic coverage. Refer to the **Failure Mode & Effects Analysis Handbook.**

Table 2-1 FMEA Fault Occurrence Ranking Chart

Probability of Failure	Ranking	Rate of Failure	R/1000
Remote	1	< 1 in 10^6	0.001
Low	2	1 in 20,000	0.05
	3	1 in 4,000	0.25
Moderate	4	1 in 1,000	1.0
	5	1 in 400	2.5
	6	1 in 80	12.5
High	7	1 in 40	25.0
	8	1 in 20	50.0
Very High	9	1 in 8	125.0
	10	1 in 2	500.0

2.4.2 I/O Fault Probability Ranking

The development of ECU diagnostics shall use the following fault probability rankings to evaluate a subsystems I/O faults:

1) Open Circuit
2) Intermittent Open Circuit
3) Intermittent Short-to-Ground
4) Short-to-Ground
5) Battery Short
6) Intermittent Short-to-Battery
7) Signal Faults

NOTE: An Open Circuit is the most prevalent type of I/O fault and shall be given a higher priority consideration for diagnostic coverage.

2.5 DIAGNOSTIC COVERAGE

According to the design standards specified in the **FORD Worldwide Customer Requirements (WCR) - Passenger Car and Light Truck (Diagnostics section 00.00.EA-D05)**, the ECU's diagnostics shall be capable of detecting the top 80 percent (highest probability of occurrence) of the prioritized system/subsystem faults listed in the FMEA.

Of the remaining 20 percent of the faults listed in the FMEA, an ECU's diagnostics shall consider identifying all faults which would have a significant impact on Warranty Costs [part costs, labor costs, and Trouble Not Identified (TNI) rates].

2.6 DIAGNOSTIC TROUBLE CODES (DTCs) & FAULT CORRELATION

ECU input and output fault symptoms (customer-observed) recorded and identified in a FMEA summary document shall be correlated to Diagnostic Trouble Codes (DTCs - Continuous and On-Demand) in the **"DTCs"** section of the **SSDS** for each ECU.

This information shall be further correlated to I/O specific Parameter Identifier (PID) and command information in the **"OVERVIEW"** section of the **SSDS** for each ECU. This **"OVERVIEW"** section of the **SSDS,** which provides an arranged snapshot of subsystem specific diagnostic capabilities, can then be used as a reference in the development of service manual pinpoint tests and test tool diagnostic flow diagrams.

3. LINK-BASED DIAGNOSTIC SYSTEM OVERVIEW

3.1 PURPOSE

This section provides an overview of the diagnostic "system" (vehicle, ECU, service technician, tester, service manual) to serve as an introduction to the detailed information presented in subsequent sections.

3.2 ECU DIAGNOSTIC SYSTEM COMPONENTS

The following subsections discuss how each element fits into the diagnostic evaluation process. Refer to Figure 3-1 for a typical setup of a vehicle undergoing diagnostic evaluation.

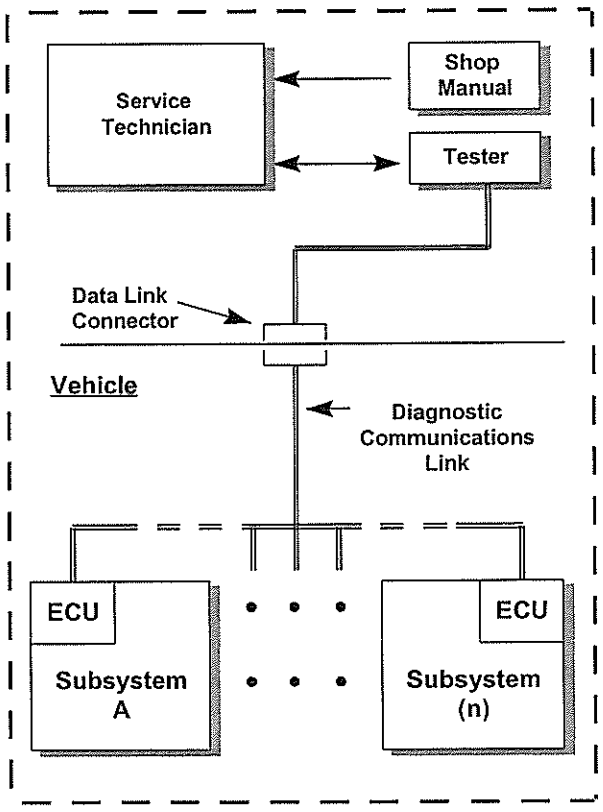


Figure 3-1 Vehicle Diagnostic Setup

3.2.1 Electronics Control Unit (ECU)

An ECU is the microprocessor-based "brains" of a vehicle subsystem. Each ECU receives input data from various switches, sensors, etc., and processes that data to control the subsystem's outputs. Each ECU supports communication over the communications link and may also receive data from other ECUs on the link.

Each ECU contains on-board diagnostics (as specified in the **SSDS** for that ECU), that allows the ECU and a set of its subsystem components to be evaluated via the vehicle's serial communication link.

3.2.2 Vehicle Serial Communications Link

The vehicle serial communications link is a hard-wired physical interface between each of the vehicle's ECUs and the vehicle data link connector (DLC). It provides the capability for tester-to-ECU and/or ECU-to-ECU communications. Actual communication occurs in accordance with a communications protocol uniquely defined for the serial link.

This specification provides diagnostic information on ECUs which incorporate either of two serial communication links and their respective protocols, listed below:

- ISO-9141-FORD (ES-F7LC-12K529-FD)
- Standard Corporate Protocol (ES-F7LC-12K529-CC)
- UART Based Protocol (UBP) - (ES-N710007BFTEFAA)
- Controller Area Network (CAN) - (ES-XS4T-12K529-AA)

3.2.2.1 ISO-9141-FORD Serial Communications Link

The ISO-9141-FORD serial communications link, which is a FORD interpretation of ISO-9141, is achieved over a single wire connection between the tester and each of the vehicle's ECUs via the DLC. The ISO-9141-FORD serial communications link protocol operates Node-to-Node using a Master-Slave relationship. The tester is the master that initiates all communications with the ECU(s) undergoing testing and responding to the tester as the slave

Tester and ECU physical layer information resides in the Ford ISO-9141 document titled "Protocol Definitions and Interface Requirements" with the part number ES-F7LC-12K529-FD.

NOTE: ISO-9141-FORD only permits tester-to-ECU communication; it does not support ECU-to-ECU communication.

3.2.2.2 SCP Serial Communications Link

The SCP serial communications link, which is based on SAE J1850, uses a twisted wire pair to provide a physical communications path between each of the vehicle's ECUs and the Tester, connected to the vehicle's DLC. The SCP serial communications link's protocol supports both ECU-to-ECU communication and Tester-to-ECU communication.

The length of the off-board extension of the SCP bus shall be limited to no more than the length specified in section 5.3 of the "SCP Vehicle Network Implementation Requirements" (SCP-003) part #: F7LC-12K529-DE. Also, specification of the ground offset is noted in section 5.9 of (SCP-003). The interface circuitry for SCP network interfaces is to be found in section 5.6 of (SCP-003).

Refer to "SCP Protocol Definition and Interface Requirements" (SCP-001) for low level protocol information including physical layer descriptions.

Refer to "SCP Vehicle Network Interface Requirements" (SCP - 003) for an application level description of the SCP network. Provides message format descriptions and physical layer information and references used to implement the protocol in a module.

SCP diagnostic dialog and message structure description is located in Appendix A of this document.

NOTE: For diagnostics, all SCP communications between a tester and the ECU is initiated by the tester.

3.2.2.3 Interaction of SCP-DIAG and SCP-CARB

SCP-CARB and SCP-DIAG communication may co-exist without interference on the same network.

3.2.3 Data Link Connector (DLC)

The data link connector (or central diagnostic connector) is the mandatory electrical connector required for On Board Diagnostics (OBD-II) emissions compliance. The data link connector is the physical entity to which a Tester is attached so it may communicate with each of the vehicle's ECUs over the serial communications link. It is located within the vehicle under the instrument panel. The data link connector is a standard corporate diagnostic connector that complies with the recommended practices specified in SAE J1962. For a detailed description of the data link connector's location and pin assignments refer to the **WCR - Passenger Car and Light Truck, Section 00.00EA-D05 .**

3.2.4 Tester

A tester provides the service technician with the user interface and control capabilities for Non-Intrusive testing of serial communications link ECU subsystems. Non-Intrusive testing is the ability to retrieve data from an ECU without having to physically disassemble a portion of the subsystem.

Example: Using a voltmeter, the voltage being supplied to an ECU may be able to be determined without having to disconnect wires for a physical measurement.

Based on the input selected by a service technician, the tester transmits requests over the serial communications link to an ECU. In response, the tester will receive data from the ECU and display the appropriate diagnostic information to the service technician. The tester is always the master in the Master-Slave association with ECUs in the tester-to-ECU diagnostic communication sequences. FORD currently provides the service arena with the following testers, each capable of supporting serial communications link diagnostics:

- NGS (New Generation Star) Tester
- FDS-2000 (Ford Diagnostic System - 2000)
- SBDS (Service Bay Diagnostic System)
- Ford Assembly Configuration and Testing System/End of Line (FACTS/EOL). Used in manufacturing only.
- Worldwide Diagnostic System (WDS)

3.2.4.1 New Generation Star (NGS) Tester

The NGS is currently the FORD mandatory essential tool for North American markets. It is required for use with the vehicle's service manuals to diagnose serial communications link ECUs. The NGS is a hand-held tester that allows the ECU to be evaluated while the vehicle is stationary or while it is moving.

3.2.4.2 FORD Diagnostic System (FDS-2000)

The FDS-2000 is currently the mandatory essential tool for European markets. It is required for use with the vehicle's service manuals to diagnose serial communications link ECUs. The FDS-2000 is a diagnostic system that consists of a base unit which downloads diagnostic information to a portable hand-held tester. The portable tester allows the ECU to be evaluated while the vehicle is stationary or while it is moving.

3.2.4.3 Service Bay Diagnostic System (SBDS)

The SBDS is a personal computer based diagnostic system comprised of a suite of diagnostic tools. The SBDS combines service procedures into vehicle-specific, touch-screen-guided diagnostics. The SBDS also provides On-the-Go diagnostic data-gathering capabilities via its portable vehicle analyzer and customer flight recorder. The portable units are used for analyzing problems that may occur only when the vehicle is moving, under load, or that may occur intermittently.

3.2.4.4 Ford Assembly Configuration and Testing System/End of Line (FACTS/EOL)

The FACTS/EOL is a computer based diagnostic system used to test a variety of electrical systems in newly manufactured vehicles prior to shipment. This global system is currently used to test every sellable unit produced in all North American assembly plants (US, Canada, and Mexico) with total European coverage underway. It is capable of providing the following major functions: testing body chassis electrical and powertrain quality in multiple

languages, providing vehicle repair information via touch screen, diagnostic data handling for collection/storage/playback and in-plant mandatory test compliance.

3.2.4.5 Worldwide Diagnostic System (WDS)

The WDS is the next generation worldwide diagnostic system that is intended to replace NGS, FDS-2000, and SBDS.

3.2.5 Service Technician

The service technician evaluates each of the vehicle's ECUs based on instructions in the vehicle's service manual, or as provided on the video screen of the SBDS, FDS2000 or NGS. The service technician is responsible for selecting (from tester menus) appropriate ECU diagnostic parameters, commands, and tests, and initiating Tester-to-ECU investigative inquiries.

Once diagnostic results are returned from the module to the tester, the service technician is responsible for determining the location and cause of the ECUs fault(s) and repairing the fault(s) based on data displayed on the NGS / FDS-2000 and in their respective service manuals or from information displayed on the SBDS Tester's video screen.

3.2.6 Service Manual

The vehicle's service manual integrates the diagnostic capabilities of the ECU(s) and the NGS / FDS-2000 testers into the procedures a service technician will follow to isolate and repair ECU failures.

The service manuals contain a section entitled **"Section 418-00, Module Communications Network"**. The purpose of this section is to provide the service technician with the diagnostic procedures necessary for isolating network communication faults.

3.3 ECU DIAGNOSTIC STRUCTURE

A state transition diagram that identifies the basic design structure (operating states and modes) of the ECU software is shown in Figure 3-2.

Operational State - The state from which the ECU executes its normal operating strategy. Part of that strategy includes supporting a limited set of diagnostic capabilities and providing access to the Diagnostic State.

Diagnostic State - The state the ECU enters, upon request from a Tester, to execute its primary set of diagnostic functions (as specified in the ECU's **SSDS**).

Test Mode - A mode the ECU enters, when requested by a Tester, to execute a diagnostic test.

Command Mode - A mode the ECU enters, when requested by a Tester, to execute a diagnostic command.

No Stored Codes Logging Mode - A mode the ECU enters, when requested by the Tester, to prevent logging of erroneous missing message continuous network DTCs.

Memory Transfer Mode - A mode the ECU enters, when requested by the Tester, to download/upload information.

Gateway Mode - A mode the ECU enters, when requested by the Tester, to allow the Tester to communicate with non-ISO-914-FORD/non-SCP ECUs.

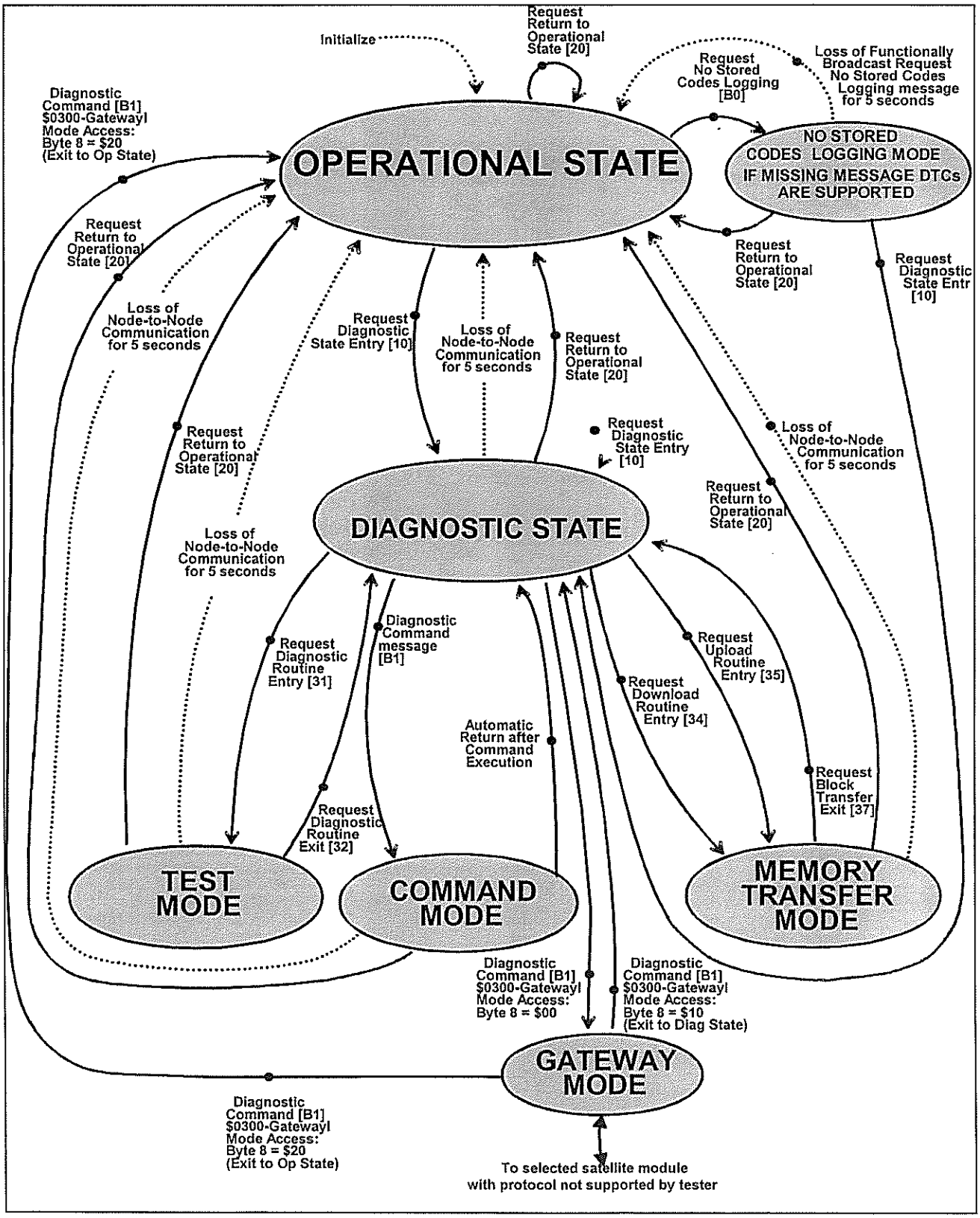


Figure 3-2 ECU Operating States/Modes

4. OPERATIONAL STATE DIAGNOSTIC REQUIREMENTS

4.1 PURPOSE

This section defines the diagnostic guidelines and requirements that all bi-directional serial communication link ECUs will support while in Operational State.

4.2 OPERATIONAL STATE REQUIREMENTS

While in Operational State, an ECU shall support the serial communications link interface requirements shown in Figure 4-1. This includes supporting the following functions:

- Continuous diagnostic trouble code (DTC) criteria → Mandatory
- Access to parameter identifier information → Mandatory
- Data access via direct memory reference (DMR) → Optional
- Entry into the Diagnostic State → Mandatory
- Network diagnostic communication strategies (SCP Only) → Mandatory
- Entry into the No Stored Codes Logging mode (SCP Only) → Optional
- Wake-Up from a message from a tester or a designated "wake-up" input → Mandatory

For detailed information on the messages that an ECU shall support while in the Operational State, refer to ***Appendix A, Message Structures & Responses.***

4.2.1 Continuous Diagnostic Trouble Code (DTC) Requirements → Mandatory

Continuous Diagnostic Trouble Codes (DTCs) are fault codes that are logged by an ECU upon the detection of ECU failures while the ECU is functioning in the Operational State.

In Operational State, an ECU shall support the following Continuous DTC requirements:

- Logging Continuous DTCs
- Reporting logged Continuous DTCs over the serial communications link upon request clearing logged Continuous DTCs upon receiving a request over the serial communications link. For safety critical modules (Ex. ABS), an internal self-test is performed after clearing continuous codes in Operational State to verify module integrity. If any safety critical functions aren't operating correctly, a module code and/or malfunction indicator (if equipped) will be set for diagnosis and reporting of the fault. The internal self-test is not to exceed 500mS.
- Automatically clearing of Continuous DTCs after certain conditions occur that are independent of tester intervention.

4.2.2 Parameter Identifier (PID) Access → Mandatory

A parameter identifier (PID) is a logical address assigned to specific module information that may be retrieved by a tester (i.e., the Tester does not need to know the physical address of the information within the ECU to obtain the data). Specific PID information shall be identified in the SSDS (Part II) document for each ECU and its subsystem.

In the Operational State, an ECU shall be capable of interpreting PID requests and reporting the requested information.

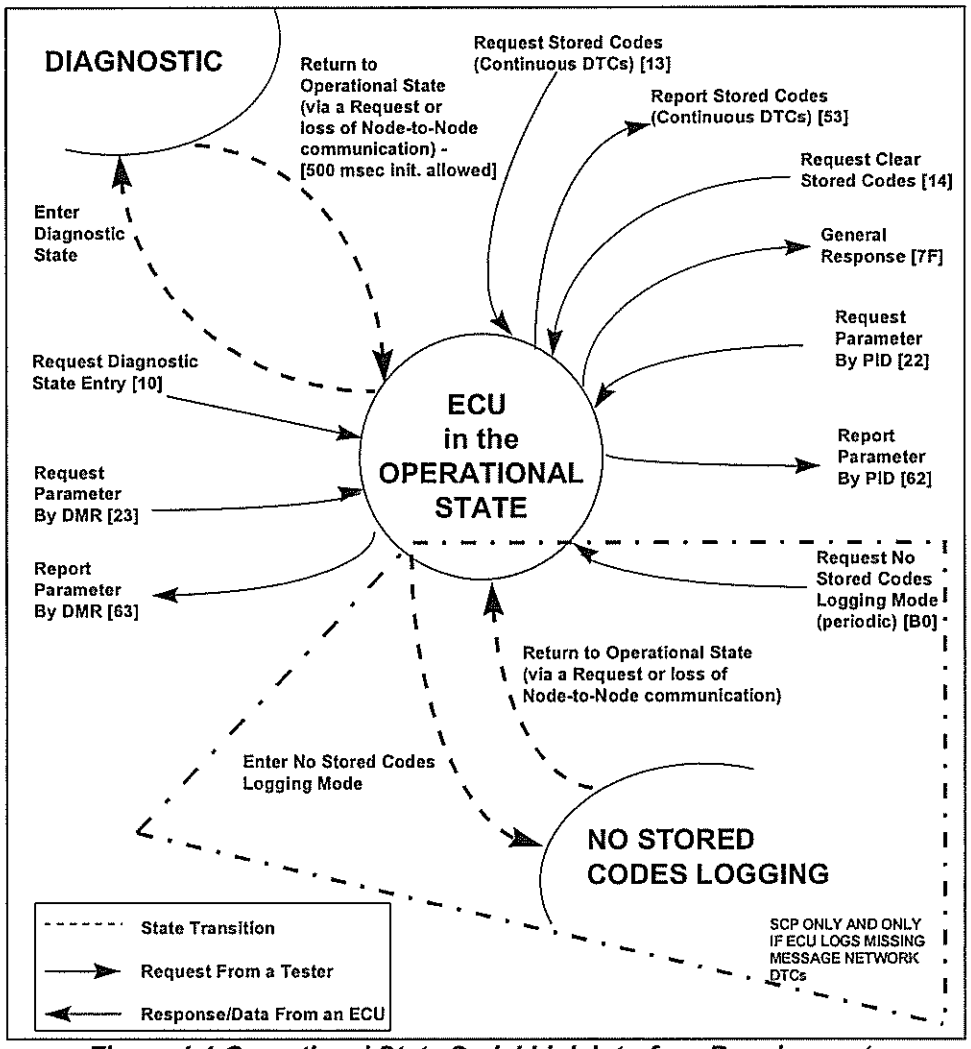


Figure 4-1 Operational State Serial Link Interface Requirements

4.2.3 Direct Memory Reference (DMR) → Optional

Direct memory reference (DMR) provides a Tester with the capability to retrieve information from an ECU by specifying the direct physical address where the data is located within the ECU. The specific DMR information to be supported by an ECU shall be identified in the ECUs SSDS. Refer to **Section 11.0, MEMORY ACCESS AND DOWNLOADING REQUIREMENTS**, for a detailed description DMR.

In the Operational State, an ECU shall be capable of interpreting DMR requests and reporting the requested information.

4.2.4 Entry into the Diagnostic State → Mandatory

An ECU shall enter the Diagnostic State from the Operational State upon receiving a ***Request Diagnostic State Entry*** message over the serial communications link, if all entry criteria specified in the ECUs SSDS have been satisfied.

4.2.5 Network Diagnostic Communication Strategies (SCP Only)→ Mandatory

The network diagnostic communication strategies provide methods for preventing and handling fault conditions which prevent valid inter-module communication from occurring over a vehicle's SCP network (e.g., missing

message network DTC). For detailed information about the network diagnostic communication strategies, refer to **Section 10.0, NETWORK COMMUNICATION DIAGNOSTIC REQUIREMENTS.**

4.2.6 No Stored Codes Logging (NSCL) Mode (SCP Only) → Optional

The No Stored Codes Logging (NSCL) Mode is a sub-mode of the Operational State that an ECU is directed to enter to prevent it from logging erroneous Missing Message Continuous Network DTCs. An ECU in the Operational State shall enter the No Stored Codes Logging mode upon receiving a ***Request No Stored Codes Logging*** message over the SCP communications link. A SCP ECU is required to support the No Stored Codes Logging Mode only if it supports logging of Missing Message Continuous Network DTCs that result from missing SCP messages.

4.2.7 ECU Wake-Up Requirements → Mandatory

All ECUs can be classified as being one of the following node types:

- Non-Sleep Node ECU
- Sleep Node ECU

4.2.7.1 Non-Sleep Node ECU Wake-Up Requirements

A Non-Sleep Node ECU never enters a power-conserving "sleep" mode of operation. A non-sleep node ECU may be designated to issue an **Ignition Switch Position - RUN** message after sensing its Ignition RUN input has gone active in order to "wake-up" sleep node ECUs.

4.2.7.2 Sleep Node ECU Wake-Up Requirements

A sleep node ECU is a module that enters a power-conserving "sleep" mode of operation when it does not sense any transitions on its inputs, or serial communications link activity, for a pre-determined period of time. A sleep node ECU shall begin normal operation after receiving some form of "wake-up" stimulus, either a message from a non-sleep node ECU (SCP only), a message from a tester, or a transition on a designated "wake-up" input. A sleep node ECU shall be able to interpret and respond appropriately to all subsequent messages, after waking-up. Since input transitions and SCP link activity are commonplace while a vehicle is in RUN, a Sleep Node module will only enter "sleep" mode when the ignition is OFF. NOTE: all ECUs MUST be able to wake-up from either a message from a tester or a transition on a designated "wake-up" input triggered by Ignition RUN.

When a SCP message is used to wake-up a Sleep Node ECU, that message may not be able to be decoded by the Sleep Node ECU because it can not wake-up and initialize in time. Upon receiving the wake-up message, the Sleep Node ECU shall be able to interpret all subsequent messages. NOTE: A sleep node ECU shall "wake-up" upon receiving a message from a tester.

5. DIAGNOSTIC STATE REQUIREMENTS

5.1 PURPOSE

This section defines the diagnostic guidelines and requirements that all bi-directional serial communications link ECUs will support while in the Diagnostic State.

5.2 ENTERING THE DIAGNOSTIC STATE

To enter Diagnostic State from Operational State or No Stored Codes Logging Mode, an ECU shall meet all entry conditions specified in the ECUs SSDS. If all entry conditions are satisfied, the ECU shall enter the Diagnostic State upon receiving a ***Request Diagnostic State Entry*** message. If any entry conditions are not met, the ECU shall not enter the Diagnostic State and shall return a ***General Response*** message with the Response Code set to ***$22 - CONDITIONS NOT CORRECT.***

While in the Diagnostic State, an ECU is only required to respond to diagnostic messages as defined in **Appendix A, Message Structures and Responses**.

5.2.1 I/O in the Diagnostic State

Upon entering the Diagnostic State, all ECU controlled outputs shall be made inactive, unless specified otherwise in the ECUs SSDS. Also, upon entering the Diagnostic State, all ECU inputs shall be isolated from directly controlling related ECU outputs (e.g., switch input will not activate output).

5.3 DIAGNOSTIC STATE REQUIREMENTS

An ECU in the Diagnostic State shall:

- Maintain a 5.0-second (-0.0, +1.0) timer that shall be reset each time it receives ANY Node-to-Node message
- Perform all diagnostic functions as defined in, **Section 5.0, DIAGNOSTIC STATE REQUIREMENTS**
- Ignore any broadcast messages

While in the Diagnostic State, an ECU shall support all of the serial communications link interface requirements shown in Figure 5-1. This includes mandatory support of some of the following functions:

- Diagnostic Tests → Mandatory
- Diagnostic Commands → Optional
- Continuous DTC requirements → Mandatory
- On-Demand DTC requirements → Mandatory
- Parameter Identifier (PID) requirements → Mandatory
- Direct Memory Reference (DMR) → Optional
- Memory Transfer → Optional
- Write Block $3B → Optional
- Read Block $3C → Optional
- Security Access $27 → Optional
- Upload/Download $35/$34 → Optional

5.3.1 Diagnostic Tests → Mandatory

Diagnostic Tests are executable routines resident within an ECU that may be invoked by a tester to evaluate the ECU and its associated subsystem component(s).

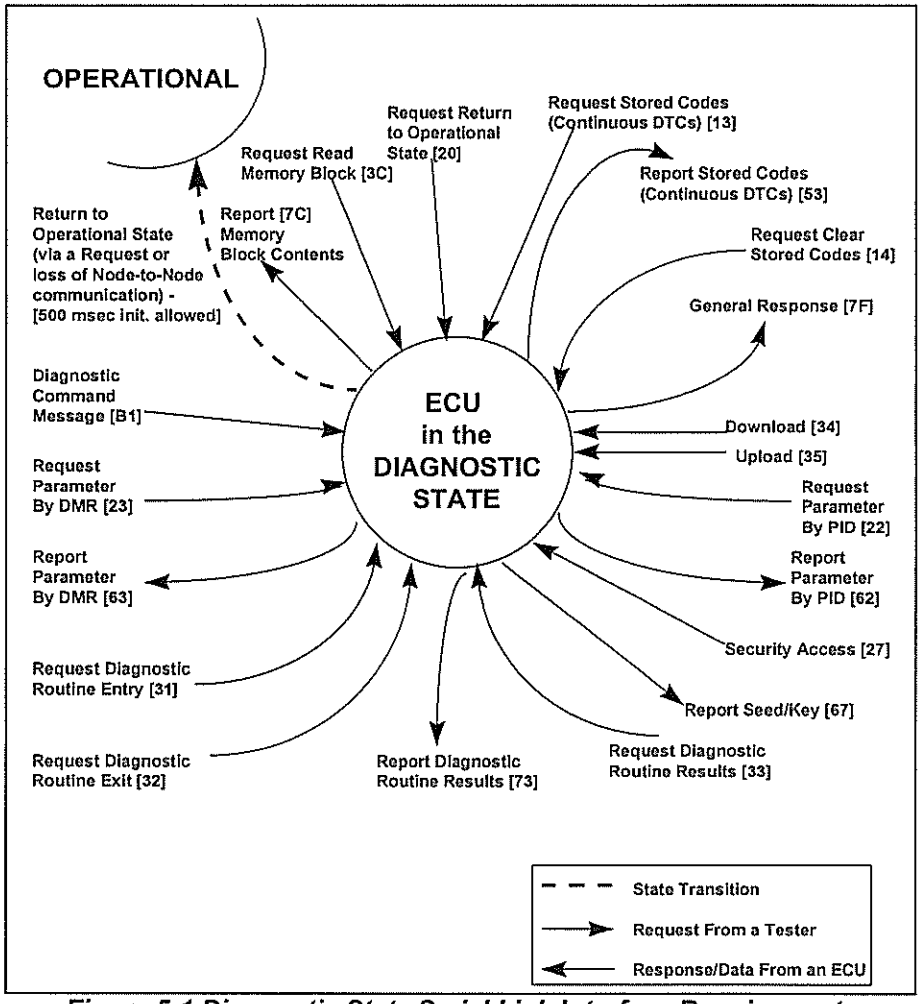


Figure 5-1 Diagnostic State Serial Link Interface Requirements

5.3.2 Diagnostic Commands → Optional

Diagnostic Commands allow a tester to control specific ECU activity such as enabling and disabling ECU outputs.

5.3.3 Continuous Diagnostic Trouble Codes (DTCs) Requirements → Mandatory

Continuous Diagnostic Trouble Codes (DTCs) are fault codes that are logged by an ECU upon the detection of ECU failures while the ECU is functioning in the Operational State.

In the Diagnostic State, an ECU shall support the following subset of the Continuous DTC:

- Reporting logged continuous DTCs over the serial communications link upon request
- Clearing logged continuous DTCs upon receiving a request over the serial communications link

Note: No continuous DTCs shall be logged by an ECU while it is executing in Diagnostic State; except DTC A141 and A477. However, continuous DTCs previously logged shall be maintained (i.e., not overwritten) by the ECU while it is executing in the Diagnostic State.

5.3.4 On-Demand Diagnostic Trouble Codes (DTCs) Requirements → Mandatory

On-Demand Diagnostic Trouble Codes (DTCs) are fault codes logged by an ECU which identify subsystem failures detected during the execution of any diagnostic test (e.g., On-Demand Self-Test) supported by the ECU.

In Diagnostic State, an ECU shall support the following On-Demand DTC requirements:

- Logging On-Demand DTCs that identify subsystem failures detected during the execution of a diagnostic test.
- Reporting On-Demand DTCs over the serial communications link upon request.

5.3.5 Parameter Identifier (PID) Requirements → Mandatory

A parameter identifier (PID) is a logical address assigned to specific module information that may be retrieved by a tester by specifying the logical PID address (i.e., the tester does not need to know the physical address of the information within the ECU to obtain the data). Specific PID information shall be identified in the SSDS (Part II) document for each ECU and its subsystem.

While in Diagnostic State, an ECU shall be capable of interpreting PID requests and reporting the requested information.

5.3.6 Direct Memory Reference (DMR) Requirements → Optional

Direct memory reference (DMR) provides a Tester with the capability to retrieve information from an ECU by specifying the direct physical address where the data is located within the ECU. The specific DMR information to be supported by an ECU shall be identified in the ECUs SSDS.

While in the Diagnostic State, an ECU shall be capable of interpreting DMR requests and reporting the requested information.

5.3.7 Memory Transfer → Optional

ECU support of the memory transfer capability will provide a Tester with the capability to download data to an ECU and/or to upload data from an ECU. The specific memory transfer information to be supported by an ECU shall be identified in the ECUs SSDS.

While in the Diagnostic State, the ECU will accept either the download or upload configuration (modes $34 or $35 respectively) that also initiates a change to memory transfer mode..

5.4 EXITING THE DIAGNOSTIC STATE

An ECU shall exit the Diagnostic State and return to the Operational State if any of the following conditions occur:

- Module receives a ***Request Return To Operational State*** message over the serial communications link
- The ECU doesn't receive ANY Node-to-Node message (supported or unsupported) within 5.0 (-0.0, +1.0) seconds of either the last message received or initial entrance into diagnostic state.
- A requirement, as specified in the ECUs SSDS, for remaining in the Diagnostic State is violated

5.4.1 I/O Requirements

Upon returning to the Operational State, an ECU shall return all I/O to their default/inactive states and resume normal operation.

5.4.2 Diagnostic to Operational State Timing

Upon returning to the Operational State, an ECU shall return to normal operation within 500 milliseconds. This time period should be sufficient for re-initialization. During the 500 millisecond re-initialization period, the ECU is not required to respond to any diagnostic messages.

5.4.3 Auto-Clearing Diagnostic Test Results

Upon returning to the Operational State, all On-Demand DTCs logged during execution of a Diagnostic Test shall be cleared.

6. DIAGNOSTIC TEST MODE REQUIREMENTS

6.1 PURPOSE

This section defines the requirements for the Diagnostic Test mode and the diagnostic tests that reside within an ECU for the purpose of evaluating the ECU and its subsystem(s). It also describes the ability for an external tester to download an executable piece of code to the ECU and execute it; this functionality is primarily for manufacturer use (supplier and Ford).

NOTE: The definitions of all tests must be approved by the AVT EESE WWDO Vehicle Diagnostics representative. All approved tests and their descriptions are maintained in a FORD Worldwide Corporate Master Reference DataBase - MRDB, maintained by the Vehicle Diagnostics organization.

6.2 DIAGNOSTIC TESTS

Diagnostic tests are executable routines resident within an ECU or downloaded from a tester that may be invoked by a tester to evaluate the ECU and its associated subsystem components. The diagnostic tests that an ECU shall support will be specified in the ECUs SSDS.

All ECUs shall support diagnostic tests as follows:

- On Demand Self Test → Mandatory
- Wiggle test → Optional
- Subsystem specific tests → Optional
- Tester downloaded execution routines/tests → Optional

NOTE: An ECU is only required to support the execution of a single test at any given time.

6.2.1 On-Demand Self-Test ($02) → Mandatory

The On-Demand Self-Test is a mandatory diagnostic test and should attempt to detect and identify all of the ECU's internal I/O circuit faults.

6.2.1.1 On-Demand Self-Test Guidelines

All requirements for an ECU On-Demand Self-Test shall adhere to the following guidelines:

- The top 80% of the faults listed in the FMEA shall be tested, this should include internal circuitry for RAM, ROM, Timers, etc. Refer to the **FORD WORLDWIDE CUSTOMER REQUIREMENTS (WCR), SECTION 00.00.EA-D05 - DIAGNOSTICS** and the **FMEA HANDBOOK** for a detailed definition of this requirement.
- All ECU inputs (e. g., door ajar, 4x4 low, etc.) shall be physically placed in a defined state, and verified to be in that state. NOTE: Since "opens" are the predominant I/O circuit fault type, inputs should be set to a state where an "open" would be detected as a circuit fault.
- All ECU outputs that employ feedback capability shall be energized long enough to detect a fault, and if one exists, to identify its type (open, short-to-Vbat, short-to-ground, etc.).
- All ECU outputs that control subsystem functions that can be monitored shall be actuated and the subsystem's functionality verified. (i. e., energizing a shock damper motor output that cycles a shock's firm/ soft ride type and monitoring a separate shock ride setting input).

NOTE: Upon completion of the On-Demand Self-Test, the ECU shall return its outputs to their initial entry conditions (in existence prior to the ECU entering On-Demand Self-Test). If the ECU returns to Operational State from a loss of module communications for 5 seconds, then the outputs will be set to the states that were set upon entering diagnostic state prior to entering the test mode.

6.2.1.2 On-Demand Self-Test Execution Time

An ECU shall complete its On-Demand Self-Test within a maximum of 30 seconds after receiving the request to perform the test.

6.2.1.3 On-Demand Self-Test I/O Circuit Omission

Omitting the evaluation of an ECU I/O circuit shall only be permitted for the following reasons:

- Testing a circuit would cause the 30-second On-Demand Self-Test execution time limit to be exceeded.
- Testing a circuit would require operator interaction during the test.
- Testing a circuit could damage the ECU and/or any of its components.
- Testing the circuit could cause a hazardous condition.
- The reliability of the circuit is good enough so that it does not fall within the top 80 percent (highest probability of occurrence) of the faults listed in the FMEA for the ECU Subsystem.

6.2.2 ECU Specific Functional Tests → Optional

ECU specific functional tests are diagnostic tests used to determine the operating condition of the ECUs circuitry and components that could not be fully tested during the On-Demand Self-Test. These tests are unique for each ECU and shall be defined in the ECUs SSDS. NOTE: an ECU specific functional test may be required to test I/O circuits omitted from the On-Demand Self-Test. Contact your local AVT EESE WWDO Vehicle Diagnostics representative for a definition of any ECU specific functional tests.

6.2.3 Wiggle Test ($0B) → Optional

A Wiggle test is a service technicians interactive test used to determine whether intermittent subsystem failures are occurring because of poor connections or broken wires.

When an ECU receives a ***Request Diagnostic Routine Entry*** message with Byte five set to ***$0B - Wiggle Test***, the ECU will monitor its I/O lines for state transitions (*low-to-high* or *high-to-low*) while a service technician "wiggles" the ECU's harness(es). The I/O lines that will be monitored will be specified in the ECUs SSDS. Upon the detection of a change-of-state fault on any I/O line, the ECU shall log an On-Demand DTC that identifies that fault. In addition, the ECU shall increment the value in PID ***$0201*** to reflect the number of individual I/O transitions that were detected.

6.2.4 Tester downloaded execution routines/tests ($05) → Optional

Intended for computer driven test stands to verify and test module functionality per manufacturer defined tests and execution routines. The module's SSDS will specify the starting address of where the execution routine is to be downloaded (Use normal download procedure specified in this document) and the amount of memory available for the routine. The execution of the routine will use Mode $31 and test number $05. This gives the flexibility to set up any needed tests for the module without re-design or re-allocation of module resources.

6.2.5 Diagnostic Test Entry Requirements

An ECU shall execute a Diagnostic Test upon receiving a ***Request Diagnostic Routine Entry*** message from a tester if the following entry requirements are satisfied:

- The ECU is in the Diagnostic State
- All entry requirements specified in the ECUs SSDS have been satisfied
- Clear ECU memory allocated for the storage of the Diagnostic Test results

6.2.6 Exiting a Diagnostic Test

An ECU shall exit Diagnostic Test Mode and return to one of the following states:

- Diagnostic State
- Operational State

6.2.6.1 Exiting a Diagnostic Test to the Diagnostic State

An ECU shall exit a diagnostic test and return to the Diagnostic State if any of the following occurs:

- The ECU receives a ***Request Diagnostic Routine Exit*** message over the serial communication link. Byte 6 specifies the exit condition: ***$00*** - exit if test is completed, ***$01*** - exit immediately.
- An exit condition, as specified in the ECUs SSDS, that causes the ECU to return to the Diagnostic State, is met. Example: If the ECUs SSDS states that "all doors shall remain closed during test execution" and a door is opened while the test is executing, the module shall immediately exit from the test and return to the Diagnostic State.

6.2.6.2 Exiting a Diagnostic Test to the Operational State

An ECU shall exit a Diagnostic Test and return to the Operational State if any of the following occurs:

- The ECU receives a ***Request Return to Operational State*** message over the serial communications link.
- The ECU doesn't receive ANY Node-to-Node message (supported or unsupported) within 5.0 (-0.0, +1.0) seconds of either the last message received or initial entrance into diagnostic state.
- An Exit condition, as specified in the ECUs SSDS, that causes the ECU to return to the Operational State, is met.

6.2.7 Logging Diagnostic Test Results

When a failure is detected during the execution of a Diagnostic Test, the ECU will log an On-Demand DTC that identifies the fault, as specified in the ECUs SSDS. Refer to **Section 9.0, DIAGNOSTIC TROUBLE CODE (DTC) REQUIREMENTS** for a detailed description of On-Demand DTCs

6.2.8 Reporting Diagnostic Test Results

While in Diagnostic State, an ECU shall be capable of reporting all results (On-Demand DTCs) generated during the most recent diagnostic test executed. The number of DTCs logged during the execution of the most recent test executed, may be retrieved by issuing a ***"Request Parameter By PID"*** message specifying PID ***$0202 - Number of Trouble Codes Set Due To Diagnostic Test***. Refer to **Section 9.0, DIAGNOSTIC TROUBLE CODE (DTC) REQUIREMENTS** for a detailed description of On-Demand DTCs.

NOTE: The data returned after the execution of a Diagnostic Test, is only expected to be correct while in the Diagnostic State.

6.2.9 Clearing Diagnostic Test Results

On-Demand DTCs logged during the execution of a diagnostic test shall be cleared automatically when an ECU executes another Diagnostic Test or returns to the Operational State. Refer to **Section 9.0, DIAGNOSTIC TROUBLE CODE (DTC) REQUIREMENTS** for a detailed description of On-Demand DTCs.

7. DIAGNOSTIC COMMAND MODE REQUIREMENTS

7.1 PURPOSE

This section explains what a Diagnostic Command is, reviews how it is used, and identifies its message structure.

7.2 DIAGNOSTIC COMMAND DEFINITION

A Diagnostic Command is a message that gets issued by a tester to an ECU and instructs that ECU to perform a specific function. The purpose of the diagnostic command is to provide a tester with the capability to enable and disable individual circuits within an ECU. All Diagnostic Commands that an ECU will support shall be defined in the ECUs SSDS.

NOTE: The definitions of all Commands must be approved by the AVT EESE WWDO Vehicle Diagnostics representative. All approved Commands and their descriptions are maintained in a FORD Worldwide Corporate Master Reference DataBase - MRDB, maintained by the WWDO Vehicle Diagnostics organization.

7.2.1 Diagnostic Command Message Information

A ***Diagnostic Command*** message is recognized by a unique mode number (***$B1***) in Byte four of the message structure. For a complete definition of the Diagnostic Command message structure and for understanding about ECU responses to command requests under various operating conditions, refer to **Appendix A, MESSAGE STRUCTURES & RESPONSES.**

7.3 DIAGNOSTIC COMMAND CLASSIFICATIONS

The following classifications of Diagnostic Commands (Table 7-1) may be used to control ECU outputs and functions. NOTE: the ECU should incorporate an internal time-out of the functions output; if applicable.

Table 7-1 Command Data Type Classifications

Command	Classification	Data Size
Direct		0 Bytes
State Encoded	SED	1-4 Byte
Bit Mapped	BMP	1-4 Bytes
Numeric	NUM	1-4 Bytes

7.3.1 Direct Commands

Direct Commands may be used to control specific ECU functions without specifying extra instructional data in the ***Diagnostic Command*** message's four Optional Command Data bytes (Bytes 7 to 10). A single Direct Command (e.g., Latching PID Command) can only be used to enable, disable, or toggle one ECU function (i.e., one direct command per function).

7.3.2 State-Encoded (SED) Commands

State-Encoded Commands use a hexadecimal value in the ***Diagnostic Command*** message's first Optional Command Data byte (Byte 7) to control up to 256 (***$00*** - ***$FF***) unique functions.

7.3.3 Bit-Mapped (BMP) Commands

Bit-Mapped Commands may be used to control up to 32 ECU outputs or functions based on the state, "1" - Enable/Activate or "0" - Disable/Deactivate, of the bits in each of the ***Diagnostic Command*** message's Optional Command Data bytes.

7.3.4 Numeric (NUM) Commands

Numeric Commands may be used to control specific ECU functions based on the Numeric value of data (BCD or Hex) supplied in the ***Diagnostic Command*** message's Optional Command Data bytes.

7.4 DIAGNOSTIC COMMAND ENTRY CRITERIA

Before a module shall be permitted to execute a Diagnostic Command, the following entry requirements shall be satisfied.

- The ECU shall be in the Diagnostic State.

7.5 DIAGNOSTIC COMMAND EXIT CRITERIA

The Diagnostic Command mode may return to one of the following states:

- Diagnostic State
- Operational State

7.5.1 Exiting the Diagnostic Command Mode to the Diagnostic State

Upon completing the execution of a Diagnostic Command, an ECU shall automatically return to the Diagnostic State.

NOTE: Toggle Commands can be used to return to the Diagnostic State. This can occur if Latching PID Command or the Personality Configuration Command ($0150) are applied.

7.5.2 Exiting the Diagnostic Command Mode to the Operational State

If any of the following conditions occur, execution of the Diagnostic Command will be terminated and the ECU shall return to the Operational State:

- The ECU receives a ***Request Return to Operational State*** message.
- The ECU doesn't receive ANY Node-to-Node message (supported or unsupported) within 5.0 (0.0, +1.0) seconds of either the last message received or initial entrance into diagnostic state.
- The ECU satisfies any of the exit criteria specified in the ECUs SSDS.

7.6 MANDATORY DIAGNOSTIC COMMANDS

There are no mandatory Diagnostic Commands required to be supported by an ECU.

7.7 ADDITIONAL COMMAND INFORMATION

The following subsections provide information about specialized commands.

7.7.1 Latching PID Command ($0001) → Optional

The ***Latching PID Command*** directs an ECU to monitor its I/O lines for activity and, upon the detection of a change-of-state (high-to-low or low-to-high), to latch the final level of the first transition into the I/O line's appropriate input or output PID. The ***Latching PID Command*** is used to verify that ECU I/O lines can properly change state. The I/O lines supported will be defined in the ECUs SSDS. If the first I/O transition is low-to-high, the bit value latched into the PID shall be high -- "***1***". If the first I/O transition is high-to-low, the bit value latched into the PID shall be low -- "***0***".

7.7.1.1 Latching PID Command Usage

A Tester will retrieve existing (pre-latched) PID information from an ECU prior to sending a ***Latching PID Command*** request to the ECU. The information retrieved will be used for a later comparison. Upon receiving the

first Latching PID Command, an ECU shall clear (i.e., zero out) PID ***$0201***. After the ***Latching PID Command*** has been issued to an ECU and PID ***$0201*** cleared, a technician will toggle all ECU subsystem inputs deemed appropriate. The ECU shall latch all initial I/O transitions and increment PID ***$0201*** each time an I/O transition is sensed, as defined in the ECUs SSDS. The tester will re-issue the ***Latching PID Command*** to inform the ECU to stop latching input transitions and incrementing PID ***$0201***. To verify that the ECU sensed a transition on each of the I/O lines toggled, the final latched PID data is compared with the initial pre-latched PID data by the tester and/or technician. Figure 7-1 shows how the Latching PID Command fits into an evaluation of an ECU's I/O functionality.

7.7.1.2 Exiting the Latching PID Command

An ECU shall continue to monitor and latch PID change-of-state information until it receives a second ***Latching PID Command*** request. Upon receiving a second ***Latching PID Command***, the ECU shall stop monitoring its I/O lines and return the PIDs associated with those I/O lines to their "normal state" (i.e., no more updating of PID ***$0201***), as defined in the ECUs SSDS. The "normal state" indicates the current state of the ECUs I/O lines.

An ECU shall also stop monitoring its I/O lines and return the PIDs to their "normal state" (i.e., no more updating of PID ***$0201***), if any of the Diagnostic Command entry criteria is violated, as defined in the ECUs SSDS.

7.7.1.3 Latching PID Time Requirements

The time required to toggle, monitor and verify the appropriate ECU I/O lines with the ***Latching PID Command*** is dependent primarily on the test environment (operator toggling sequence) and is unique for each ECU and implementation.

7.8 ECU SPECIFIC COMMANDS

These commands are unique for each ECU, contact your local AVT EESE WWDO Vehicle Diagnostics representative for a definition of any ECU specific commands.

7.9 SUPPLIER RESERVED COMMAND RANGE

The range of Commands from ***$F000*** through ***$FFFF*** are reserved for use by ECU suppliers, with the exception of ***$FACE (refer to Section 12 - Gateway Mode Diagnostic Requirements)*** which is reserved. Supplier reserved commands do not need to be approved by the AVT EESE WWDO Vehicle Diagnostic representative. All supplier reserved commands shall be defined in the ECUs SSDS. NOTE: Supplier reserved commands are not supported by diagnostic service tools (NGS, SBDS, FDS-2000, & EOL).

EXAMPLE: Assume that the service technician has made the appropriate selection from the tester to initiate the sequence of events and that the ECU has already entered the Diagnostic State.

Step	Message or Responsibility			Event
n				
n+1	Tester	ECU	-	PID requests for existing (pre-latched) data
+2	Tester	ECU	-	PID responses: existing PID data
+3	Tester		-	saves existing PID data for later comparison.
+4	Tester	ECU	-	Latching PID command (**$0001**)
+5	Tester	ECU	-	General response (affirmative)
+6		ECU	-	monitors I/O lines for state transitions.
+7	Technician		-	Toggles switches, etc.: creates I/O transitions.
+8		ECU	-	increments PID **$0201** each time an I/O transition occurs
+9		ECU	-	sets the I/O line's PID bits to the final level of the initial transition
+10	Technician		-	notifies Tester that all switches have been toggled
+11	Tester	ECU	-	PID request for latched data
+12	Tester	ECU	-	PID response: latched data
+13	Tester	ECU	-	Latching PID command (to quit)
+14	Tester	ECU	-	General response (affirmative)
+15		ECU	-	returns PID bits to current state of I/O lines (unlatched)
+16	Tester or Technician		-	compares latched PID data to pre-latched data (returned in step n+2) to determine whether lines that should have been toggled did not change state.

Figure 7-1 How Latching PID Command Fits into an Evaluation of ECU's I/O Functionality

8. PARAMETER IDENTIFIER (PID) REQUIREMENTS

8.1 PURPOSE

This section describes the potential for using Parameter Identifiers (PIDs), explains PID types, how PID data is accessed, and defines mandatory PIDs that all serial communications link ECUs shall support.

NOTE: The definitions of all PIDs must be approved by the AVT EESE WWDO Vehicle Diagnostics representative. All approved PIDs and their descriptions are maintained in a FORD Worldwide Corporate Master Reference DataBase - MRDB.

8.2 PID REQUIREMENTS

An ECU shall be capable of returning PID information to a Tester while executing in the following states and modes:

- Operational State
- No Stored Codes Logging Mode
- Diagnostic State
- Diagnostic Test Mode
- Command Mode
- Memory Transfer Mode
- Gateway Mode

NOTE: For direction about responses to PID requests in the various operating states, refer to **Appendix A, MESSAGE STRUCTURES & RESPONSES.**

8.3 PID CLASSIFICATIONS and CAPABILITIES

A PID may provide information on discrete I/O signals, analog I/O signals, time duration, activity counts, or information about the configuration of the ECU. The information specified by a PID shall be defined in the appropriate SSDS for that ECU.

The amount of data that an ECU may return to a Tester per PID request can range from one to four Bytes. PID information is subdivided into *data type classifications* as shown in Table 8-1. The following subsections provide an overview of the information accessible via the PID classifications.

Table 8-1 PID Data Type Classifications

Type	Classification	Data Size
ASCII	ASC	4 Bytes
Binary Coded Decimal	BCD	4 Bytes
State Encoded	SED	1 Byte
Bit Mapped	BMP	4 Bytes
Numeric	NUM	4 Bytes
Packeted	PKT	4 Bytes

8.3.1 ASCII (ASC) PIDs

ASCII PIDs are used to provide information such as File Names, Vehicle Identification (VIN) Numbers (e.g., PIDs ***$E300*** to ***$E307***), Module Serial Numbers (e.g., PIDs ***$E221*** to ***$E226***), etc. encoded using standard ASCII characters between **$00** and **$7F**.

8.3.2 Binary-Coded-Decimal (BCD) PIDs

Binary-Coded-Decimal (BCD) PIDs are used to provide information, such as Times, Dates, Serial Numbers (e.g., PID ***$E220***, PIDs ***$E231*** to ***$E236***), etc. encoded using BCD values. BCD PIDs can range from one to eight Nibbles in length (i. e., one to eight BCD digits per PID) and contain only values for the decimal digits from zero to nine.

8.3.3 State-Encoded (SED) PIDs

State-Encoded PIDs use hexadecimal values to represent unique states. Each State-Encoded PID is one byte in length and can be up to "**N**" bits in length ($1 \leq \mathbf{N} \leq 8$) and can represent up to $\mathbf{2^N}$ unique states. For example, a 4-bit SED PID can be used to represent the months of the year (**01h** - January through **0Ch** - December).

8.3.4 Bit-Mapped (BMP) PIDs

Bit-Mapped PIDs are used to provide information on the discrete state of ECU I/O signals (enabled or disabled, opened or closed, shorted-to-ground, shorted-to-battery, open-circuited, etc.,) or to show whether an ECU supports a specific function. Bit-Mapped PIDs specify information based on the state, "1" or "0", of the bits in each PID data Byte.

8.3.5 Numeric (NUM) PIDs

Numeric PIDs are used to represent signed or unsigned data that has a defined range of values and a specific resolution that can be represented through hexadecimal encoding.

8.3.5.1 Signed Numeric PIDs

Signed Numeric PIDs are used to represent data which may have a negative or positive value such as temperature. Signed Numeric PIDs can be from one to four Bytes in length and are encoded using *2's* Complement arithmetic. If the Most Significant Bit of the PID data is a "1" , then the data is a negative value determined by computing the data's *2's* Complement. Signed Numeric PIDs have a defined range of values and an assigned resolution per count that is used to convert the computed *2's* Complement value to an actual engineering value.

8.3.5.2 Unsigned Numeric PIDs

Unsigned Numeric PIDs are used to represent data that takes on a range of positive values such as Vehicle Speed or the Number of Continuous DTCs being stored by the ECU. Unsigned Numeric PIDs have a defined range and are assigned a resolution per count and offset that is used to convert the PID value to an actual engineering value.

8.3.6 Packeted (PKT) PIDs

A Packeted PID is actually a packet of data that is a combination of "related" data type from other PIDs. Packeted PIDs can be used to increase the retrieval rate of PID data that would otherwise be retrieved through a series of sequential requests for individual PIDs.

8.4 PID SCALING AND MASKING CAPABILITY (OPTIONAL)

PID scaling and/or masking information is requested using mode $24. The response information is sent in mode $64 from the ECU.

Scaling is used only for singed and unsigned numeric PID types and masking is used only for bit-packed types (See modes $24 and $64 of Appendix A for a table showing the message structure for these modes).

The following table describes the encoding of the "Parameter Information Byte" byte 8 of mode $64:

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
TYPE OF PARAMETER : 0000 = UNSIGNED NUMERIC 0001 = SIGNED NUMERIC 0010 = BIT PACKED PARAMETER 0011 = BIT MAP W/MASK 0100 = BCD 0101 = STATE ENCODED 0110 = ASCII CHARACTER 0111 = FLOATING POINT 1000 = PACKET 1001 THRU 1111 RESERVED				PID SIZE : NUMBER OF BYTES 0000 = RESERVED 0001 = 1 BYTE 0010 = 2 BYTES 0011 = 3 BYTES 0100 = 4 BYTES 0101 THRU 1111 = RESERVED			

The high order nibble of the Information Byte defines the type of information encoding that is used to represent the parameter. The low order nibble can be read directly to determine the number of bytes used to represent the parameter; this is the parameter size. The size ma be 1,2,3 or 4 bytes.

Byte 9 of mode $64 contains either scaling or masking information. The number represented in this byte specifies scaling by indicating the binary point location. The binary point may be moved 127 places to the right (FF hex) or 128 places to the left (00hex). The binary point is initially assumed to be to the right of the LSB without this information.

Byte 9 can also contain mask information that specifies which bits of the PID are supported for the current application. For bit mapped parameters which contain two bytes of bit mapped information, the second mask byte is reported in byte 10 of the mode $64 message.

8.5 SUPPLIER-RESERVED PID RANGE

The range of PIDs from ***$C900*** through ***$C9FF*** are reserved for use by ECU suppliers, with the exception of ***$C9F8 through $C9FF*** which are reserved only if configuration is implemented. Supplier reserved PIDs do not need to be approved by the AVT EESE WWDO Vehicle Diagnostic representative. All supplier reserved PIDs shall be defined in the SSDS for that ECU. Note: The service tools (NGS, SBDS, FDS-2000) do not support supplier reserved PIDs.

8.6 PID ACCESS

A Tester shall retrieve PID information from an ECU through a ***Request Parameter by PID*** message. The ECU shall return the requested PID data via the ***Report Parameter by PID*** message. In the event that the ECU cannot concur with a PID request, the ECU shall respond according to the message response guidelines specified in **Appendix A, MESSAGE STRUCTURES & RESPONSES**. NOTE: ECU I/O signal information returned in a PID shall indicate the current status of the ECU signals at the time of the request.

8.7 MANDATORY PIDs

Table 8-2 defines the mandatory PIDs that shall be supported by all ECUs that support SCP or ISO-9141-FORD serial communications. Contact your local AVT EESE WWDO Vehicle Diagnostic representative for a definition of any mandatory PID.

Table 8-2 Mandatory Supported PIDs

PID	Description	Classification	Size
$0200	Number of Continuous DTCs	NUM	1 Byte
$0202	Number of DTCs from most recent test	NUM	1 Byte
$D100	ECU Operating State / Mode	SED	1 Byte
$E200	Software Version Number	PKT	3 Bytes
$E217	Part Number Identification Base	PKT	4 Bytes
$E219	Part Number Identification Suffix	PKT	2 Bytes
$E21A	Part Number Identification Prefix	PKT	4 Bytes

8.7.1 Number of Continuous DTCs (PID $0200)

The ***Number of Continuous DTCs*** PID instructs an ECU to return the number of Continuous DTCs currently being stored by the ECU.

8.7.2 Number of Trouble Codes Set Due to Diagnostic Test (PID $0202)

The ***Number of Trouble Codes Set Due to Diagnostic Test*** PID instructs the ECU to return the number of On-Demand DTCs generated during the most recent diagnostic test executed by the ECU.

8.7.3 ECU Operating State (PID $D100)

The ***ECU Operating State*** PID instructs an ECU to return a value that identifies its current operating state/mode as defined in Table 8-3.

Table 8-3 ECU Operating State (PID $D100)

Value	Operating State / Mode
$01	Operational State
$02	Diagnostic State
$03	On-Demand Self-Test Mode
$04	Wiggle Test Mode
$05	General Test Mode
$06	Command Mode
$07	Memory Transfer Mode
$09	No Stored Codes Logging Mode
$85	Power Seat Calibration Test Mode
$FF	Invalid Data

NOTE: The Command Mode value is not required to be supported by an ECU unless it incorporates a command that executes longer than 55 milliseconds or the Latching PID Command.

8.7.4 Software Version Number (PID $E200)

The ***Software Version Number*** PID instructs an ECU to return the software revision level and release date as defined in Table 8-4.

Table 8-4 Software Version Number (PID $E200)

Byte 1		Byte 2		Byte 3	
Bits 7-4	*Revision Level*	*Bits 7-0*	*Day of Month*	*Bits 7-0*	*Year*
	0 ($0) through 15 ($F)		01($01) through 31($1F)		00 ($00) = 1900 through 255 ($FF) = 2155
Bits 3-0	*Month of Year*				
	0001 = January				
	0010 = February				
	0011 = March				
	0100 = April				
	0101 = May				
	0110 = June				
	0111 = July				
	1000 = August				
	1001 = September				
	1010 = October				
	1011 = November				
	1100 = December				

8.7.5 Part Number Identification Base (PID $E217)

The base part number PID specifies the FORD part number's "group" and "detail" information in four Bytes of data as specified in Table 8-5. The part number information supplied by this PID supports the worldwide FAO part

number identification systems. In order to accommodate differences in the definition of a FORD part number by the responsible FORD organizations, the assignment of the base part number PIDs characters must begin with Byte 1 and progress through Byte 4.

EXAMPLE:
What is the encoding for an airbag module with the base part number 14B056 ?

14	=	$14	BCD encoding
B	=	$0B	Alphanumeric encoding
0	=	$00	Alphanumeric encoding
56	=	$56	BCD encoding

What is the encoding for the anti-lock braking module with base part number 2M110 ?

2	=	$02	BCD encoding
M	=	$16	Alphanumeric encoding
1	=	$01	Alphanumeric encoding
10	=	$10	BCD encoding

8.7.6 Part Number Identification Prefix (PID $E21A)

The prefix part number PID specifies the year/decade, product line, and design responsibility in four bytes of data as specified in Table 8-6. The part number information supplied by this PID supports the worldwide FAO part number identification systems. The data that an ECU shall return in response to a request for this PID is a direct ASCII transformation of the Codes listed for the Year/Decade, Product Line and Design Responsibility. PID $E21A - Part Number identification Number - replaces PID $E218 for Model Year 1999 and beyond vehicle programs.

EXAMPLE:
What is the Part Number Prefix for a fuel pump control module released by the Electrical and Fuel Handling Division for a Jaguar X200 ?

Decade / Year	-	2001	-	1
Product Line	-	Jaguar X200	-	**R8**
Design Resp.	-	Elec. Fuel & Handling Div.	-	**U**

Part Number Prefix = **1R8U**

The encoded data that shall be returned by an ECU in response to a Tester request for PID $E21A is as follows:

1	-	Byte 1	=	$31 - ASCII value for '1'
R	-	Byte 2	=	$52
8	-	Byte 3	=	$38
U	-	Byte 4	=	$55 - ASCII value for 'U'

Table 8-5 Base Part Number Structure (PID $E217)

CODING FOR BASE PART NUMBER

	BYTE ONE								
	7	6	5	4		3	2	1	0
	BCD					BCD			
0	0	0	0	0		0	0	0	0
1	0	0	0	1		0	0	0	1
2	0	0	1	0		0	0	1	0
3	0	0	1	1		0	0	1	1
4	0	1	0	0		0	1	0	0
5	0	1	0	1		0	1	0	1
6	0	1	1	0		0	1	1	0
7	0	1	1	1		0	1	1	1
8	1	0	0	0		1	0	0	0
9	1	0	0	1		1	0	0	1

$A thru $F are N/A.

$A thru $F are N/A.

Code	BYTE TWO								
	7	6	5	4	3	2	1	0	
	ALPHANUMERIC								
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	2
3	0	0	0	0	0	0	1	1	3
4	0	0	0	0	0	1	0	0	4
5	0	0	0	0	0	1	0	1	5
6	0	0	0	0	0	1	1	0	6
7	0	0	0	0	0	1	1	1	7
8	0	0	0	0	1	0	0	0	8
9	0	0	0	0	1	0	0	1	9
A	0	0	0	0	1	0	1	0	
B	0	0	0	0	1	0	1	1	
C	0	0	0	0	1	1	0	0	
D	0	0	0	0	1	1	0	1	
E	0	0	0	0	1	1	1	0	
F	0	0	0	0	1	1	1	1	
G	0	0	0	1	0	0	0	0	
H	0	0	0	1	0	0	0	1	
I	0	0	0	1	0	0	1	0	
J	0	0	0	1	0	0	1	1	
K	0	0	0	1	0	1	0	0	
L	0	0	0	1	0	1	0	1	
M	0	0	0	1	0	1	1	0	
N	0	0	0	1	0	1	1	1	
O	0	0	0	1	1	0	0	0	
P	0	0	0	1	1	0	0	1	
Q	0	0	0	1	1	0	1	0	
R	0	0	0	1	1	0	1	1	
S	0	0	0	1	1	1	0	0	
T	0	0	0	1	1	1	0	1	
U	0	0	0	1	1	1	1	0	
V	0	0	0	1	1	1	1	1	
W	0	0	1	0	0	0	0	0	
X	0	0	1	0	0	0	0	1	
Y	0	0	1	0	0	0	1	0	
Z	0	0	1	0	0	0	1	1	

NOTE: $26 through $3F are N/A.

Code	BYTE THREE								
	7	6	5	4	3	2	1	0	
	ALPHANUMERIC								
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	2
3	0	0	0	0	0	0	1	1	3
4	0	0	0	0	0	1	0	0	4
5	0	0	0	0	0	1	0	1	5
6	0	0	0	0	0	1	1	0	6
7	0	0	0	0	0	1	1	1	7
8	0	0	0	0	1	0	0	0	8
9	0	0	0	0	1	0	0	1	9
A	0	0	0	0	1	0	1	0	
B	0	0	0	0	1	0	1	1	
C	0	0	0	0	1	1	0	0	
D	0	0	0	0	1	1	0	1	
E	0	0	0	0	1	1	1	0	
F	0	0	0	0	1	1	1	1	
G	0	0	0	1	0	0	0	0	
H	0	0	0	1	0	0	0	1	
I	0	0	0	1	0	0	1	0	
J	0	0	0	1	0	0	1	1	
K	0	0	0	1	0	1	0	0	
L	0	0	0	1	0	1	0	1	
M	0	0	0	1	0	1	1	0	
N	0	0	0	1	0	1	1	1	
O	0	0	0	1	1	0	0	0	
P	0	0	0	1	1	0	0	1	
Q	0	0	0	1	1	0	1	0	
R	0	0	0	1	1	0	1	1	
S	0	0	0	1	1	1	0	0	
T	0	0	0	1	1	1	0	1	
U	0	0	0	1	1	1	1	0	
V	0	0	0	1	1	1	1	1	
W	0	0	1	0	0	0	0	0	
X	0	0	1	0	0	0	0	1	
Y	0	0	1	0	0	0	1	0	
Z	0	0	1	0	0	0	1	1	

NOTE: $26 through $3F are N/A.

	BYTE FOUR								
	7	6	5	4		3	2	1	0
	BCD					BCD			
0	0	0	0	0		0	0	0	0
1	0	0	0	1		0	0	0	1
2	0	0	1	0		0	0	1	0
3	0	0	1	1		0	0	1	1
4	0	1	0	0		0	1	0	0
5	0	1	0	1		0	1	0	1
6	0	1	1	0		0	1	1	0
7	0	1	1	1		0	1	1	1
8	1	0	0	0		1	0	0	0
9	1	0	0	1		1	0	0	1

$A thru $F are N/A.

$A thru $F are N/A.

Table 8-6 Part Number Identification Prefix (PID $E21A)

Byte 1 - Decade / Year			Bytes 2 & 3 - Product Line			Byte 4 - Design Responsibility		
Code	Hex	Year	Code	Hex	Definition	Code	Hex	Definition
1	$31	2001	S1	$5331	Aspire	1	$31	VC1
2	$32	2002	S2	$5332	Contour	2	$32	VC2
3	$33	2003	S3	not used		3	$33	VC3
4	$34	2004	S4	$5334	Escort	4	$34	VC4
5	$35	2005	S5	$5335	BE146	5	$35	VC5
6	$36	2006	S6	$5336	Fiesta	6	not used	
7	$37	2007	S7	$5337	Mondeo	7	not used	
8	$38	2008	S8	$5338	Probe	8	$38	Electric Vehicle
9	$39	2009	S9	$5339	Mystique	9	not used	
A	$41	2010	M1	$4D31	Tracer	A	$41	AVT Core Operations
B	$42	2011	F1	$4631	Taurus	B	not used	
C	$43	2012	F2	$4632	Windstar	C	$43	Chassis Operations
D	$44	2013	F3	$4633	Continental	D	$44	Overseas Product Engrg (AVT)
E	$45	2014	F4	$4634	Sable	E	$45	Engine Engrg, Engine Product & Mfg. Engrg
F	$46	2015	F5	$4635	Villager	F	$46	Electronics Engrg Operations (ACD)
G	$47	2016	F6	$4636	T-Bird	G	not used	
H	$48	2017	F7	$4637	EN158	H	$48	Climate Control Ops.
I	not used		F8	$4638	FW178	I	not used	
J	$4A	2018	L1	$4C31	Expedition/Bronco	J	$4A	FCSD Parts & Access
K	$4B	2019	L2	$4C32	Explorer	K	$4B	Import Release Sect.
L	$4C	2020	L3	$4C33	Light Truck (F150 Thru F250)	L	$4C	FCSD Power Products
M	$4D	2021	L4	$4C34	Maverick 4 x 4	M	$4D	Performance Ops & Special Vehicle Ops
N	$4E	2022	L5	$4C35	Ranger	N	not used	
O	not used		L6	$4C36	Aerostar	O	not used	
P	$50	2023	L7	$4C37	Navigator	P	$50	Trans. & Axle Product & Mfg. Engrg (Auto. Trans)
Q	not used		R1	$5231	Aston Martin	Q	$51	Diesel Engine Engrg
R	$52	2024	R2	$5232	Falcon	R	$52	Trans & Axle Product & Mfg. Engrg (Manual Trans.)
S	not used		R3	$5233	Mustang	S	$53	Light & Heavy Truck Engrg, Truck Special Order Dept.
T	not used		R4	$5234	Scorpio	T	$54	Electrical/Electronic Systems Engrg (AVT)
U	not used		R5	$5235	Fairlane	U	$55	Electrical & Fuel Handling Div.
V	not used		R6	$5236	LTD	V	$56	Domestic Special Order Engrg. Section (Car Special Order) & (Special Vehicle Engrg.)
W	not used		R7	not used		W	$57	Trans. & Axle Product & Mfg. Engrg (Axle & Driveline)
X	$58	1999	R8	$5238	Jaguar X200	X	$58	Plastic & Trim Products Ops (ACD)
Y	$59	2000	R9	$5239	Mark VIII	Y	not used	

Byte 1 - Decade / Year			Bytes 2 & 3 - Product Line			Byte 4 - Design Responsibility		
Z	not used		W1	$5731	Town Car	Z	$5A	FCSD Product Analysis
			W2	$5732	Cougar			
			W3	$5733	Grand Marquis			
			W4	$5734	D/EW98			
			W5	$5735	EW171			
			W6	$5736	T-Bird RWD			
			W7	$5737	EN114 Crown Vic			
			W8	$5738	Jaguar X100			
			W9	$5739	Jaguar X300			
			C1	$4331	Transit			
			C2	$4332	Econoline			
			C3	$4333	Medium Truck (F250 over 8500 GVW Thru F900, and B-Series			
			C4	$4334	Heavy Truck (L-Series and Cargo-CF Series)			
			U1	$5531	Power Products			
			U2	$5532	Motorcraft Brand			
			U3	$5533	Outside Sales			
			U4	$5534	Vehicles Imported from FOE or parts designed by US for FOE vehicles			
			L8	$4C38	Joint venture between Ford and Mazda.			

8.7.7 Part Number Identification Suffix (PID $E219)

The suffix part number PID specifies the ECU's basic design code, alternate part code, and change level as defined in Table 8-7. The information supplied by this PID supports all FAO (NAAO and IAO) part number identification systems.

Table 8-7 Part Number Identification Suffix (PID $E219)

Code: BASIC DESIGN **Code: ALTERNATE PART** **Code: CHANGE LEVEL**

* = No Alternate

BYTE 1 BYTE 2

	7	6	5	4	3	2	1
A	0	0	0	0	0	0	0
B	0	0	0	0	0	0	1
C	0	0	0	0	0	1	0
D	0	0	0	0	0	1	1
E	0	0	0	0	1	0	0
F	0	0	0	0	1	0	1
G	0	0	0	0	1	1	0
H	0	0	0	0	1	1	1
J	0	0	0	1	0	0	0
K	0	0	0	1	0	0	1
L	0	0	0	1	0	1	0
M	0	0	0	1	0	1	1
N	0	0	0	1	1	0	0
P	0	0	0	1	1	0	1
R	0	0	0	1	1	1	0
S	0	0	0	1	1	1	1
T	0	0	1	0	0	0	0
U	0	0	1	0	0	0	1
V	0	0	1	0	0	1	0
X	0	0	1	0	0	1	1
Y	0	0	1	0	1	0	0
Z	0	0	1	0	1	0	1

	0	7	6	5
0*	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

	4	3	2	1	0
A	0	0	0	0	0
B	0	0	0	0	1
C	0	0	0	1	0
D	0	0	0	1	1
E	0	0	1	0	0
F	0	0	1	0	1
G	0	0	1	1	0
H	0	0	1	1	1
J	0	1	0	0	0
K	0	1	0	0	1
L	0	1	0	1	0
M	0	1	0	1	1
N	0	1	1	0	0
P	0	1	1	0	1
R	0	1	1	1	0
S	0	1	1	1	1
T	1	0	0	0	0
U	1	0	0	0	1
V	1	0	0	1	0
X	1	0	1	0	0
Y	1	0	1	0	1
Z	1	0	1	1	0

Examples:

AA = 0000000 0000 00000

AZ = 0000000 0000 10110

ZZ = 0010101 0000 10110

A1B = 0000000 0001 00001

C2Z = 0000010 0010 10110

8.8 SPECIAL PURPOSE PIDs

Table 8-8 defines PIDs that shall be supported by an ECU if specified in the ECUs SSDS. Contact your local AVT EESE WWDO Vehicle Diagnostics representative for a definition of any of these PIDs.

Table 8-8 As Required PIDs

PID	Description	Classification	Size
$0201	Number of I/O Transitions	NUM	1 Byte
$C9F8 through $C9FF	Module Configuration	ECU Specific	4 Bytes each
$D12D	Node Address	SED	1 Byte
$E221 through $E226	Module Serial Number	ASC	4 Bytes each
$E231 through $E236	Module Serial Number	BCD	4Byte each
$E300 through $E307	Vehicle Identification Number	ASC	4 Bytes each

8.8.1 Number of I/O Transitions (PID $0201)

The Number of I/O Transitions PID instructs the ECU to return the number of I/O transitions detected during the Wiggle Test, the Latching PID Command sequence, or any other activity identifying its usage as defined in the ECUs SSDS.

8.8.2 Module Configuration (PIDs $C9F8 thru $C9FF)

The Module Configuration PIDs defines the module configuration data as defined in the ECUs SSDS. Note: Unique PIDs are defined for module configuration method 1 and 2.

8.8.3 Node Address (PID $D12D)

This PID is to identify the satellite module being communicated via the gateway module.

8.8.4 Serial Number (PIDs $E221 thru $E226 & $E231 thru $E236)

The Serial Number PIDs instructs the ECU to return ECU serial number as defined in the ECUs SSDS. Note: Unique PIDs are defined for ACSII and BCD formats.

8.8.5 Vehicle Identification Number (PIDs $E300 thru $E307)

The Vehicle Identification Number PIDs instructs the ECU to return the vehicle identification number as defined in the ECUs SSDS.

8.9 ECU SPECIFIC PIDs

PIDs specific to each ECU shall be defined in the ECUs SSDS. Contact your local AVT EESE WWDO Vehicle Diagnostics representative for a definition of any ECU specific PIDs.

9. DIAGNOSTIC TROUBLE CODE (DTC) REQUIREMENTS

9.1 PURPOSE

This section defines the requirements associated with Diagnostic Trouble Codes (DTCs).

Each DTC is a 2-Byte hexadecimal number that is used to report an ECU failure. The assignment of a DTC shall attempt to isolate an ECU failure to a single field replaceable unit (i.e. air compressor, shock, switch, etc.). NOTE: The definitions of all DTCs must be approved by the AVT EESE WWDO Vehicle Diagnostics representative. All approved DTCs and their descriptions are maintained in a FORD Worldwide Corporate Master Reference DataBase - MRDB.

9.2 DTC CLASSIFICATIONS

There are two classifications of DTCs that an ECU shall support:

- Continuous DTCs
- On-Demand DTCs

9.2.1 CONTINUOUS DTCS

Continuous DTCs are diagnostic trouble codes logged by an ECU upon the detection of on-board and/or I/O ECU faults while it is functioning in the Operational State. Continuous DTCs are specific to an ECU and shall be defined in the appropriate ECUs SSDS. Refer to **Section 10.0, NETWORK COMMUNICATIONS DIAGNOSTIC REQUIREMENTS** for a description of continuous network DTCs. NOTE: The only Continuous DTCs that can be logged in the Diagnostic State are Configuration DTCs (A141 and A477). Refer to **Section 11.0, MEMORY ACCESS & DATA TRANSFER REQUIREMENTS.**

9.2.1.1 Time Period for Maintaining Continuous DTCs

Each Continuous DTC logged by an ECU shall be maintained for a time period ranging from ***no less than 80 to no more than 128 ECU power-up cycles*** after the most recent occurrence of the fault which caused the Continuous DTC to be logged.

9.2.1.2 Continuous DTC Clearing Counters

When an ECU detects a fault and logs a Continuous DTC, it shall log a count in the " clearing counter " in conjunction with the fault. A clearing counter is a counter used to track the number of ECU power-up cycles since the most recent detection of a fault for which the ECU has logged a Continuous DTC. Each time an ECU powers-up, the clearing counters for all logged Continuous DTCs shall be updated. The reason for logging and maintaining a clearing counter is to limit how long a Continuous DTC will remain logged by an ECU when the fault which caused the Continuous DTC to be logged is not detected on a continuous basis.

When an ECU detects a fault that already has a Continuous DTC logged, the clearing counter for that Continuous DTC shall be reset to its initial value. When a clearing counter's time period expires (between 80 and 128 ECU power-up cycles after the last detection of a corresponding fault), the Continuous DTC shall be cleared.

9.2.1.3 Continuous DTC Memory Requirements

An ECU shall be capable of logging all Continuous DTCs and their associated clearing counters when the number of DTCs to be supported by an ECU is less than 15. An ECU shall be capable of logging a minimum of 15 Continuous DTCs and their associated clearing counters when the number of DTCs to be supported by an ECU is 15 or more. Any deviation from these requirements shall be specified in the SSDS (Part II) for that ECU.

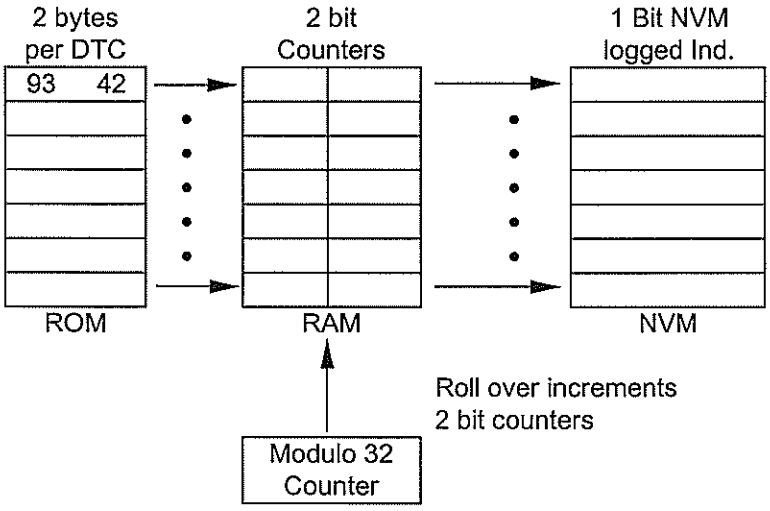
9.2.1.3.1 Guidelines for Continuous DTC Updating Of All DTCs

When storing all of the continuous DTCs, the DTC Code should be stored in ROM, the Aging Counter should be stored in RAM, and the DTC logged indicator (1 bit / DTC) should be stored in Non Volatile Memory (NVM - e.g., EEPROM)

Each DTC should be mapped to a 2 bit count (modulo 3 DTC counter) and a 1bit indicator in NVM. A modulo 32 counter (main counter) should be setup to increment the 2 bit DTC counters. On each power cycle, the main counter should increment. When this counter transitions from 31 to 0, the DTC counters should be incremented. When the DTC counter transitions from 3 (11 binary) to 0 (zero), the DTC indicator bit in NVM should be cleared. Upon receipt of a Clear Codes request, all RAM counters and NVM indicators are to be cleared.

Upon detection of a particular fault (existing or new) the corresponding DTC will be logged as follows:
1. The DTC counter (2 bit counter in RAM) will be reset
2. The logged indicator bit in NVM will be set.

Table 9-1 Example of Memory when Storing all Continuous DTCs



With the above method, the DTCs can be arranged in the ECU such that the DTC with the highest priority, or most critical, is read out and displayed in descending order.

9.2.1.3.2 Guidelines for Continuous DTC Updating Of 15 DTCs

When storing only 15 of the continuous DTCs, the DTC Code should be stored in ROM, the Aging Counter should be stored in RAM, and the DTC logged (2 bytes / DTC) should be stored in Non Volatile Memory (NVM - e.g., EEPROM)

Each DTC NVM location should be mapped to a 2 bit counter (DTC aging counter). A modulo 32 counter (main counter) should be setup to increment the 2 bit DTC counters. On each power cycle the main counter should increment. When this counter transitions from 31 to 0, the DTC counters should be incremented. When the DTC counter transitions from 3 (11 binary) to 0 (zero), the DTC indicator bit in NVM should be cleared. Upon receipt of a Clear Codes request, all RAM counters and NVM indicators are to be cleared.

Upon detection of a particular fault (existing or new) the corresponding DTC should be logged as follows:
1. The DTC counter (2 bit counter in RAM) will be reset
2. The 2 byte DTC number will be written to the first available 2 byte NVM location .

If 15 DTCs are logged and the 16th unique DTC has been detected, the last oldest DTC encountered should be replaced with the new DTC in NVM and the 2 bit aging counter reset.

Table 9-2 Example of Memory when Storing Only 15 Continuous DTCs

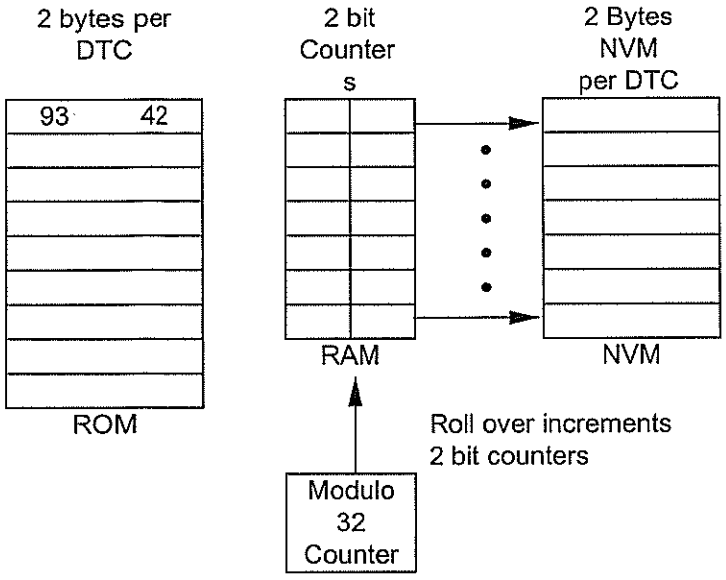


Table 9-3 Example of Memory Usage When Storing All Continuous DTCs Vs Storing Only 15 Continuous DTCs For an ECU with 24 Continuous DTCs

Resource Requirements	Store All	Store only 15
RAM (bytes)	6	4
NVM (bytes)	3	30
Search Routine	None	x bytes of RAM & ROM

9.2.1.4 Reporting Continuous DTCs

In response to a ***Request Parameter by PID*** message, with the address set to ***$0200 - Number of Continuous DTCs***, an ECU shall return a ***Report Parameter by PID*** message with the number of Continuous DTCs currently logged as the data. In response to a single ***Request Stored Codes*** message, an ECU shall report all Continuous DTCs being logged by returning as many consecutive ***Report Stored Codes*** messages as is required. Refer to Table 9-4 for an example of consecutive messages returned. Refer to **Appendix A, MESSAGE STRUCTURES & RESPONSES,** for direction about responses to stored code requests under various conditions. Note: There is no provision for a Tester to request, or for an ECU to report, less than all of the Continuous DTCs being logged by the module.

9.2.1.5 Clearing Continuous DTCs

Each ECU shall support the capability to clear Continuous DTCs via two methods:

- manually
- automatically

Table 9-4 Consecutive Messages Returned When Reporting DTCs

Continuous_DTCs Logged	Messages Returned	Method
0	1	All six Bytes of the message reserved for three DTCs shall be padded with $00
1 or 2	1	DTC(s) in Bytes 5 & 6 and/or Bytes 7 & 8; the remaining Bytes shall be padded with $00
3	1	All three DTCs shall be reported in Bytes 5 to 10 of the message
4 or 5	2	The first message shall return three DTCs. The second message shall follow the method used for one or two DTCs as explained above
6	2	Each message shall return three DTCs
n	m	Follow the same method presented above for four to six DTCs

9.2.1.5.1 Manual Clearing of Continuous DTCs

An ECU shall clear all of Continuous DTCs it has logged upon receiving a ***Request Clear Stored Codes*** message; except DTC A141 and DTC A477 (Refer to **Section 11, MEMORY ACCESS & DATA TRANSFER REQUIREMENTS**, for a definition of how DTC A141 and DTC A477 are cleared.. Once the DTCs have been cleared, the ECU shall respond to the request with a ***General Response [7F]*** message of ***00 - Affirmative.*** Note: There are no ISO-9141-FORD or SCP messages for clearing individual Continuous DTCs.

9.2.1.5.2 Automatic Clearing of Continuous DTCs

An ECU shall automatically clear a Continuous DTC when the fault which caused the Continuous DTC to be logged is not detected for a period of 80 to 128 ECU power-up cycles; except DTC A141 and DTC A477 (Refer to **Section 11, MEMORY ACCESS & DATA TRANSFER REQUIREMENTS**, for a definition of how DTC A141 and DTC A477 are cleared. This is the point where the IGNITION Key-ON clearing counter for the DTC has reached its final value. The exact number of ECU power-up cycles required before a Continuous DTC is cleared shall be specified in the ECUs SSDS.

9.2.2 ON-DEMAND DTCs

On-Demand DTCs are those that are temporarily logged (i.e., RAM) by an ECU when it detects a failure while executing any diagnostic test.

9.2.2.1 Reporting On-Demand DTCs

In response to a ***Request Parameter by PID*** message, with the address set to **$0202 - Number of Trouble Codes Set Due to Diagnostic Test**, an ECU shall return a ***Report Parameter by PID*** message with the number of On-Demand DTCs generated during the most recent diagnostic test executed as the data. In response to a single ***Request Diagnostic Routine Results*** message, an ECU shall report all On-Demand DTCs being logged by returning as many consecutive ***Report Diagnostic Routine Results*** messages as is required. Refer to Table 9-4 for an example of consecutive messages returned. Refer to **Appendix A, MESSAGE STRUCTURES & RESPONSES** for direction about responses to diagnostic routine result requests in the various operating states/modes. Note: There is no provision for a Tester to request, or for an ECU to report, less than all of the On-Demand DTCs being logged by the module.

9.2.2.2 Maintaining On-Demand DTCs

An ECU maintains On-Demand DTCs for a given Diagnostic Test, while in the Diagnostic State and another diagnostic test has not been executed. An ECU does not have to maintain the results to more than one diagnostic test at any given time. ECU memory allocated for the storage of Diagnostic Test results, may be overwritten each

time a diagnostic test is executed. The Tester is responsible for obtaining the results to a Diagnostic Test prior to the execution of another test.

9.2.2.3 Clearing On-Demand DTCs

On-Demand DTCs shall be cleared when the ECU returns to the Operational State or when the ECU executes another Diagnostic Test. An ECU shall NOT clear On-Demand DTCs upon receiving a Request Clear Stored Codes message; this message is only used for clearing Continuous DTCs. Note: There are no ISO-9141-FORD or SCP messages for clearing On-Demand DTCs.

9.3 MANDATORY DTCs

Table 9-5 defines the mandatory DTCs that shall be supported by all ECUs which communicate via ISO-9141-FORD or SCP.

Table 9-5 Mandatory DTCs

DTC	Description
$9342	ECU is Defective

9.3.1 ECU is Defective ($9342)

DTC **$9342** shall be logged by an ECU when it detects a failure with its internal circuitry while in the Operational State or while executing a test in the Diagnostic State.

9.4 DTC CROSS REFERENCE

All Continuous and On-Demand DTCs received by a Tester from an ECU shall be modified by the Tester to comply with **SAE J2012 - Recommended Practice for Diagnostic Trouble Code Definitions** prior to being displayed on the Tester. The modifications are shown in Table 9-6.

The first column (**Vehicle Partition**) of the Table identifies four SAE-designated vehicle partitions to which DTC ranges are assigned.

- Powertrain
- Chassis
- Body
- Network Communication

All DTCs generated by an ECU (column 2), are composed of a hexadecimal digit followed by three BCD digits. The SAE equivalent code that will be displayed on the Tester (column 3), is produced by modifying the first digit of the DTC generated by the ECU into a 2-digit alphanumeric code. The three BCD digits remain the same.
Example:
What SAE equivalent code will be displayed on the NGS upon receiving DTC **$5237**?

To determine the SAE equivalent code translate the first digit **"5"** and append the last three digits **"237*"** to the new prefix. DTC **$5237** lies within the SAE equivalent range **C1000 - C1999**. Therefore, the code that will be displayed on the NGS will be **"C1237"**.
As shown in the final column, the first group of DTCs in each vehicle partition is reserved for industry standard SAE designations. The DTCs in the next two groups are free to be assigned by the vehicle manufacturer (in our case FORD). The remaining DTC grouping is **Reserved** for future considerations.

Table 9-6 ECU Tester DTC Cross Reference Chart

VEHICLE PARTITION Sub-Partition Description	Continuous and On-Demand DTCs Logged by an ECU	Equivalent SAE Trouble Codes Displayed on a Tester	Range Defined by:
POWERTRAIN			
	0000 - 0099	P0000 - P0099	SAE
Reserved	0100 - 0299	P0100 - P0299	"
Fuel & Air Metering	0300 - 0399	P0300 - P0399	"
Ignition System or Misfire	0400 - 0499	P0400 - P0499	"
Aux. Emission Controls	0500 - 0599	P0500 - P0599	"
Vehicle Speed & Idle Control	0600 - 0699	P0600 - P0699	"
Computer & Output Circuits	0700 - 0899	P0700 - P0899	"
Transmission	0900 - 0999	P0900 - P0999	"
Reserved	1000 - 1099	P1000 - P1099	FORD
TBD by SAE	1100 - 1299	P1100 - P1299	"
Fuel & Air Metering	1300 - 1399	P1300 - P1399	"
Ignition System or Misfire	1400 - 1499	P1400 - P1499	"
Aux. Emission Controls	1500 - 1599	P1500 - P1599	"
Vehicle Speed & Idle Control	1600 - 1699	P1600 - P1699	"
Computer & Output Circuits	1700 - 1899	P1700 - P1899	"
Transmission	1900 - 1999	P1900 - P1999	"
TBD by SAE	2000 - 2999	P2000 - P2999	SAE Reserved
	3000 - 3999	P3000 - P3999	SAE Reserved
CHASSIS			
	4000 - 4999	C0000 - C0999	SAE
	5000 - 5999	C1000 - C1999	FORD
	6000 - 6999	C2000 - C2999	FORD
	7000 - 7999	C3000 - C3999	**Reserved**
BODY			
	8000 - 8999	B0000 - B0999	SAE
	9000 - 9999	B1000 - B1999	FORD
	A000 - A999	B2000 - B2999	FORD
	B000 - B999	B3000 - B3999	**Reserved**
NETWORK COMMUNICATIONS			
	C000 - C999	U0000 - U0999	SAE
	D000 - D255	U1000 - U1255	FORD
	D256 - D299	U1256 - U1299	FORD
	D300 - D555	U1300 - U1555	FORD
	D556 - D599	U1556 - U1599	FORD
	D600 - D855	U1600 - U1855	FORD
	D856 - D999	U1856 - U1999	FORD
	E000 - E999	U2000 - U2999	FORD
	F000 - F999	U3000 - U3999	**Reserved**

9.4.1 Network Communication DTCs

The last SAE-designated vehicle DTC partition identifies the range of DTCs that are assigned to faults which may occur during inter-module network communication. Two categories of Network Communications DTCs exist within this partition:

- Invalid Data Network DTCs
- Missing Message Network DTC ($D262)

To determine the DTC that should be logged by a module for an "Invalid Data Network Communication Fault:

1. Convert the hexadecimal primary ID of the message received to a BCD number (hex to decimal conversion).
2. Add this BCD number to the base DTC Number for the Network Communication Fault Category - D000.

Example:
ECU transmits a Status Request message to ECU (B) and ECU (B) responds with a message that contains "Invalid Data".

The Primary ID is of the desired information is $73.

What DTC should be logged by the ECU?

1. Convert $73 to BCD : (7 x 16) + (3 x 1) = 115
2. Add 115 to the Base DTC : D000 + 115 = D115

9.5 Circuit Malfunction DTCs

Circuit malfunction DTCs are to be logged in ECUs that cannot distinguish between the type of fault.

Example: If an ECU input is high, the ECU may not be able to distinguish between a short to Batt+ or an open circuit.

A generic DTC specifying that there is a circuit malfunction should be logged in the condition for the above example or any that the ECU determines that there is a circuit failure but cannot describe exactly what the fault is.

10. NETWORK COMMUNICATIONS DIAGNOSTIC REQUIREMENTS (SCP Only)

10.1 PURPOSE

This section defines the diagnostic strategies that an ECU shall implement to handle fault conditions which prevent valid inter-module dialogue from occurring over a vehicle's SCP Network.

10.2 NETWORK FAULT TYPES

Network faults can be categorized into two types, as follows:

- Invalid Data network faults
- Missing Message network faults

10.2.1 Invalid Data Network Faults

An Invalid Data Network fault is a Continuous DTC that shall be logged by an ECU when "Invalid Data" is received in a SCP network message. Invalid Data Network Faults are those which cause an ECU to substitute one or more "Invalid Data" fault values in place of valid functional data in an inter-module network message. Invalid Data network faults are initiated by faults, or incorrect data, on inputs to the module. "Invalid Data" is defined as follows:

- $3F - for digital (True/False, On/Off, . .) data
- $XX - for analog information ["XX" is typically FF but may be any value as defined in the vehicle program's VIDS (Part III)].

The structure of all inter-module network messages implemented in a specific vehicle can be found in the **Vehicle Interface Diagnostic Specification (Part III).**

10.2.1.1 Digital Invalid Data Network Fault Strategy

If a network message's valid functional data is normally digital information, (such as enabled/disabled or status), the transmitting ECU shall place "Invalid Data" information into the network message according to the following procedure.

1. Shift all data that is located to the right of the message's Source Address (byte 3) one byte position to the right
2. Place the Digital "Invalid Data" value - $3F into the message to the immediate right of the Source Address. This is the location previously held by the message's Secondary ID.

The following example provides a visual indication of how a network message is modified to provide Digital "Invalid Data" information.

Digital Invalid Data Example:
For Brake Lamp Pedal Switch Status - Active

Normal Message: 41 33 70 A2
"Invalid Data" Message: 41 33 70 **3F** A2

10.2.1.2 Analog Invalid Data Network Fault Strategy

If an ECU that is responsible for providing analog data in a network message, detects a fault that would render all or a portion of that analog data invalid, it shall replace each affected Byte of normally valid information with "Invalid Data" as defined in the VIDS (Part III). The following example provides a visual indication of how a network message is modified to provide Analog "Invalid Data" information. Other than modifying the required data bytes,

the structure of the message that gets transmitted is the same as that sent with "valid data". A network message's valid functional data may consist of one to six bytes of analog information (such as a temperature or voltage value).

Analog Invalid Data Examples:
For Engine Coolant Fluid Temperature Status;

Normal Message 81 49 10 10 xx
"Invalid Data" Message 81 49 10 10 88

Note: "88" is defined as the "Invalid Data" value

For Actual Axle Torque w/min. & max. Available;

Normal Message: A1 09 10 22 xx xx yy yy zz zz
"Invalid Data" Message: A1 09 10 22 FF FF yy yy zz zz (FF is the invalid data)

10.2.1.3 Digital and Analog Invalid Data Network Fault Strategy

If an ECU that is responsible for providing digital and analog data in a network message, detects a fault that would render all or a portion of that data invalid, it shall follow the requirements defined above for digital invalid data and analog invalid data. The following example provides a visual indication of how a network message is modified to provide Digital and Analog "Invalid Data" information.

Digital and Analog Invalid Data Examples:
For Backlighting Intensity & Dimming Curve w/ Headlamps Command - ON;

Normal Message 41 DA 60 12 xx xx
"Invalid Data" Message 41 DA 60 **3F** 12 xx xx (Digital Data)
OR
Normal Message 41 DA 60 12 xx xx
"Invalid Data" Message 41 DA 60 12 **FF** xx (Analog Data)

10.3 Missing Message Network Faults

A Missing Message Network fault may be logged by an ECU upon failure to receive a message expected from another ECU within a defined Retry period. There is only one Continuous DTC; **$D262,** that is allocated for an ECU to identify all possible Missing Message Network Faults that it may detect while communicating with other ECUs. This Missing Message Network Continuous DTC; **$D262,** is optional and shall NOT be supported by an ECU unless the "Missing Message" is deemed a critical failure to the function of the ECU as agreed upon by the AVT EESE Diagnostic representative. Implementation of the Missing Message Network Continuous DTC shall be defined in the appropriate Vehicle Interface Diagnostic Specification (VIDS - Part III) for the vehicle program. Causes of Missing Message Network Faults include the following:

- ECU transmitter failures
- ECU receiver failures
- SCP dual wire opens or shorts
- Initialization timing differences
- Power up and/or Power down timing differences
- ECU placed into Diagnostic State
- ECU disconnected by service technician

10.3.1 Missing Message Network Fault Strategy

If an ECU, which supports the Missing Message Network DTC **$D262**, does not receive an expected message it shall set a missing message flag in RAM and initialize a 5.0- second (-0.0, +1.0) timer. If during this time period the ECU does not receive other unique messages, it shall set missing message flags unique to each message and

reset the 5.0 second timer for each new missing message flag [an ECU need only have one 5.0-second (-0.0 +1.0) timer].

If a message is received during the 5.0-second (-0.0, +1.0) interval, its missing message flag shall be reset. If all flags are reset prior to the expiration of the timer, Network DTC - **$D262** shall NOT be logged by the ECU. If any missing message flag remains set for the 5.0-second (-0.0 +1.0) interval, the ECU shall log DTC **$D262** in non-volatile memory. This method is referred to as Network DTC Aging and prevents the Missing Message Network DTC from being logged due to random intermittent occurrences or during Vehicle Power Up and/or Down because not all ECUs Power Up and/or Down at the same rate.

10.3.2 Missing Message Network Fault Strategy for Display Driven ECUs

If a Display Driven ECU, which supports the "Missing Message" Network DTC **$D262**, does not receive an expected message, the Display Driven ECU shall set a missing message flag in RAM and initialize/monitor a missing message timer. A unique missing message flag is required for each missing message. The missing message timer shall only be initialized/reset when there are NO other missing message flags. The missing message timers shall be either 1.0 second (-0.0, +0.25), 2.0 seconds(-0.0, +0.5), or 5.0 seconds (-0.0, +1.0). The missing message timer shall be determined by the Display Driven ECU function and the specific missing message. Note: a Display Driven ECU only needs to have one 1 second (-0.0, +0.25) missing message timer, one 2 second (-0.0, +0.5) missing message timer, and one 5 second (-0.0, +1.0) missing message timer.

If after the missing message time period has expired and the Display Driven ECU has not received the expected message, the Display Driven ECU shall log DTC **$D262**. If the expected missing message is received before the missing message time period has expired, the Display Driven ECU shall reset the missing message flag. If NO other missing message flags are set, the Display Driven ECU shall reset the missing message timer. If all missing message flags are reset/cleared, DTC **$D262** shall NOT be logged by the Display Driven ECU.

It is intended, that the display driven ECU shall log DTC **$D262**, whenever an event that is visible to the customer is caused because of a missing message. An example of an event that would be visible to the customer would be if the vehicle speed message is missing for more than 1.0 second (-0.0, +0.25), the instrument cluster would move the pointer to zero and therefore, shall log DTC **$D262**.

NOTE: Any visual indicator (LED, Message Center display, ...) tied to the detection of missing message faults shall NOT be illuminated until DTC $D262 is logged into non-volatile memory

10.3.2.1 Examples of Missing Message Timers for Display Driven ECUs

An example of a display function and message which shall use the 1.0 second (-0.0, +0.25) timer is speedometer/vehicle speed or tachometer/engine RPM.

An example of a display function and message which shall use the 2.0 second (-0.0, +0.5) timer is electronic PRNDL/transmission PRNDL range selected status.

An example of a display function and message which shall use the 5.0 second (-0.0, +1.0) timer is fuel level/fuel level sensor A/D output status or average fuel economy/fuel flow and odometer rolling count.

10.3.3 TEST TOOL MISSING MESSAGE NETWORK TEST

To determine the location, or path, of an inter-module "Missing Message" communication fault, all FORD Test Tools shall implement a Network Test in which the Test Tool attempts to establish communication with each of the ECUs on the link . If, during this Network Test, the Test Tool fails to establish communication with one or more ECU, it shall display information that indicates which ECU could not establish communication.

10.4 MESSAGE TYPES & DIAGNOSTIC NETWORK FAULT SRATEGY

The Diagnostic Network Fault Strategy applies to all SCP network message types. Refer to the following subsections for an explanation of how the strategy applies to ECUs that transmit and receive the following message types:

- Function Read Broadcast Messages
- Status Request Broadcast Messages
- Event-Driven Broadcast Messages
- Periodic Broadcast Messages

10.4.1 Function Read Broadcast Message - Diagnostic Network Fault Strategy

A Function Read Message is a request message that has 1 of a possible 256 primary "function IDs" as the target address rather than an ECU physical address. The response to a Function Read Request Message is an "acknowledge" that consists of a single Byte of data. For transmitting and receiving Function Read Broadcast Messages, refer to the following subsections.

10.4.1.1 Transmitting Function Read Broadcast Message

If the ECU that transmits a Function Read Request receives an acknowledge that contains "Invalid Data", it shall immediately log the associated invalid data network DTC into non-volatile memory. If the ECU that transmits a Function Read Request does not receive an acknowledge within the Defined Retry Strategy Period, it shall NOT log a DTC. Refer to Figure 10-1 and 10-2 for a description of the Diagnostic Network Fault Strategy for an ECU transmitting a Function Read Request Message.

10.4.1.2 Receiving Function Read Broadcast Message

A module responsible for responding to a Function Read Request is not responsible for logging any fault information related to that request.

10.4.2 Status Request Broadcast Message - Diagnostic Network Fault Strategy

A Status Request Broadcast Message is a functionally addressed message that is used to retrieve information from a specific ECU which supports the "function ID" specified as the Target Address of the request. The response to a Status Request Broadcast Message is a message from the ECU that received the request. For transmitting and receiving Status Request Broadcast Messages, refer to the following subsections.

10.4.2.1 Transmitting Status Request Broadcast Message

If the ECU that transmits a Status Request Message receives a response that contains "invalid Data", it shall immediately log the associated "*invalid data*" Network DTC into non-volatile memory. If the ECU, which supports Network DTC **$D262**, transmits a Status Request Broadcast Message and does not receive a response within the Defined Retry Period, it shall "age" a Network DTC, which identifies the function associated with the "Missing Message" fault, for a period of 5.0 seconds (-0.0, +1.0) before logging it into non-volatile memory. Refer to Figure 10-3 for a description of the Diagnostic Network Fault Strategy for an ECU that transmits a Status Request Broadcast Message.

10.4.2.2 Receiving Status Request Broadcast Message

An ECU that is the target of a Status Request Broadcast Message is not responsible for logging any fault information related to that request.

10.4.3 Event-Driven Broadcast Message - Diagnostic Network Fault Strategy

An Event-Driven Broadcast Message is a Non-Periodic Broadcast Message that has 1 of a possible 256 primary "function IDs" as the Target Address of the message. Each module on the network that supports the "targeted

function" is an intended recipient of an Event-Driven Broadcast Message. For transmitting and receiving Event-Driven Broadcast Messages, refer to the following subsections. NOTE: Periodic and Non-Periodic Event-Driven Broadcast messages are all treated as Event-Driven Broadcast messages.

10.4.3.1 Transmitting Event-Driven Broadcast Message

Upon each occurrence of an event where a fault is present, the ECU responsible for transmitting an Event-Driven Broadcast Message shall immediately send the message with "Invalid Data" information. When the fault is no longer present, the ECU shall return to sending valid data upon the occurrence of subsequent events. If the ECU that transmits an Event-Driven Broadcast Message does not receive an acknowledge within the Defined Retry Strategy Period, it shall NOT log a DTC that indicates the lack of acknowledgment.

10.4.3.2 Receiving Event-Driven Broadcast Message

If an ECU receives an Event-Driven Broadcast Message that contains "Invalid Data", it shall log the Network DTC associated with that function into non-volatile memory. A receiving ECU shall NOT log a "Missing Message" Network DTC because it can not know when an Event-Driven Message is sent. Refer to Figure 10-4 for a description of the Diagnostic Network Fault Strategy to be implemented by an ECU that is the target of an Event-Driven Broadcast Message.

10.4.4 Periodic Broadcast Message - Diagnostic Network Fault Strategy

A Periodic Broadcast Message is a functionally addressed network message that is sent at defined periodic intervals, by an ECU to one or more other ECUs which have a specific need for the information. Periodic Broadcast Messages may be initiated at Power-Up or upon the occurrence of an event. For transmitting and receiving Periodic Broadcast Messages, refer to the following subsections.

10.4.4.1 Transmitting Periodic Broadcast Message

If an ECU that is responsible for transmitting a periodic broadcast message detects an "Invalid Data" fault, it shall transmit the message with "Invalid Data" at the next scheduled message transmission and continuously thereafter at the defined periodic interval as long as the fault exists. When the invalid data fault is no longer present, the ECU shall return to sending "Valid Data" at the defined periodic interval. If the ECU that transmits a periodic broadcast message does not receive an acknowledge within the Defined Retry Strategy Period, it shall NOT log a DTC that indicates the lack of acknowledgment.

10.4.4.2 Receiving Periodic Broadcast Message

If an ECU receives a Periodic Broadcast Message that contains "Invalid Data", it shall log a network DTC, which identifies the function associated with that "Invalid Data", into non-volatile memory. If an ECU, which supports Network DTC **$D262**, does not receive a Periodic Broadcast Message within the defined retry strategy period, it shall "age" a network DTC which identifies the function associated with the missing message fault for a period of 5.0 seconds (-0.0, +1.0) before logging it into non-volatile memory. Refer to Figure 10-5 for a description of the Diagnostic Network Fault Strategy to be implemented by an ECU receiving a Periodic Broadcast Message.

10.4.5 ECU DIAGNOSTIC NETWORK FAULT STRATEGY

To prevent communications-related network DTCs from being logged (erroneously) because not all ECUs may be ready to communicate at the same time, an ECU shall implement one of the following Diagnostic Network Strategies based on its node type.

- Non-Sleep Node ECU Diagnostic Network Fault Strategy Initialization
- Sleep Node ECU Diagnostic Network Fault Strategy Initialization

NOTE: These Node Network Diagnostic Strategies do not prohibit an ECU from logging continuous DTCs which are non-network related during this time period.

10.4.5.1 Non-Sleep Node ECU Diagnostic Network Fault Strategy Initialization

A Non-Sleep Node ECU SHALL NOT start its Diagnostic Network Fault Strategy until 5.0 seconds (-0.0, +1.0) after its Ignition RUN input, or equivalent signal input, has gone active. After the Ignition Switch is turned to RUN , the System Initialization Phase starts. Between 40ms and 115ms an ***Ignition Switch Position - RUN*** message will be transmitted by a Non Sleeping Node ECU, responsible for initializing one or more sleep node modules. At 500ms all ECUs are to be capable of receiving network traffic.

10.4.5.2 Sleep Node ECU Diagnostic Network Fault Strategy Initialization

A Sleep Node ECU shall NOT start its diagnostic network fault strategy until 5.0 seconds (-0.0, +1.0) after it has received an ***Ignition Switch Position - RUN*** message, or equivalent message, over the SCP serial communications link.

10.4.5.3 Discontinuing Sleep Node ECU Diagnostic Network Fault Strategy

A sleep node ECU shall discontinue its Diagnostic Network Fault Strategy upon receiving a message which states that the vehicle's Ignition switch is no longer in the RUN position.

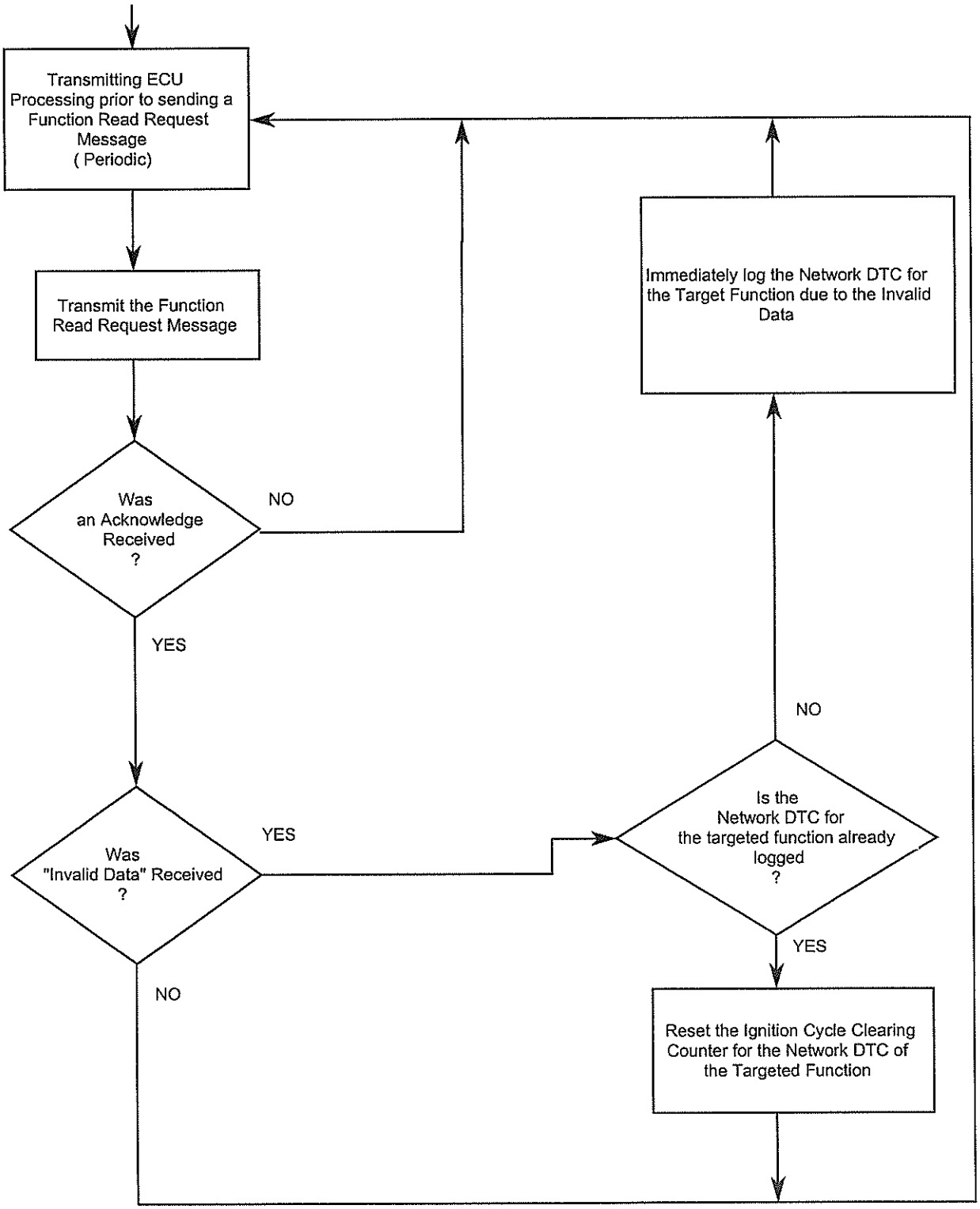


Figure 10-1 Function Read Periodic Broadcast Message - Diagnostic Network Fault Strategy (Transmitting ECU)

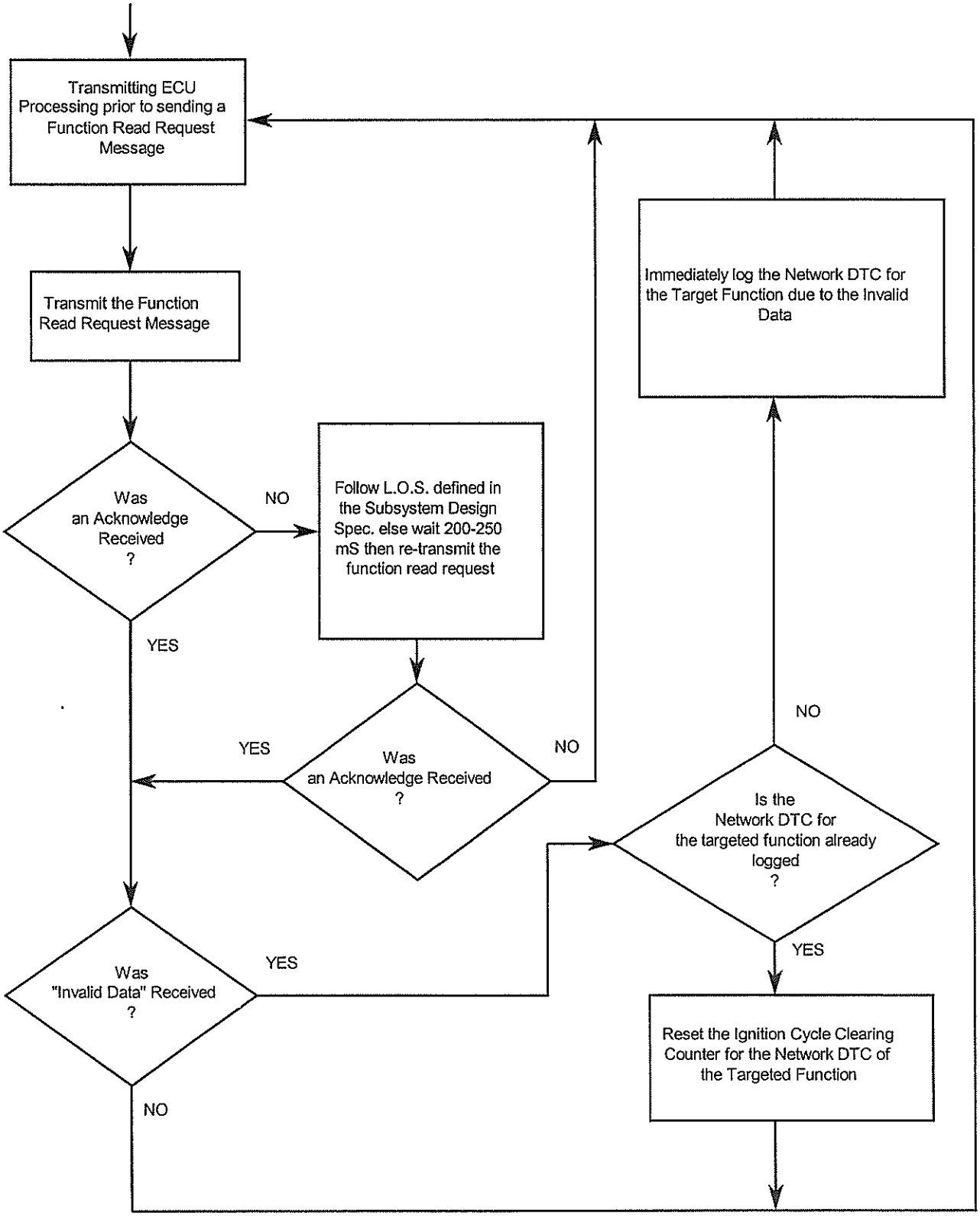


Figure 10-2 Function Read Non-Periodic Broadcast Message - Diagnostic Network Fault Strategy (Transmitting ECU)

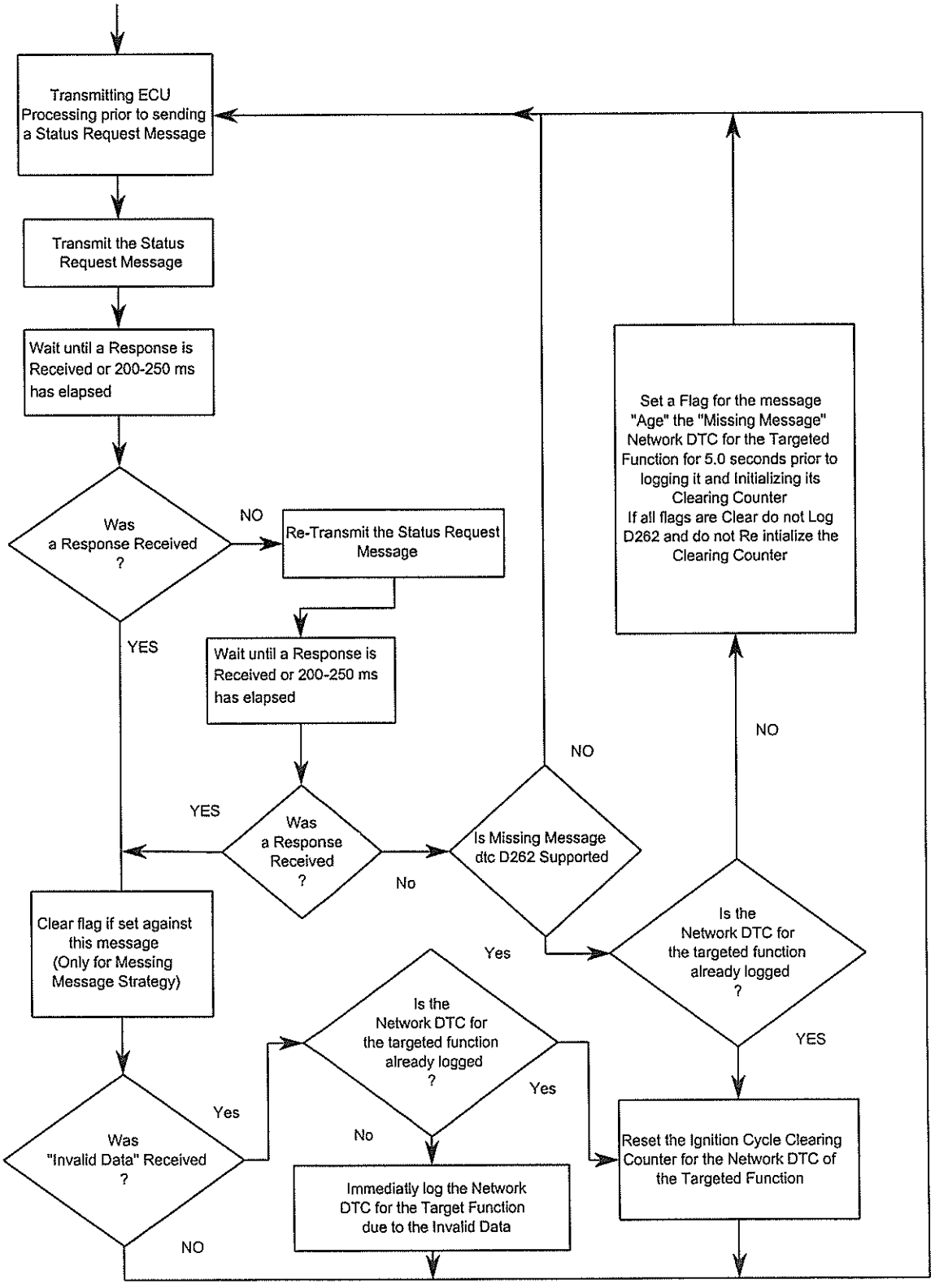


Figure 10-3 Status Request Broadcast Message - Diagnostic Network Fault Strategy (Transmitting ECU)

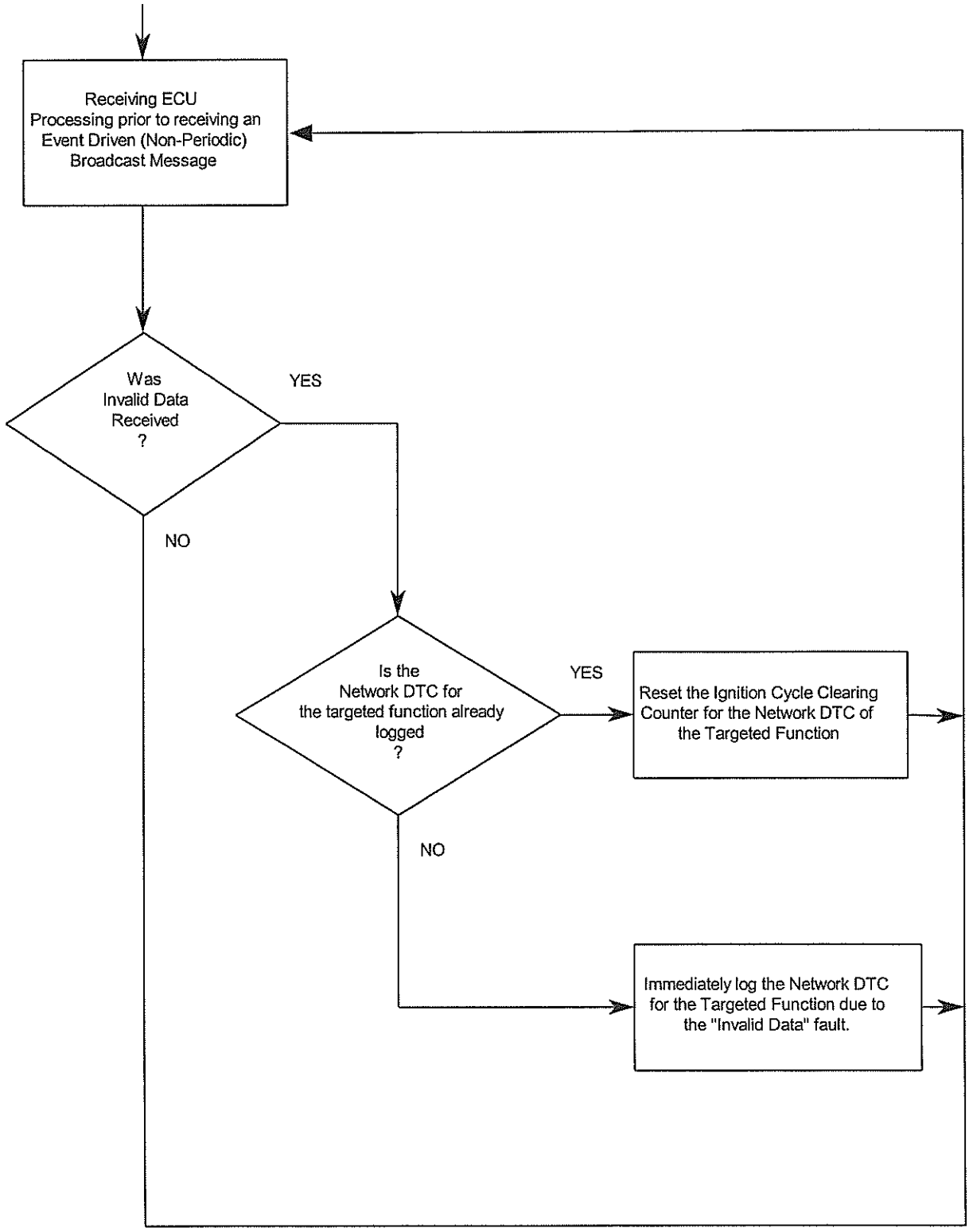


Figure 10-4 Event Driven Broadcast Message - Diagnostic Network Fault Strategy (Receiving ECU)

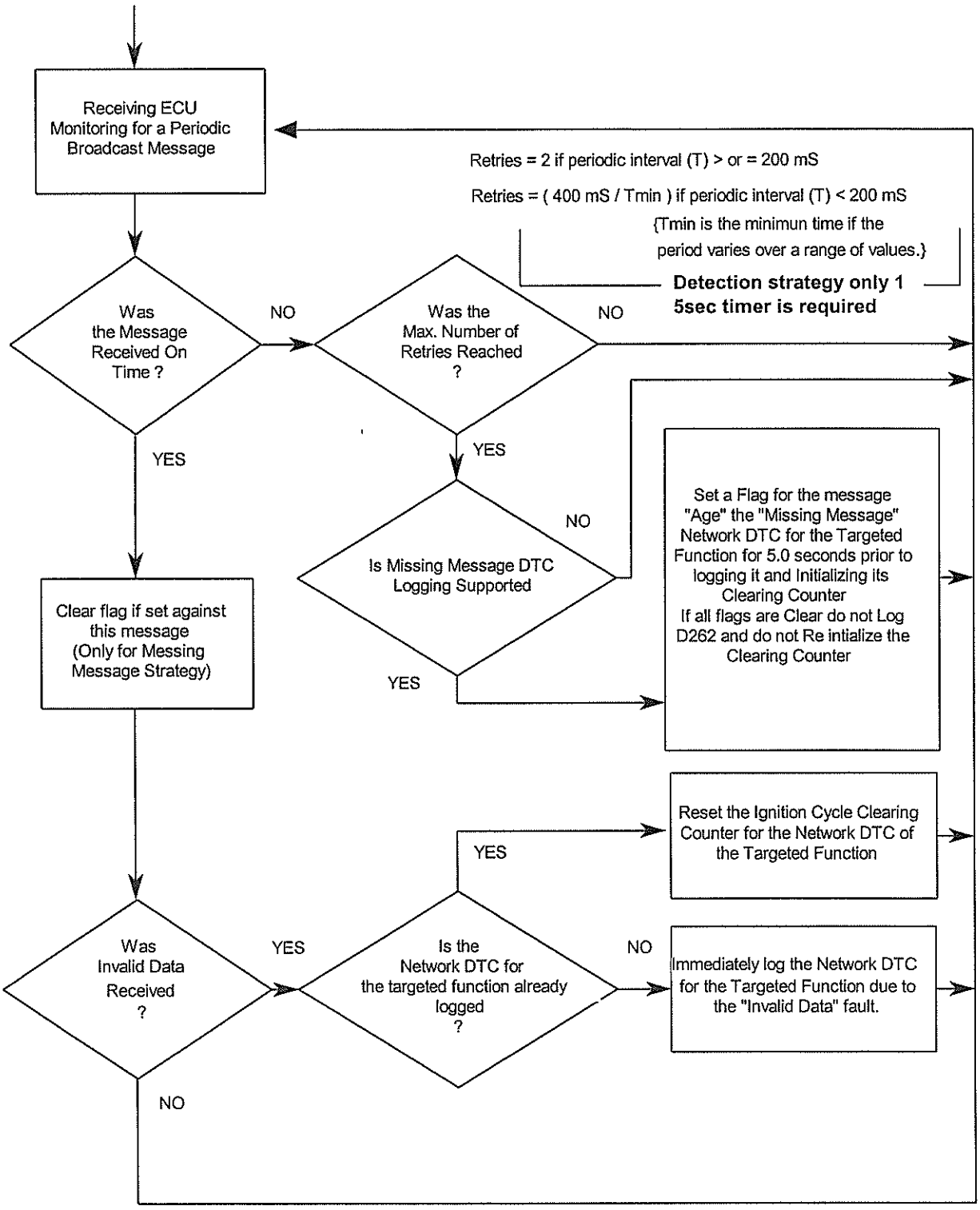


Figure 10-5 Periodic Broadcast Message - Diagnostic Network Fault Strategy (Receiving ECU)

10.5 NO STORED CODES LOGGING (NSCL) Mode

All ECUs that support logging of a Missing Message Network DTC, shall implement the No Stored Codes Logging (NSCL) Mode. This strategy prevents erroneous network DTCs from being logged by ECUs that are involved in inter-module communication with other modules which are inhibited from properly communicating over the SCP serial communications link. Refer to **Section 4.0, Operational State Diagnostic Requirements**, and **Section 5.0, Diagnostic State Requirements**, for additional information about the No Stored Codes Logging mode.

NOTE: The NSCL Mode requirements only pertains to ECUs which communicate via SCP. It is not valid for ECUs which communicate via ISO-9141-FORD, because they are not required to support inter-module communication. An SCP ECU is only required to support the No Stored Codes Logging Mode, if it supports logging of Missing Message Network DTCs.

10.5.1 NSCL Mode Diagnostic Requirements

While in the No Stored Codes Logging mode, an ECU shall support all of the serial communications link interface requirements shown in Figure 10-6. This includes supporting the following functions:

- NOT log ANY Continuous DTCs → Mandatory
- Access to parameter identifier (PID) information → Mandatory
- Data access via direct memory reference (DMR) → Optional
- Entry into the Diagnostic State → Mandatory
- Respond normally to all messages received from a Tester → Mandatory
- NOT respond with "Invalid Data" to messages received from other ECUs on SCP. → Mandatory
- Respond with "Valid Data" to all messages received from other ECUs on SCP. → Mandatory

Refer to Figure 10-7 for a flowchart containing the No Stored Codes Logging Strategy. For detailed information about the serial communications link messages that an ECU shall support while executing in the No Stored Codes Logging mode, refer to **Appendix A, MESSAGE STRUCTURES & RESPONSES.** NOTE: The module is not required to return a response to the ***Request No Stored Codes Logging*** message

10.5.1.1 Entering No Stored Codes Logging Mode

An ECU in the Operational State shall enter the No Stored Codes Logging mode upon receiving a ***Request No Stored Codes Logging*** message over the SCP communications link.

Upon receiving this message while in the Operational State, an ECU shall:

- Initialize and enable a 5.0 second (-0.0, +1.0) "Remain in the No Stored Codes Logging Mode" timer. This timer is used by the ECU to determine whether it should remain in the No Stored Codes Logging mode or return to the Operational State.
- Update PID ***$D100 - ECU Operating State Mode*** to ***$09 - No Stored Codes Logging Mode***

10.5.1.2 Exiting No Stored Codes Logging Mode to the Operational State

An ECU shall exit the No Stored Codes Logging mode and return to the Operational State if any of the following conditions occur:

- The ECU receives a ***Request Return to Operational State*** message.
- The ECU does not continue to receive the functionally broadcast ***Request No Stored Codes Logging*** message within 5.0 (-0.0, +1.0) seconds of either the last broadcast message received or initial entrance into No Stored Codes Logging Mode.
- A requirement, as specified in the ECUs SSDS, to remain in the No Stored Codes Logging Mode is violated.

10.5.1.3 NSCL Mode to Operational State Timing

Upon returning to the Operational State, from the NSCL mode, an ECU shall return to normal operation (i.e., continue logging Continuous DTCs) within 500 milliseconds. NOTE: The ECU never left the Operational State and therefore shall not re-initialize.

10.5.1.4 Exiting No Stored Codes Logging Mode to the Diagnostic State

An ECU shall exit the No Stored Codes Logging mode and enter the Diagnostic State if any of the following conditions occur:

- The ECU receives a ***Request Diagnostic State Entry*** message.

10.5.1.5 NSCL Mode Tester Timing Requirements

Prior to placing an SCP ECU in the Diagnostic State, a Tester shall broadcast a ***Request No Stored Codes Logging*** message. It shall continue broadcasting this message within 4.5-second intervals.

A Tester shall broadcast a ***Request No Stored Codes Logging*** message over the SCP Link 400 milliseconds before instructing any ECU on the SCP link to enter the Diagnostic State.

A Tester shall broadcast a ***Request No Stored Codes Logging*** message within less than 100 milliseconds after issuing a ***Request Return to Operational State*** (ECU in Diagnostic State) message to any ECU on the SCP Link.

After returning an ECU to the Operational State, from the Diagnostic State, a Tester shall broadcast a ***Request No Stored Codes Logging*** message within 550 milliseconds of issuing the ***Request Return to Operational State*** message. This is to ensure that the ECU enters the NSCL mode right after the 500 millisecond re-initialization period, prior to logging erroneous Missing Message Network DTCs.

No.	Action	Source	Target	Diag. No.
1	Request No Stored Codes Logging	Tester	ECU 1&2	
2	Wait 400 ms			1
3	Request Diag Sate Entry	Tester	ECU 1	
4	Transmit NSCL every 4.5 sec	Tester	ECU 1&2	
•	•	•	•	
•	•	•	•	
x	•	•	•	
x+1	Request Return to Operational State	Tester	ECU 1	2
x+2	Within 100 ms of about action Transmit NSCL message	Tester	ECU 1&2	2
x+3	ECU 1 Does not have to respond for the next 500 ms			3
x+4	550 ms after action x+1 transmit NSCL	Tester		4

Refer to Figure 10-8 for the Tester timing requirements when supporting the No Stored Codes Logging Strategy.

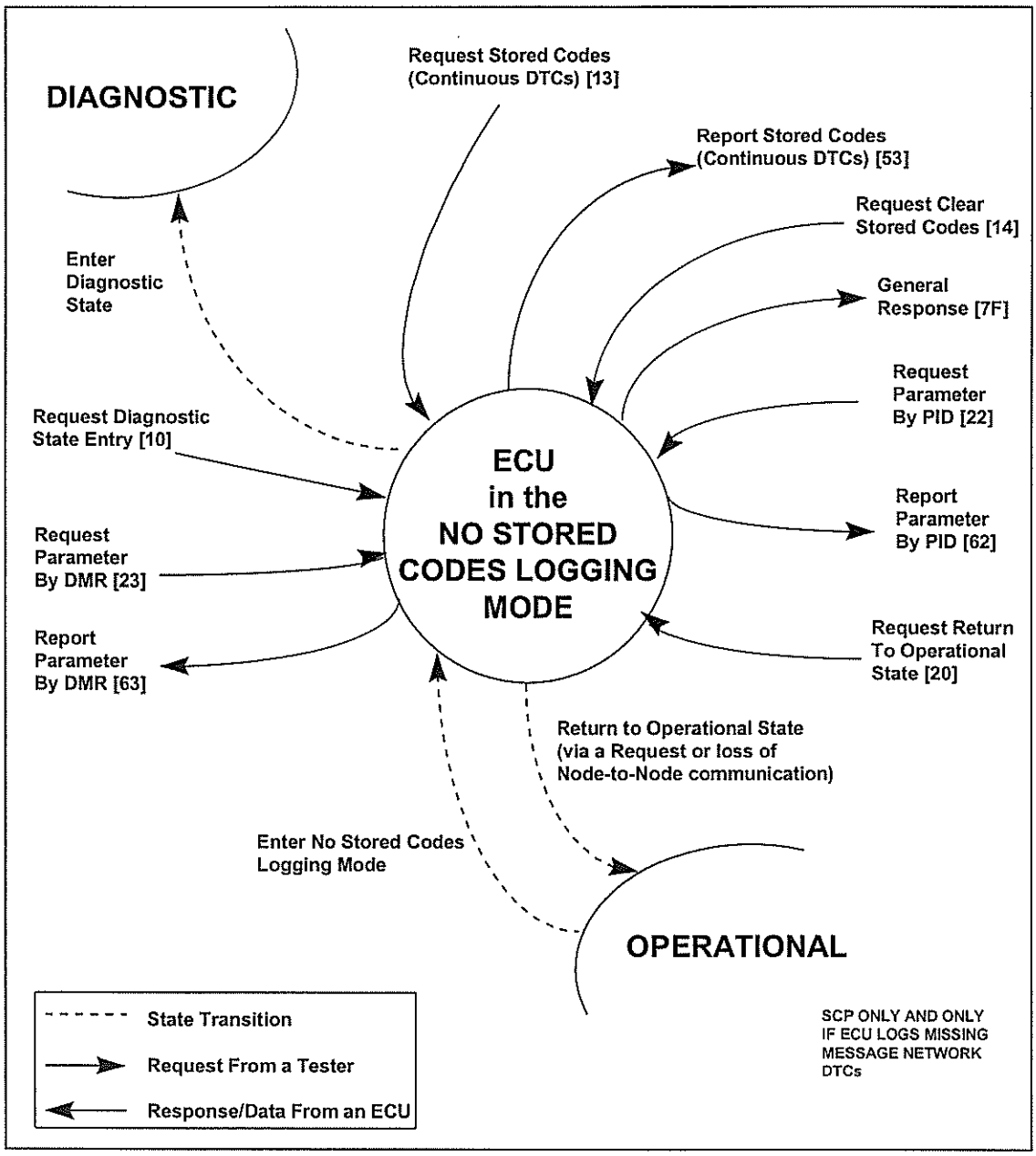


Figure 10-6 No Stored Codes Logging Mode Serial Link Interface Requirements

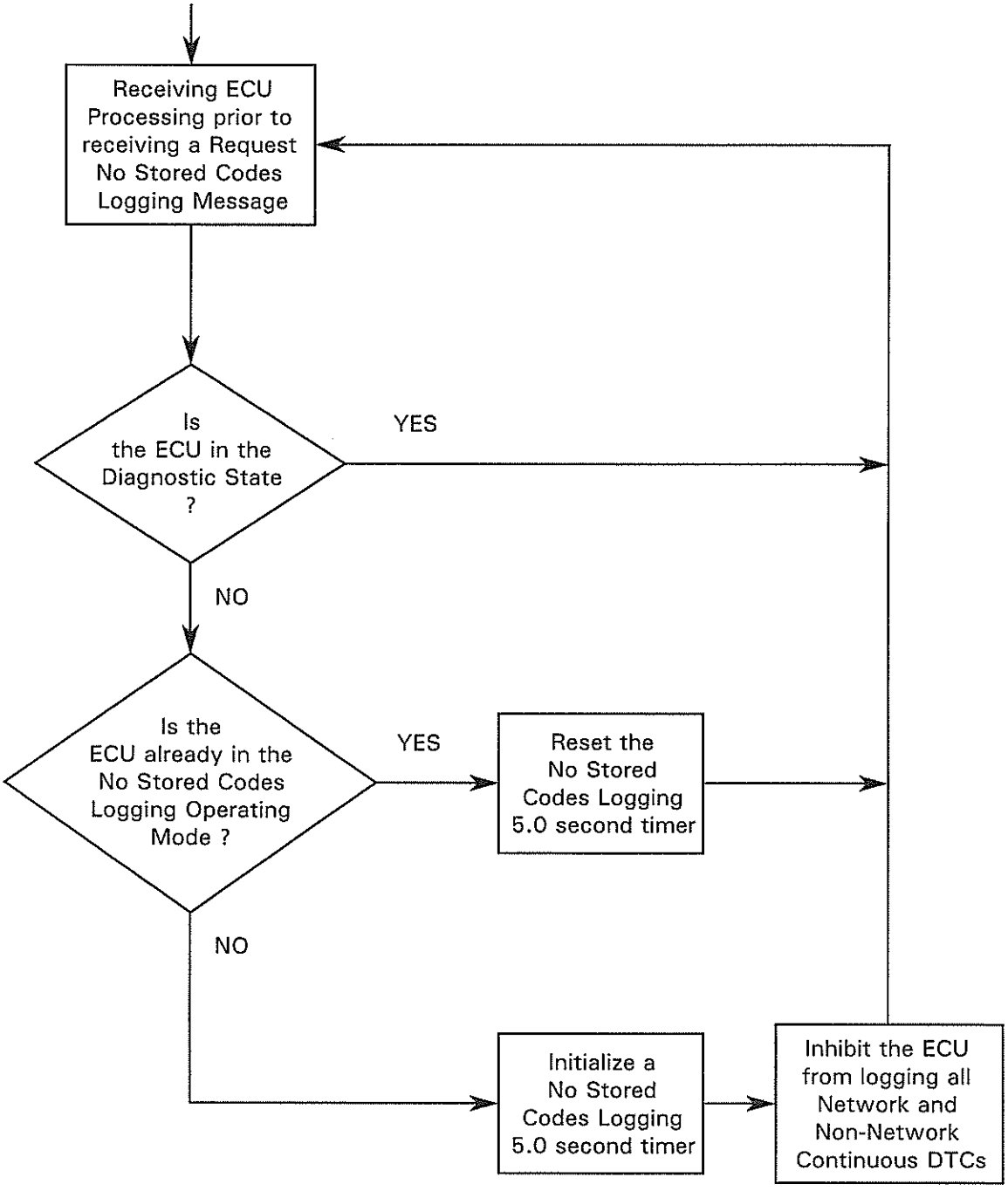
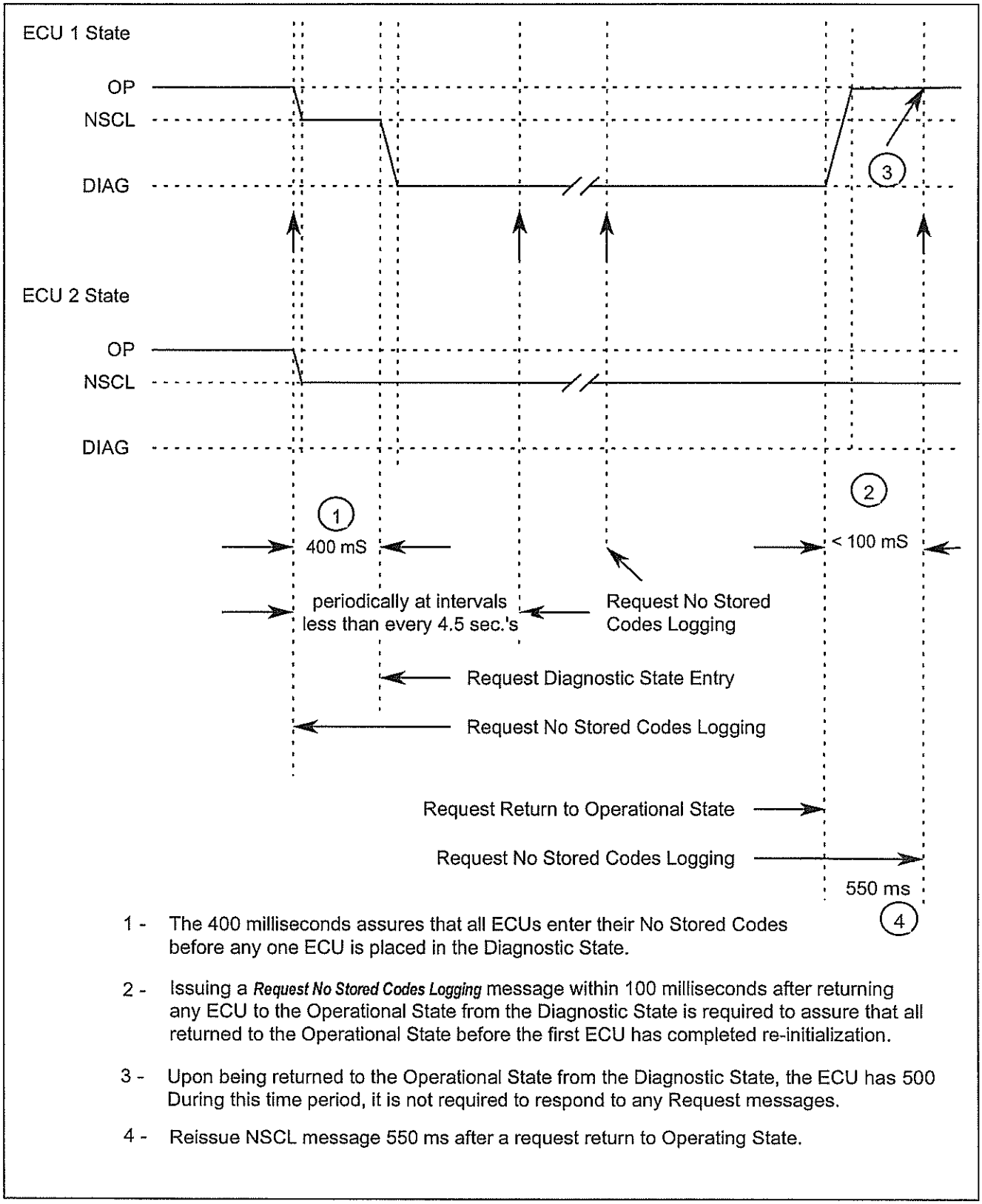


Figure 10-7 No Stored Codes Logging Strategy (Receiving ECU)



1 - The 400 milliseconds assures that all ECUs enter their No Stored Codes before any one ECU is placed in the Diagnostic State.

2 - Issuing a ***Request No Stored Codes Logging*** message within 100 milliseconds after returning any ECU to the Operational State from the Diagnostic State is required to assure that all returned to the Operational State before the first ECU has completed re-initialization.

3 - Upon being returned to the Operational State from the Diagnostic State, the ECU has 500 During this time period, it is not required to respond to any Request messages.

4 - Reissue NSCL message 550 ms after a request return to Operating State.

Figure 10-8 No Stored Codes Logging Strategy Timing Requirements (Tester)

11. MEMORY ACCESS & DATA TRANSFER REQUIREMENTS

11.1 PURPOSE

This section defines the methods for reading from, and writing to, ECU memory. Reading from ECU memory shall be referred to as ***Uploading***. Writing to ECU memory shall be referred to as ***Downloading***.

11.2 DIRECT MEMORY REFERENCE (DMR)

Direct memory reference is the capability whereby information contained within an ECU can be retrieved by a Tester by specifying the ECU's physical address of where the data is located in the module. This capability can be used to retrieve from an ECU, information for which no PID was assigned

NOTE: DMR is used for development only (i.e., no service tools support for DMR capability will be provided).

11.2.1 DIRECT MEMORY REFERENCE (DMR) Requirements

When a ***Request Parameter by DMR*** message is received by the ECU which contains the address of a sensitive memory location, or out of range data, or an invalid address; the ECU shall report a ***General Response*** message having a response code of **$12; *Invalid Data*** for modules that support DMRs.

11.3 SECURITY ACCESS PROCEDURE

The Security Access Procedure (shown in Figure 11-1) shall be used when downloading data to an ECU and intentional modification of the data could cause harm to the occupants of a vehicle or to the subsystem.

The Security Access Procedure (shown in Figure 11-5) shall be used when uploading data from an ECU that contains any type of information, that if read, could result in unauthorized access to the vehicle entry or anti-theft security systems.

11.4 MEMORY CONFIGURATION/DOWNLOAD PROCEDURE

11.4.1 METHOD 1 CONFIGURATION/DOWNLOAD PROCEDURE

The Method 1 Configuration Procedure utilizes the diagnostic command function to configure features/functions within an ECU's software. Refer to the **Module Programming/Configuration Design & Process Specification identified in Appendix D** for further definition of use.

11.4.2 METHOD 2 DOWNLOAD PROCEDURE

The Method 2 Download Procedure (shown in Figure 11-2) utilizes the Write Memory Block method of configuration which allows up to 5 bytes of data to be written to the ECU. The purpose of this mode ($3B) is to allow an off-board tester to modify the contents of memory locations that are referenced by block number. Refer to the **Module Programming/Configuration Design & Process Specification identified in Appendix D** for further definition of use.

11.4.3 METHOD 3 DOWNLOAD PROCEDURE

The Method 3 Download Procedure (shown in Figure 11-3 and 11-4) utilizes the download block method which provides a means for an off-board tester to send multiple bytes of data to the ECU. This method should be used when the configuration data exceeds the five byte limit of the previous methods. The download block method supports data transfer of up to 64K. Refer to the **Module Programming/Configuration Design & Process Specification identified in Appendix D** for further definition of use.

11.4.4 METHOD 4 FLASH PROGRAMMING PROCEDURE

The Method 4 Flash Programming Procedure is currently being used by the Powertrain Control Module (PCM). The procedure is used to flash the entire memory of the PCM. Refer to the **Module Programming/Configuration Design & Process Specification identified in Appendix D** for further definition of use.

11.4.5 METHOD 5 UPLOAD PROCEDURE

The Method 5 Upload Procedure (shown in Figure 11-5 and 11-6) utilizes the diagnostic block transfer mode to upload memory blocks within an ECU's memory to an off-board tester. The maximum amount of data that can be uploaded with this procedure is 64K.

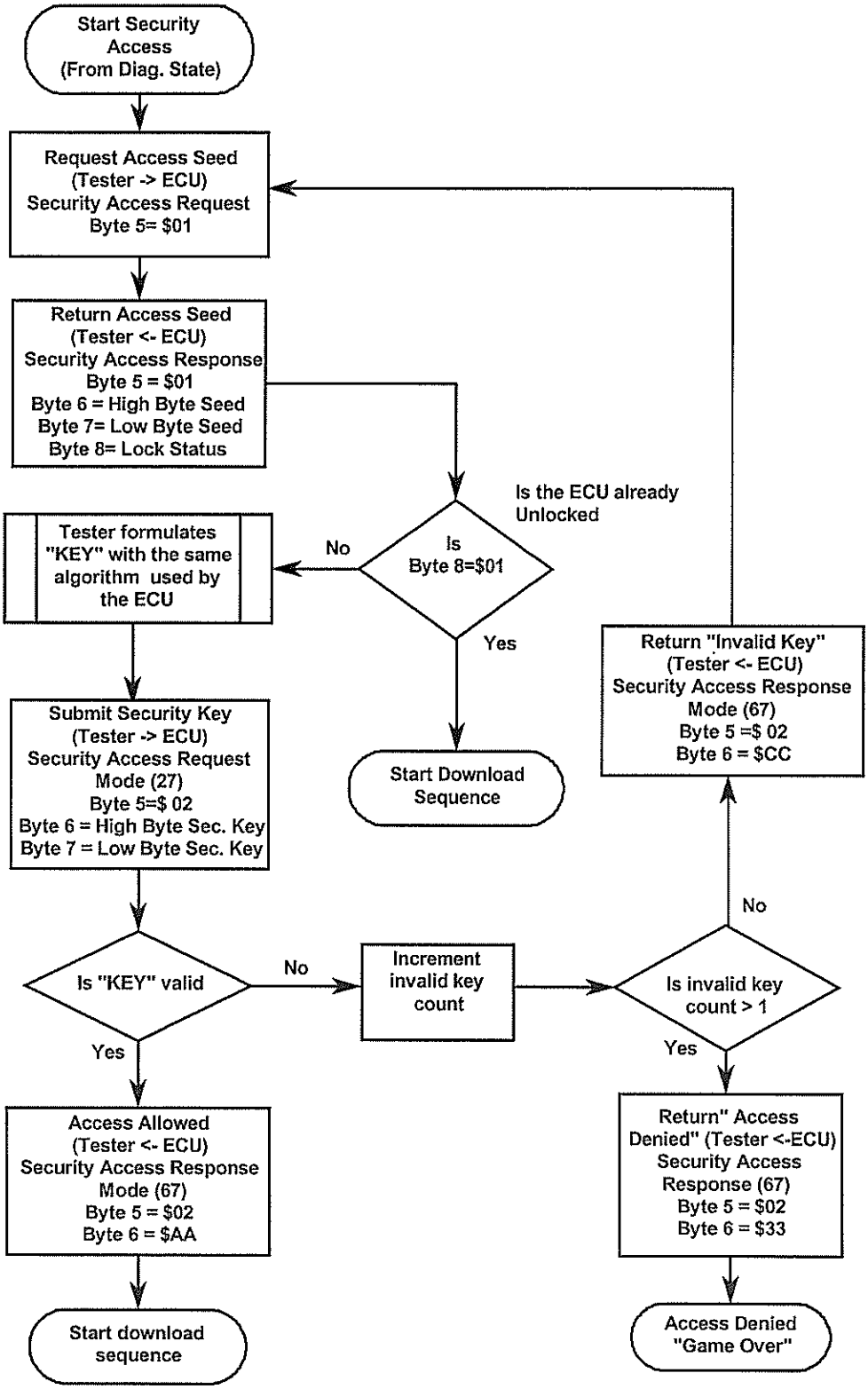


Figure 11-1 Security Access Procedure

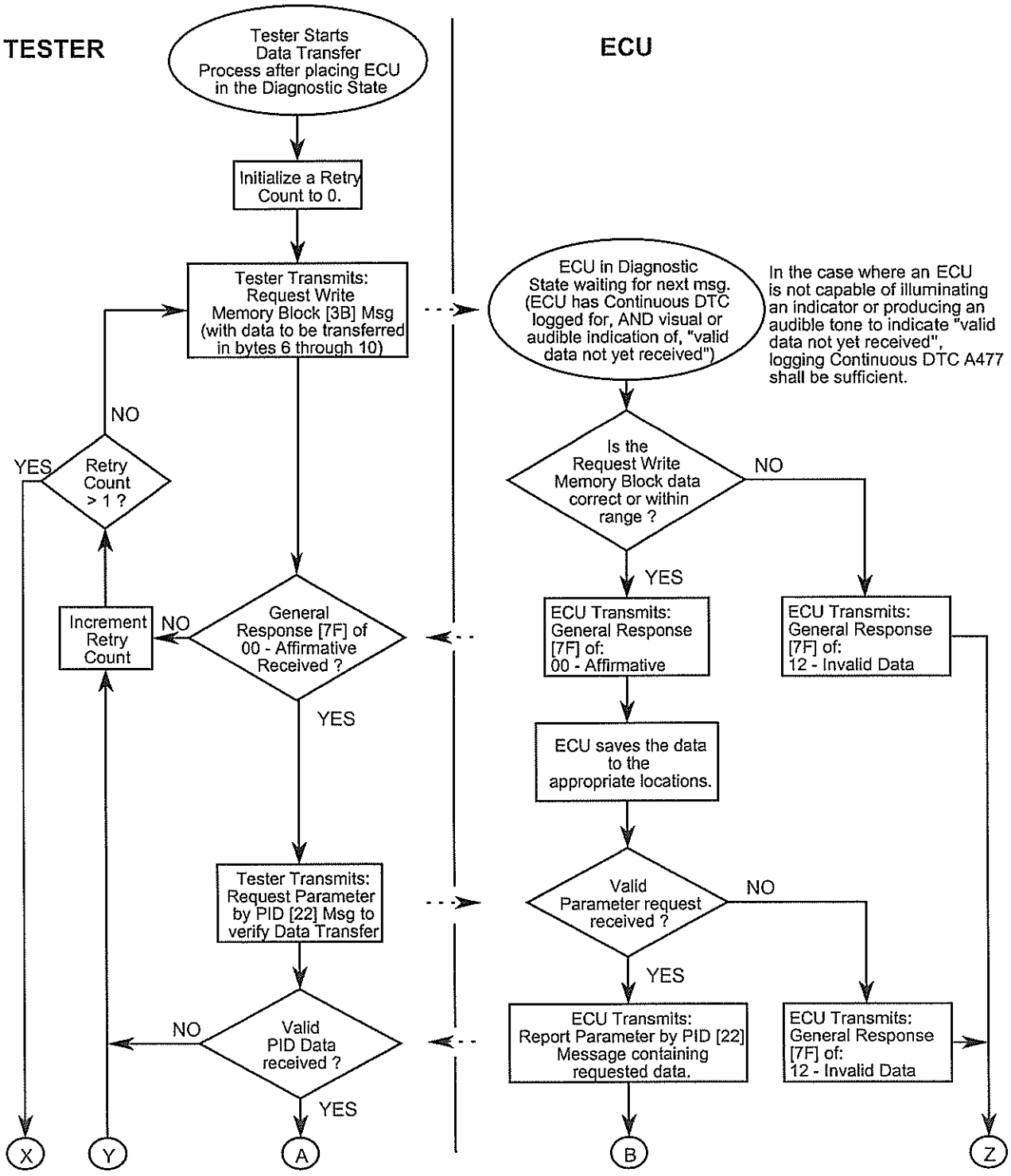


Figure 11-2 Method 2 Download Procedure (Sheet 1 of 2)

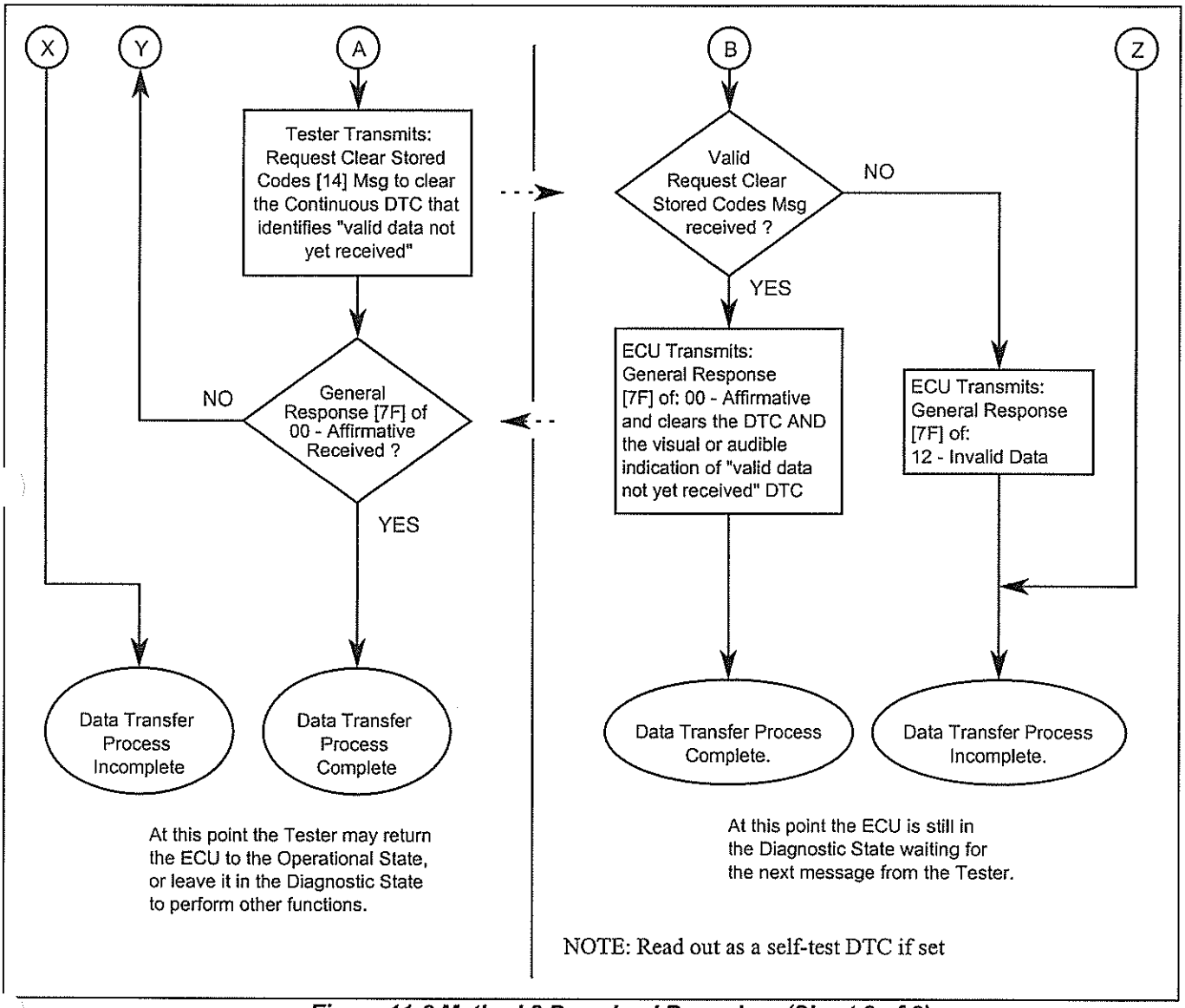


Figure 11-2 Method 2 Download Procedure (Sheet 2 of 2)

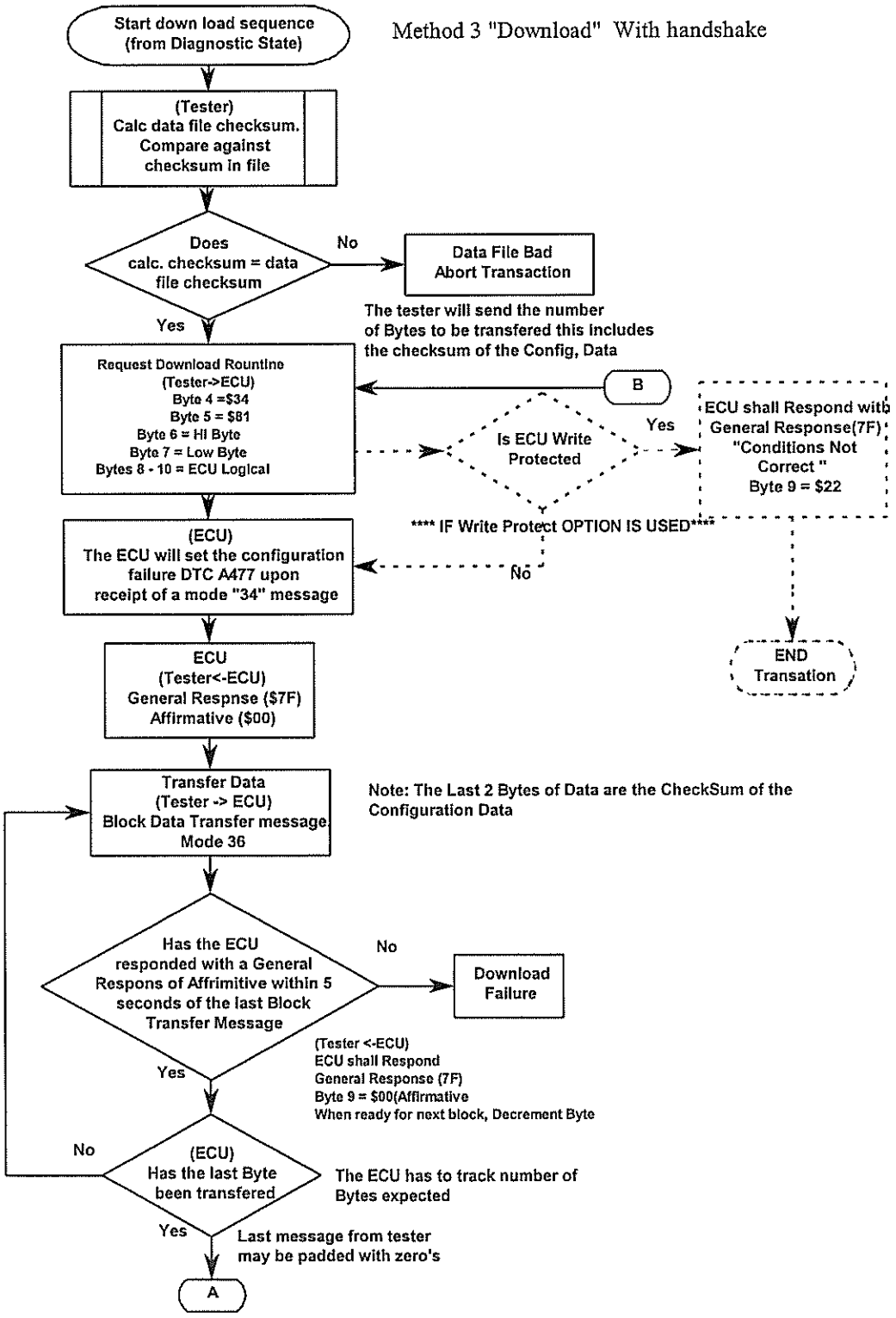


Figure 11-3 Method 3 Download Procedure With Handshake (Sheet 1 of 2)

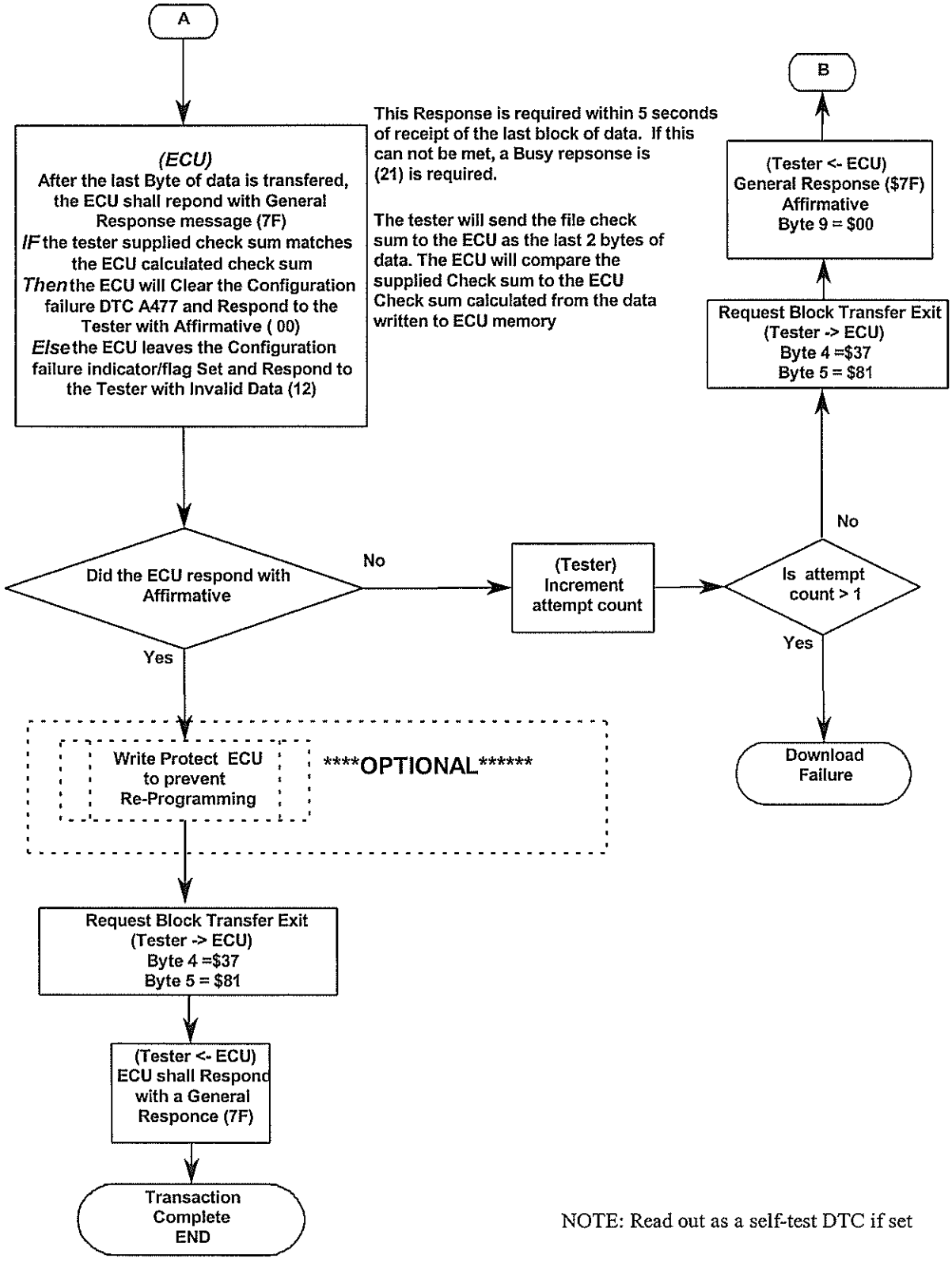


Figure 11-3 Method 3 Download Procedure With Handshake (Sheet 2 of 2)

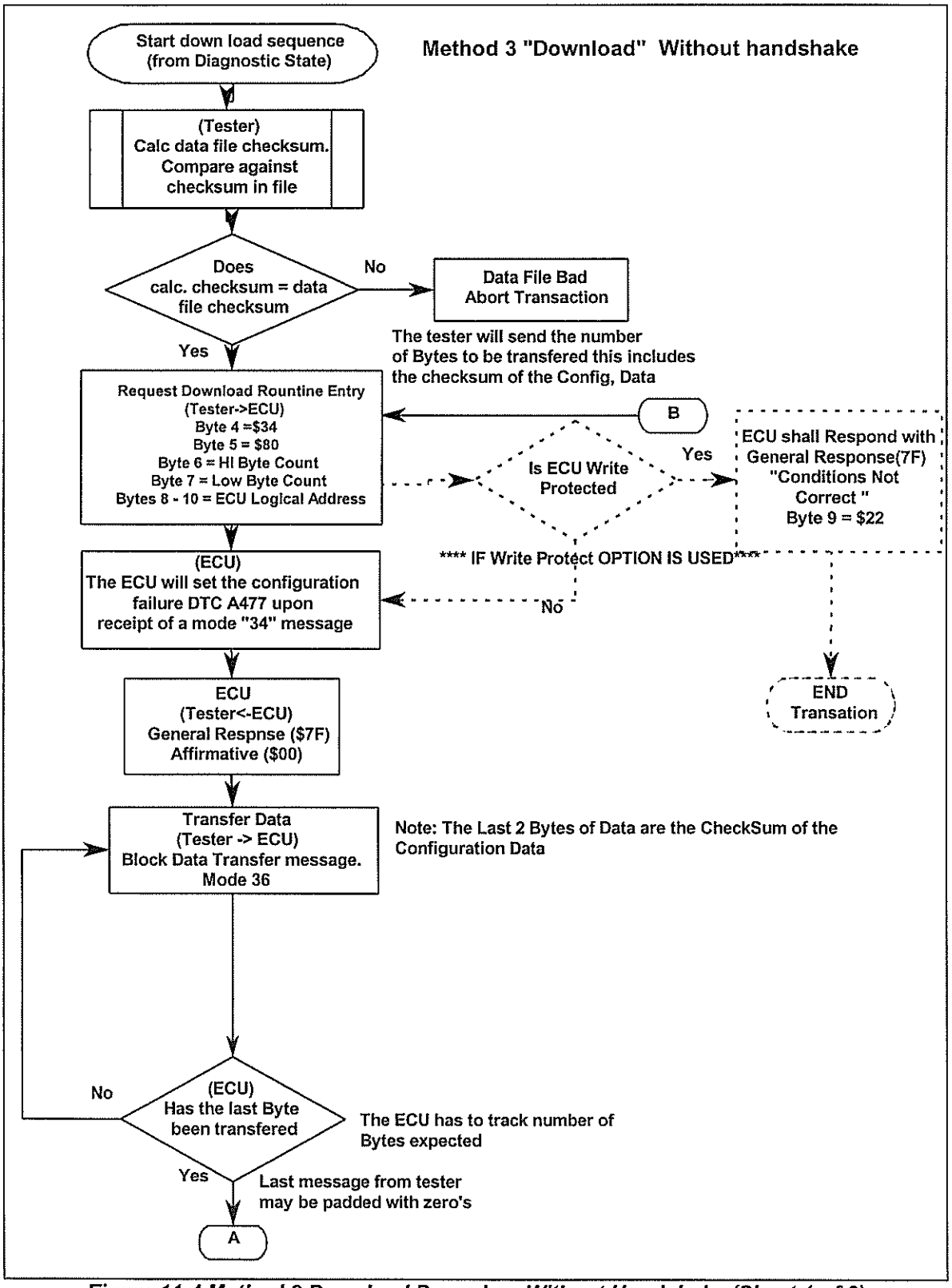


Figure 11-4 Method 3 Download Procedure Without Handshake (Sheet 1 of 2)

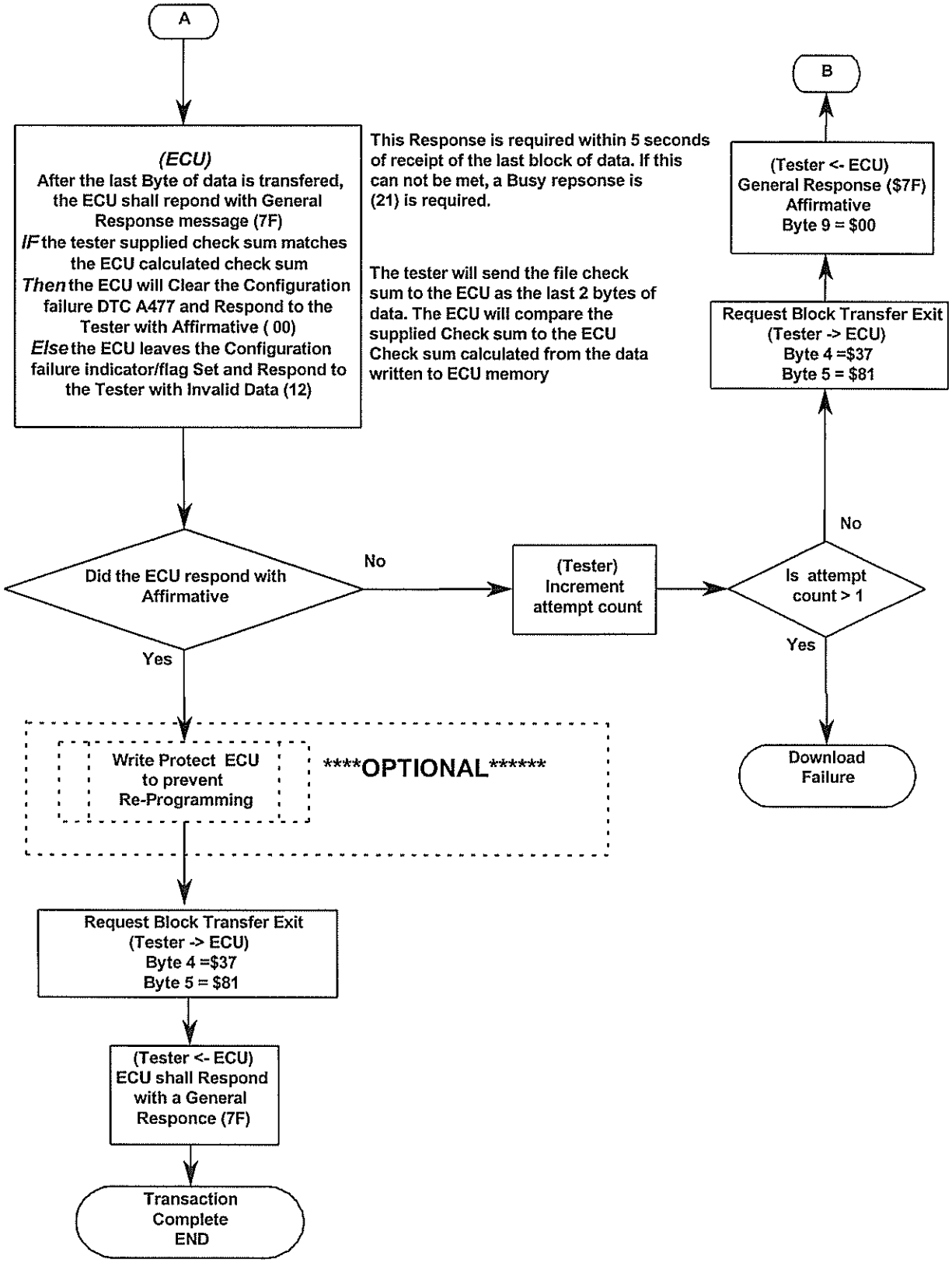


Figure 11-4 Method 3 Download Procedure Without Handshake (Sheet 2 of 2)

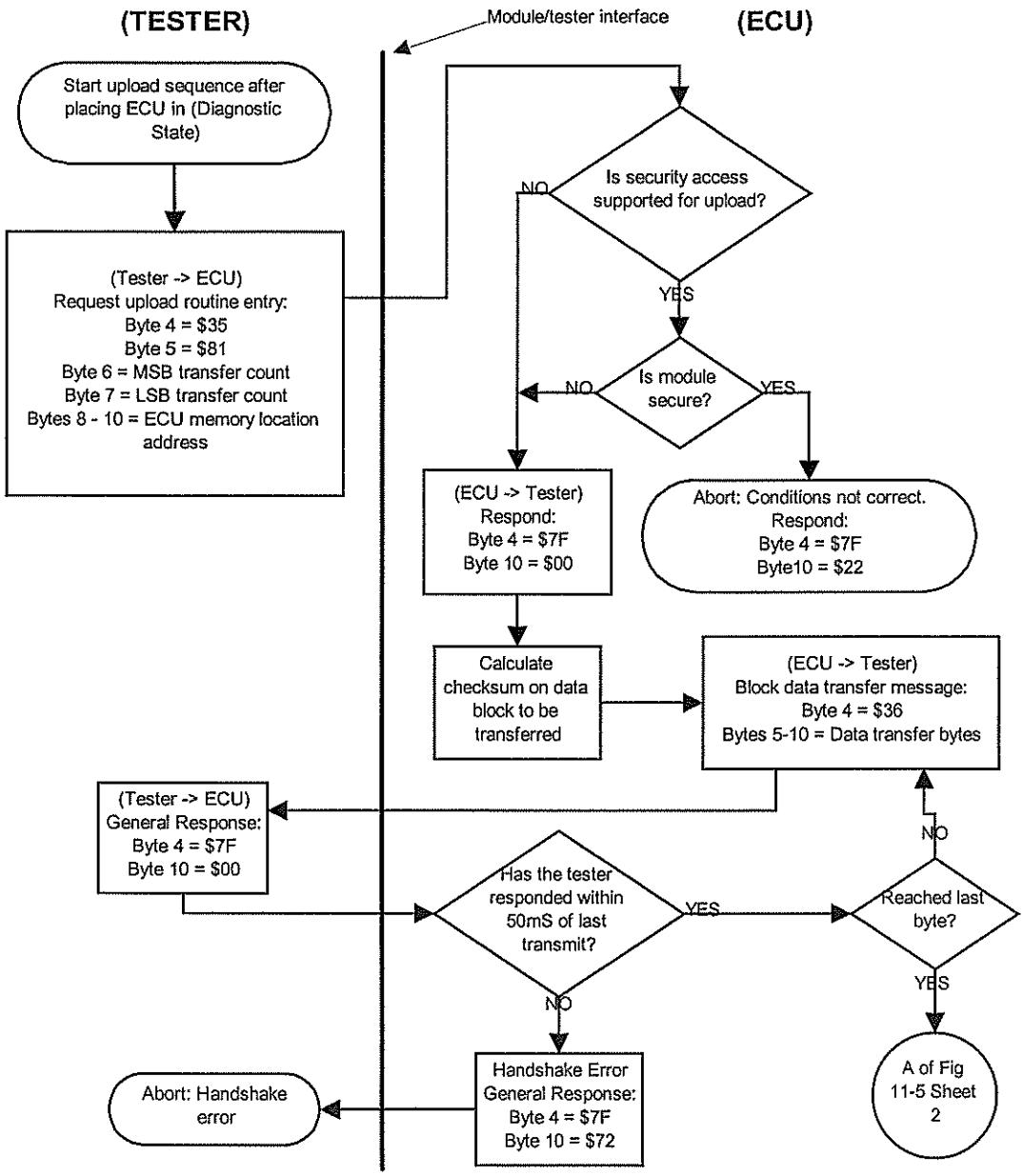


Figure 11-5 Method 5 Upload Procedure with Handshake (Sheet 1 of 2)

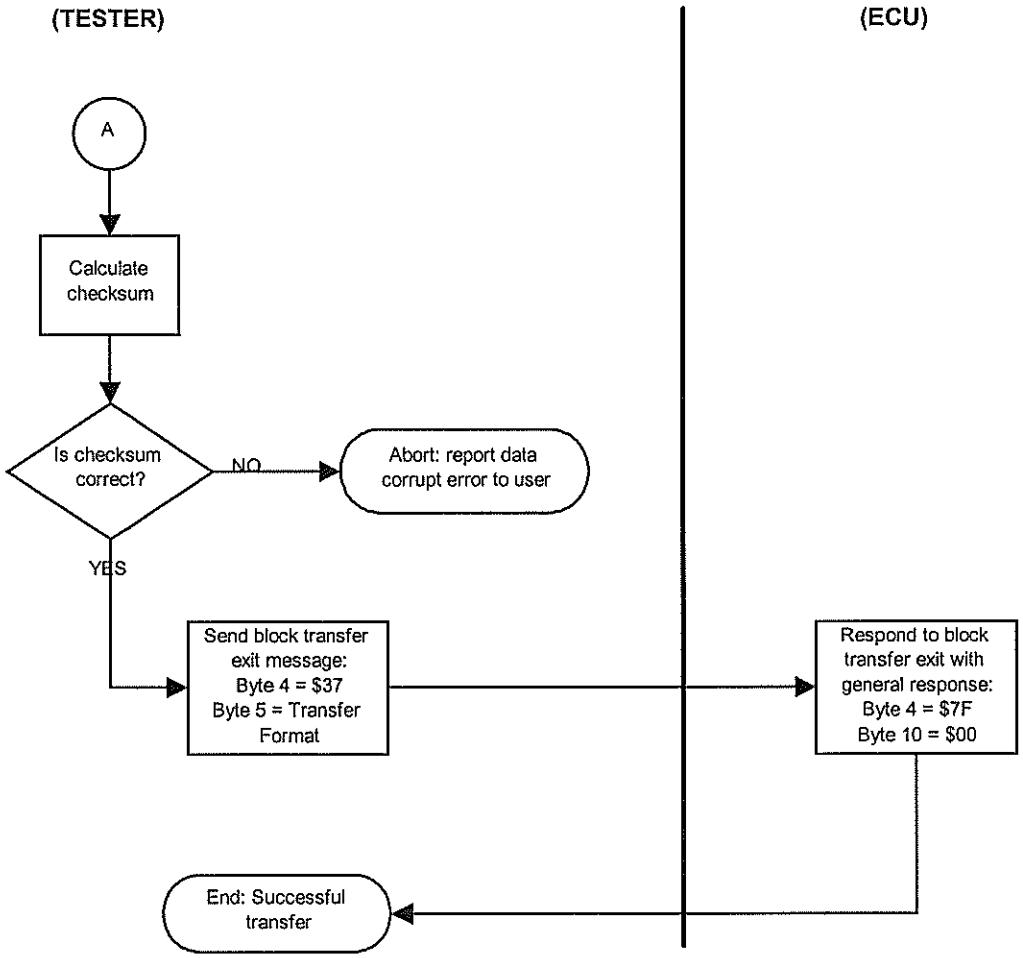


Figure 11-5 Method 5 Upload Procedure with Handshake (sheet 2 of 2)

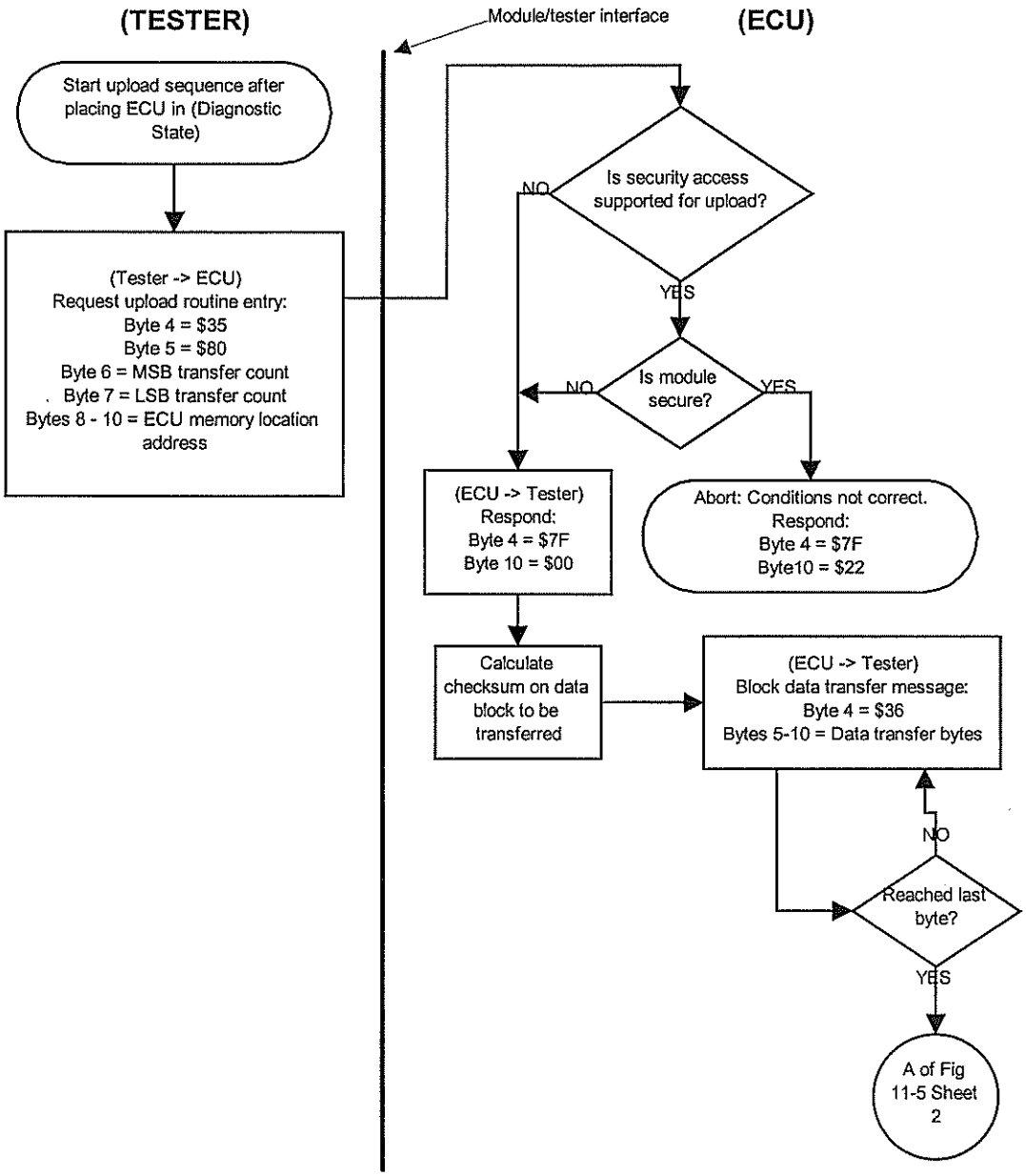


Figure 11-6 Method 5 Upload Procedure without Handshake (Sheet 1 of 2)

12. GATEWAY MODE DIAGNOSTIC REQUIREMENTS

12.1 PURPOSE

This section defines the guidelines for the implementation of diagnostics in a multi-module system (see Figure 12-1) that is comprised of a Gateway Module which supports an ISO-9141-FORD / SCP communication interface to a Tester and Satellite Modules that communicate with the Gateway Module via a non-ISO-9141-FORD / non-SCP protocol.

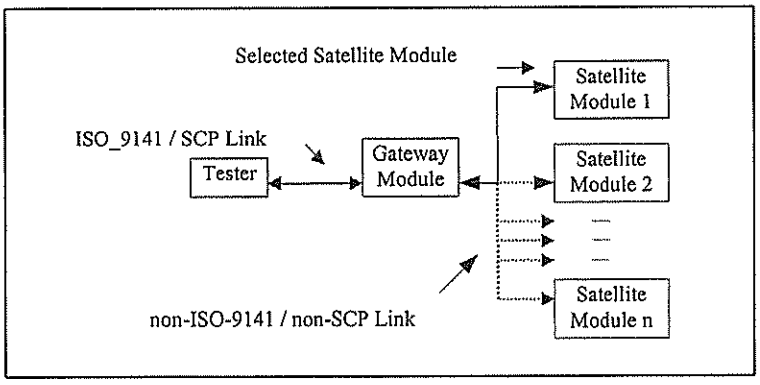


Figure 12-1 Multi-Module Gateway System

12.2 GATEWAY MODULE

While in the Operational State, Diagnostic State, Diagnostic Test Mode, Command Mode or Memory Transfer Mode (i.e., prior to being placed in a special Gateway Mode), the Gateway Module shall respond directly to all request messages directed to it from a Tester as defined in ***Appendix A; Message Structures & Responses.*** The Gateway Module is the interface through which a Tester may communicate with the Satellite Modules.

12.2.1 INITIALIZATION REQUIREMENTS

During initialization, after power-up or reset, the Gateway Module shall evaluate the interface to each of the Satellite Modules. If the Gateway Module can not communicate with one or more of the Satellite Modules during initialization, it shall log a DTC to identify this fault condition.

12.2.2 On-Demand Self Test (ODST) REQUIREMENTS

The Gateway Module shall support all of the On-Demand Self Test requirements defined in *Section 6.0, DIAGNOSTIC TEST MODE TEST REQUIREMENTS,* of this specification. In addition, during the execution of the On-Demand Self-Test, the Gateway Module shall evaluate the interface to each of the Satellite Modules. If the Gateway Module can not communicate with one or more of the Satellite Modules during the execution of its On-demand Self-Test, it shall log a DTC to identify this fault condition.

12.3 GATEWAY MODE

Gateway Mode is the operating mode that a Gateway Module is directed to enter by a Tester which allows the Tester to indirectly communicate with the Satellite Modules.

12.3.1 Entering Gateway Mode

To enter the Gateway Mode, the Gateway Module shall meet the following entry requirements.

- The Gateway Module shall be in the Diagnostic State.
- No subsystem specific conditions shall exist that prevent the Gateway Module from entering the Gateway Mode.
- The Gateway Module shall receive Diagnostic Command [B1] $0300; Gateway Mode Access, that has the following message structure.

Byte	Data	Description
1	x4	A4 - ISO-9141 Length/Type or C4 - SCP Priority/Header/Type
2	xx	Gateway Module Node ID
3	xx	Tester Source Address
4	B1	Diagnostic Command Mode
5	03	Gateway Mode Access MSB
6	00	Gateway Mode Access LSB
7	xx	Satellite Module Node ID
8	00	Provide Gateway Access
9	00	- reserved for future use -
10	00	- reserved for future use -
11	xx	ISO-9141 Checksum or SCP CRC

Upon receipt of this message, the Gateway Module shall set up a pointer to the Satellite Module specified in Byte 7 of the Diagnostic Command **[B1]** message. This Satellite Module is the target for diagnostic evaluation.

12.3.1.1 Gateway Module in Gateway Mode

After being placed in Gateway Mode, the Gateway Module shall translate request messages received from a Tester into the non-ISO-9141 / non-SCP protocol supported by the Satellite Modules and forward those requests to the Satellite Module pointed to by the Gateway Module. The Gateway Module's ISO-9141/SCP to non-ISO-9141/non-SCP translation algorithm shall be capable of translating the collective set (equivalent to a logical OR) of all the messages supported by the Satellite Modules. The Gateway Module shall also translate response messages received from a Satellite Module into the ISO-9141 / SCP protocol that it supports and returns those responses to the Tester.

12.3.1.2 Holding Gateway Mode

The Gateway Module shall support Command **$FACE, *Hold Gateway Mode.*** This Command provides the capability to keep the Gateway Module from timing out and returning to Operational State if it does not receive any supported or unsupported Node-to-Node message within 5.0-second(-0.0, +1.0) intervals. The method for handling this Command is discussed in the following subsection. NOTE: The **$FACE** Command lies within the Manufacturer Specific Command range and shall NOT be supported by any service tools.

12.3.1.3 Gateway Mode Message Handling

Once the Gateway Module is in the Gateway Mode, the following rules for Tester-Gateway-Satellite communication apply (refer to Table 12-1 for an overview of the information presented below).

Messages: Tester to Gateway

- All request messages received by the Gateway Module from a Tester, that are formatted correctly (less Diagnostic Command [B1]: $0300; Gateway Mode Access) shall be translated to the non-ISO-9141 / non-

12.3.1 Entering Gateway Mode

To enter the Gateway Mode, the Gateway Module shall meet the following entry requirements.

- The Gateway Module shall be in the Diagnostic State.
- No subsystem specific conditions shall exist that prevent the Gateway Module from entering the Gateway Mode.
- The Gateway Module shall receive Diagnostic Command [B1] $0300; Gateway Mode Access, that has the following message structure.

Byte	Data	Description
1	x4	A4 - ISO-9141 Length/Type or C4 - SCP Priority/Header/Type
2	xx	Gateway Module Node ID
3	xx	Tester Source Address
4	B1	Diagnostic Command Mode
5	03	Gateway Mode Access MSB
6	00	Gateway Mode Access LSB
7	xx	Satellite Module Node ID
8	00	Provide Gateway Access
9	00	- reserved for future use -
10	00	- reserved for future use -
11	xx	ISO-9141 Checksum or SCP CRC

Upon receipt of this message, the Gateway Module shall set up a pointer to the Satellite Module specified in Byte 7 of the Diagnostic Command **[B1]** message. This Satellite Module is the target for diagnostic evaluation.

12.3.1.1 Gateway Module in Gateway Mode

After being placed in Gateway Mode, the Gateway Module shall translate request messages received from a Tester into the non-ISO-9141 / non-SCP protocol supported by the Satellite Modules and forward those requests to the Satellite Module pointed to by the Gateway Module. The Gateway Module's ISO-9141/SCP to non-ISO-9141/non-SCP translation algorithm shall be capable of translating the collective set (equivalent to a logical OR) of all the messages supported by the Satellite Modules. The Gateway Module shall also translate response messages received from a Satellite Module into the ISO-9141 / SCP protocol that it supports and returns those responses to the Tester.

12.3.1.2 Holding Gateway Mode

The Gateway Module shall support Command **$FACE, *Hold Gateway Mode*.** This Command provides the capability to keep the Gateway Module from timing out and returning to Operational State if it does not receive any supported or unsupported Node-to-Node message within 5.0-second(-0.0, +1.0) intervals. The method for handling this Command is discussed in the following subsection. NOTE: The **$FACE** Command lies within the Manufacturer Specific Command range and shall NOT be supported by any service tools.

12.3.1.3 Gateway Mode Message Handling

Once the Gateway Module is in the Gateway Mode, the following rules for Tester-Gateway-Satellite communication apply (refer to Table 12-1 for an overview of the information presented below).

Messages: Tester to Gateway

- All request messages received by the Gateway Module from a Tester, that are formatted correctly (less Diagnostic Command [B1]: $0300; Gateway Mode Access) shall be translated to the non-ISO-9141 / non-

SCP protocol supported by the Satellite Modules and forward to the Satellite Module pointed to the Gateway Module.

- If the Gateway Module receives Diagnostic Command [B1]: $0300; Gateway Mode Access, with Byte 8 = $00, it shall return a General Response [7F] of: $00; Affirmative, and switch its pointer to the Satellite Module identified in Byte 7 of the message.
- If the Gateway Module receives Diagnostic Command [B1]: $0300; Gateway Mode Access, with Byte 8 = $10 it shall return a General Response [7F] of: $00; Affirmative, and exit the Gateway Mode to the Diagnostic State. Note: Prior to existing the Gateway Mode, the Tester shall return the Satellite Module to the Operational State.
- If the Gateway Module receives Diagnostic Command [B1]: $0300; Gateway Mode Access, with Byte 8 = $20, it shall return a General Response [7F] of: $00; Affirmative, and exit the Gateway Mode to the Operational State. Note: Prior to existing the Gateway Mode, the Tester shall return the Satellite Module to the Operational State.

Upon returning to the Operational State, the Gateway Module shall have 500 milliseconds to reinitialize. During this time period, the Gateway Module is not required to respond to requests received from a Tester.

- If the Gateway Module receives Diagnostic Command [B1]: $0300; Gateway Mode Access, with Byte 8 not equal to $00, $10 or $20, it shall return a General Response [7F] of: $12; Invalid Data.
- If the Gateway Module receives Diagnostic Command [B1]: $0300; Gateway Mode Access, with Byte 7 containing a value that is not a recognized Satellite Module address, it shall return a General Response [7F] of: $12; Invalid Data.
- If the Gateway Module receives Diagnostic Command [B1]: $FACE - Hold Gateway Mode while in the Gateway Mode, it shall return a General Response [7F] of: 00 - Affirmative, and inhibit the time-out function that causes the Gateway Module to return to the Operational State if it does not receive any supported or unsupported Node-to-Node message within 5.0-second (-0.0, +1.0) intervals. Upon receiving this Command a second time, the Gateway Module shall re-establish the time-out function so that it may return to the Operational State if it does not receive any supported or unsupported Node-to-Node message within 5.0-second (-0.0, +1.0) intervals.
- If the Gateway Module receives Diagnostic Command [B1]: $FACE - Hold Gateway Mode while NOT in the Gateway Mode, the Gateway Module shall respond with a General Response [7F] of: 22 - Conditions Not Correct.
- If the Gateway Module receives a Request Message (Mode) that is not supported in the translation algorithm of the Gateway Module, it shall return a General Response [7F] of: 11 - Mode Not Supported.

Messages: Gateway to Satellite

- If a Satellite Module receives a translated request message (mode) from the Gateway Module, that is supported by the Satellite Module and formatted correctly, it shall formulate a proper response and return it to the Gateway Module.
- If a Satellite Module receives a translated request message (mode) from the Gateway Module, that represents a mode that is not supported by the Satellite Module, the Satellite Module shall formulate and return a response to the Gateway Module that the Gateway Module can translate into a General Response [7F] of: 11 - Mode Not Supported.
- If a Satellite Module receives a translated request message (mode) from the Gateway Module, that contains invalid data, the Satellite Module shall formulate and return a response to the Gateway Module that the Gateway Module can translate into a General Response [7F] of: 12 - Invalid Data.

Messages: Satellite to Gateway

- All response messages received by the Gateway Module from a Satellite Module, that are formatted correctly shall be translated to the ISO-9141 / SCP protocol supported by the Gateway Module and returned to the Tester.

- Any response received by the Gateway Module from a Satellite Module that is not currently being pointed to by the Gateway Module shall be ignored by the Gateway Module.
- Any response received by the Gateway Module from a Satellite Module that identifies a Mode that does not correspond to the request sent to the Satellite Module shall be ignored by the Gateway Module.
- Any response received by the Gateway Module from a Satellite Module that is not formatted correctly shall be ignored by the Gateway Module.

Messages: Gateway to Tester

- All response messages received by the Gateway Module from a Satellite Module, that are formatted correctly shall be translated to the ISO-9141 / SCP protocol supported by the Gateway Module and returned to the Tester.

12.3.2 Exiting Gateway Mode to Diagnostic State

The Gateway Module shall exit the Gateway Mode and return to the Diagnostic State if:

- It receives the following Diagnostic Command [B1] $0300; Gateway Mode Access, that instructs it to return to the Diagnostic State (see Byte 8).

Byte	Data	Description
1	x4	A4 - ISO-9141 Length/Type or C4 - SCP Pri./Header/Type
2	xx	Gateway Module ID
3	xx	Tester Source Address
4	B1	Diagnostic Command Mode
5	03	Gateway Mode Access MSB
6	00	Gateway Mode Access LSB
7	yy	Gateway Module ID
8	10	Return to Diagnostic State
9	00	- reserved for future use -
10	00	- reserved for future use -
11	xx	ISO-9141 Checksum or SCP CRC

12.3.3 Exiting Gateway Mode to Operational State

The Gateway Module shall exit the Gateway Mode and return to the Operational State if:

- The ECU doesn't receive ANY Node-to-Node message (supported or unsupported) within 5.0 (-0.0, +1.0) seconds of either the last message received or initial entrance into gateway mode and the Gateway Hold Mode (**$FACE**) is not enabled.
- It meets any exit requirements specified in the SSDS (Part II) for the Gateway Module.
- It receives the following Diagnostic Command [B1] $0300; Gateway Mode Access, that instructs it to return to the Operational State (see Byte 8).

Byte	Data	Description
1	x4	A4 - ISO-9141 Length/Type or C4 - SCP Pri./Header/Type
2	xx	Gateway Module ID
3	xx	Tester Source Address
4	B1	Diagnostic Command Mode
5	03	Gateway Mode Access MSB
6	00	Gateway Mode Access LSB
7	yy	Gateway Module ID
8	20	Return to Operational State

9	00	- reserved for future use -
10	00	- reserved for future use -
11	xx	ISO-9141 Checksum or SCP CRC

12.4 GATEWAY MODE TIMING REQUIREMENTS

The Response Time between the trailing edge of the last bit of a Request Message issued by a Tester and the leading edge of the Response Message returned to the Tester shall not exceed 275 milliseconds; the Tester shall wait 300 milliseconds.

The time required for the Gateway Module to translate an ISO-9141 / SCP message and begin re-transmitting a non-ISO-9141 / non-SCP message shall not exceed 20 milliseconds.

Likewise, the time required for the Gateway Module to translate a non-ISO-9141 / non-SCP message and begin re-transmitting an ISO-9141 / SCP message also shall not exceed 20 milliseconds.

12.5 SATELLITE MODULES

Satellite Modules shall adhere to all requirements specified within this specification. Satellite Modules are the indirect target of Tester Request Messages when the Gateway Module is in the Gateway Mode. Note: All Tester Messages are sent to the Gateway Module address.

12.5.1 Satellite Module In Diagnostic State

A Satellite Module shall remain in Diagnostic State as long as it receives any supported or unsupported Node-to-Node message within 5.0- second (-0.0 + 1.0) intervals or until it is instructed to exit.

12.5.2 Satellite Module Mandatory PIDs

In addition to supporting the mandatory PIDs defined in ***Section 8.0; PARAMETER IDENTIFIER (PID) REQUIREMENTS,*** of this specification, all Satellite Modules shall support PID **$D12D; Node Address.**

12.6 REDIRECTING COMMUNICATION

To redirect communication to another Satellite Module when the Gateway Module is already in Gateway Mode, the Tester shall send Diagnostic Command **[B1] $0300 - *Gateway Mode Access,*** with the new Satellite Module Node ID specified in Byte 7 of the message and Byte 8 = $00.

Table 12-1 Gateway Mode Message Handling

TESTER		GATEWAY MODULE		SATELLITE MODULE	COMMENTS
Diagnostic Request Message (sent prior to placing the Gateway Module in the Gateway Mode)	→				A Request sent by the Tester to the Gateway Module prior to placing it in the Gateway Mode is defined as being intended for the Gateway Module. The Gateway Module shall respond to the Request as defined in Appendix A of this document.
	←	Process request and respond as defined in Appendix A - Message Structures & Responses of this document.			
Diagnostic Command [B1]: $0300 - Gateway Mode Access (Byte 8 = $00: Enter Gateway Mode)	→	Issue General Response [7F] of: 00 - Affirmative then			Upon receipt of the Gateway Mode Access command that has Byte 8 = $00 - Enter Gateway Mode , the Gateway Module responds with a General Response [7F] of: 00 - Affirmative & enters Gateway Mode.
	←	Enter Gateway Mode			
Diagnostic Request Message (Mode supported by targeted Satellite Module and all data contained in the message is valid)	→	Translate to non-ISO-9141 / non-SCP Satellite protocol and forward request to Satellite Module.	→		A Request is sent by the Tester that can be translated by the Gateway Module and transferred to the Satellite Module. The Satellite Module then formulates the response and returns it to the Gateway Module
	←	Translate to ISO-9141 / SCP Gateway protocol and return response to Tester.	←	Process translated request and Report Response	
Diagnostic Request Message (Mode supported by targeted Satellite Module and all data contained in the message is valid)	→	Translate to non-ISO-9141 / non-SCP Satellite protocol and forward request to Satellite Module.	→		An invalid Response returned by the Targeted Satellite Module is ignored by the Gateway Module.
		Ignore, Do Not Translate or Forward.	←	Process translated request and Report Invalid Response	
Diagnostic Request Message (Mode supported by targeted Satellite Module and all data contained in the message is valid)	→	Translate to non-ISO-9141 / non-SCP Satellite protocol and forward request to Satellite Module.	→		A valid Response returned by a non-Targeted Satellite Module is ignored by the Gateway Module.
		Ignore, Do Not Translate or Forward.	←	Response from non-Targeted Satellite Module	
Diagnostic Request Message (Mode Not Supported by targeted Satellite Module)	→	Translate to non-ISO-9141 / non-SCP Satellite protocol and forward request to Satellite Module.	→		The Request Message (Mode) received by the Gateway Module is translated and passed on to the Satellite Module, but the Satellite Module does not support the message.
	←	Translate to ISO-9141 / SCP Gateway protocol and return General Response [7F] of: 11 - Mode Not Supported .	←	Response of "Mode Not Supported" from Target Satellite Module	
Diagnostic Request Message (Mode supported by the targeted Satellite Module but contain Invalid Data)	→	Translate to non-ISO-9141 / non-SCP Satellite protocol and forward request to Satellite Module.	→		The translated Request Message received by the Satellite Module contains data that the Satellite Module can not interpret.
	←	Translate to ISO-9141 / SCP Gateway protocol and return General Response [7F] of: 12 - Invalid Data.	←	Response of "Invalid Data" from Target Satellite Module	
Diagnostic Request Message (Mode Not contained in Gateway Module translation table - Not Supported by any Satellite Module)	→				The Request Message (Mode) received by the Gateway Module is not in its translation table of messages supported by the Satellite Modules.
	←	Return General Response [7F] of: 11 - Mode Not Supported			
Diagnostic Command [B1]: $0300 - Gateway Mode Access (Byte 7 = New Satellite Module ID) (Byte 8 = 00 Enter Gateway Mode)	→				Upon receiving the Gateway Mode Access command with a New Satellite Module ID, the Gateway Module switches its pointer to direct further messages to the new Satellite Module.
	←	Return a General Response [7F] of: 00 - Affirmative then Update Satellite Pointer			
Diagnostic Command [B1]: $0300 - Gateway Mode Access (Byte 8 = $10: Exit to Diagnostic State)	→	Return a General Response [7F] of: 00 - Affirmative then Return to Diagnostic State			Upon receiving the Gateway Mode Access command to return to the Diagnostic State, Gateway Module returns to its Diagnostic State.
	←				
Diagnostic Command [B1]: $0300 - Gateway Mode Access (Byte 8 = $20: Exit to Operational State)	→	Return a General Response [7F] of: 00 - Affirmative then Return to Operational State			Upon receiving the Gateway Mode Access command to return to the Operational State, Gateway Module returns to the Operational State.
	←				